



**Escola Tècnica Superior d'Enginyeria Informàtica Universitat
Politécnica de València**

-STICKPRINT

Grado en Ingeniería Industrial y Robótica

Integrantes del Grupo:

**Francisco Lorente Ruzickova
Jesús Monchales Domingo
María Belén Marca Ábalos
Andrea Quintanilla Torrez
Rida Ballaoui Bouddih**

Fecha: 15 de mayo de 2026

Contacto: florruz@upv.edu.es

TABLA DE CONTENIDOS

1. Resumen	9
2. Introducción	10
2.1. Contexto del proyecto	10
2.2. Rol del grupo de trabajo	11
2.3. Justificación de la automatización	12
2.4. Descripción detallada del proceso a automatizar	13
3. Propuesta de Automatización	15
3.1 Análisis de alternativas	15
3.1.1 Alternativa 1: Mantenimiento del proceso manual	16
3.1.2 Alternativa 2: Automatización parcial del proceso	16
3.1.3 Alternativa 3: Automatización completa mediante robótica industrial y comunicaciones IoT	17
3.1.4 Justificación de la alternativa seleccionada	18
3.2 Objetivos específicos	18
3.2.1 Automatizar la clasificación de los palillos	18
3.2.2 Automatizar la manipulación y distribución de las piezas	19
3.2.3 Optimizar el proceso de serigrafía	19
3.2.4 Integrar todos los dispositivos mediante una arquitectura de comunicación común	19
3.2.5 Implementar un sistema de supervisión basado en IoT	19
3.2.6 Desarrollar un sistema de almacenamiento y gestión de datos	20
3.2.7 Incrementar la productividad de la línea de producción	20
3.2.8 Reducir la presencia de errores humanos	20
3.2.9 Mejorar la seguridad y las condiciones de trabajo	20
3.2.10 Validar la solución mediante simulación virtual	20
3.3 Funcionamiento del sistema tras la automatización	21
3.3.1 Alimentación de los palillos	21
3.3.2 Sistema de inspección y clasificación	21



3.3.3 Manipulación mediante robot Delta ABB IRB 360-1/800 4D	22
3.3.4 Gestión de piezas defectuosas	22
3.3.5 Estación de serigrafía robotizada	23
3.3.6 Sistema de comunicación MQTT	23
3.3.7 Sistema de control basado en ESP32-S3	24
3.3.8 Gestión de datos y base de datos	25
3.3.9 Salida y almacenamiento del producto final	25
3.3.10 Simulación integral en RoboDK	26
3.4 Criterios y metas cuantificables (KPIs)	26
3.4.1 Tiempo medio de producción	26
3.4.2 Productividad de la línea	27
3.4.3 Gestión de piezas defectuosas	29
3.4.5 Fiabilidad de las comunicaciones MQTT	29
3.4.6 Latencia de las comunicaciones MQTT	30
3.4.7 Control del contenedor de rechazo	30
3.4.8 Control de iluminación	30
3.4.9 Trazabilidad y almacenamiento de información	30
3.4.10 Resumen de KPIs	31
3.5 Mejoras	32
3.5.1 Impacto sobre los recursos humanos	32
3.5.2 Impacto sobre el proceso productivo	33
3.5.3 Mejora de la calidad del producto	33
3.5.4 Gestión de la información y trazabilidad	33
3.5.5 Supervisión y control de la instalación	34
3.5.6 Optimización de recursos materiales	34
3.5.7 Posibilidades de ampliación futura	34
3.6 Impactos Adicionales	34
3.6.1 Impacto ambiental	35
3.6.2 Impacto tecnológico	35

3.6.3 Impacto logístico.....	35
3.6.4 Impacto sobre el personal	36
3.6.5 Impacto económico	36
3.6.6 Impacto sobre el crecimiento futuro	36
3.7 Límites y restricciones del proyecto	37
3.7.1 Naturaleza simulada del proyecto	37
3.7.2 Limitaciones del sistema de visión artificial	37
3.7.3 Dependencia de las comunicaciones MQTT	37
3.7.4 Dependencia de la correcta sincronización entre sistemas	38
3.7.5 Limitaciones de los sensores utilizados	38
3.7.6 Alcance académico del proyecto	38
3.7.7 Consideraciones para una implantación industrial real	39
4. Diseño de la Solución	39
4.1 Elementos de la automatización	39
4.1.1 Tabla resumen de dispositivos.....	40
4.1.2 Tolva de alimentación y sistema de entrada.....	41
4.1.3 Sistema de visión artificial OAK-D	41
4.1.4 Robot Delta ABB IRB 360-1/800 4D	42
4.1.5 RobotiQ EPick Vacuum Gripper	42
4.1.6 Sistema de transporte.....	43
4.1.7 Robots colaborativos UR3e y estación de serigrafía	44
4.1.8 Herramienta láser de serigrafía	44
4.1.9 Sistema de control basado en ESP32-S3	45
4.1.10 Sensores y actuadores auxiliares	46
4.1.11 Vallado de seguridad	47
4.1.12 Sistema de comunicaciones MQTT	47
4.1.13 Base de datos PostgreSQL	48
4.1.14 RoboDK y Python	49
4.1.15 Diagramas de flujo.....	49



4.1.15.1 Diagrama de flujo general del proceso.....	49
4.1.15.2 Diagrama de flujo de comunicaciones MQTT	50
4.1.15.3 Diagrama de funcionamiento de la ESP32-S3	51
4.2 Distribución en Planta (Layout)	52
4.2.1 Zona de alimentación.....	53
4.2.2 Zona de clasificación	54
4.2.3 Zona de rechazo	54
4.2.4 Zona de carga de bandejas.....	54
4.2.5 Zona de serigrafía	54
4.2.6 Seguridad de la instalación	55
4.3 Estrategia de Persistencia y Datos	56
4.3.1 Utilización de buffers para la gestión de eventos.....	56
4.3.2 Programación orientada a objetos	56
4.4 Persistencia de Datos y Base de Datos Relacional	56
4.4.1 Modelo relacional utilizado	57
4.4.2 Tabla PALILLO	57
4.4.3 Tabla PRODUCCIÓN.....	58
4.4.4 Tabla ROBOT e INTERVENCION_ROBOT.....	58
4.4.5 Tabla SERIGRAFIADO.....	59
4.4.6 Tablas de clientes, proveedores, compras y stock	60
4.4.7 Funcionamiento de la persistencia desde Python	61
4.4.8 Diagrama del modelo relacional	61
4.4.9 Posibles ampliaciones futuras.....	62
4.5 Arquitectura de Integración	62
4.5.1 Componentes de la arquitectura de integración	62
4.5.2 Flujo general de comunicación.....	63
4.5.3 Topics MQTT utilizados	63
4.5.4 Mensajes intercambiados.....	64
4.5.5 Publicadores y suscriptores	65

4.5.6 Integración RoboDK, Python y MQTT.....	65
4.5.7 Integración ESP32-S3 y MQTT.....	66
4.5.8 Gestión de errores y casos especiales.....	66
4.5.9 Diagrama de arquitectura de integración.....	67
4.5.10 Justificación de la arquitectura seleccionada.....	67
4.6 Diseño eléctrico y conexionado.....	67
4.6.1 Esquema general de conexiones.....	68
4.6.2 Pantalla LCD I2C.....	69
4.6.3 Sensor ultrasónico HC-SR04.....	70
4.6.4 Sensor LDR.....	71
4.6.5 LEDs RGB indicadores.....	73
4.6.6 Pulsador de parada y reanudación.....	74
5. Gestión del Proyecto.....	75
5.1 Gestión del Proyecto.....	75
5.1.1 Herramienta de gestión utilizada.....	75
5.1.2 Fases del proyecto.....	75
5.1.3 Desglose de tareas realizadas.....	77
5.1.4 Hitos del proyecto.....	77
5.1.5 Planificación temporal.....	78
5.1.6 Recursos empleados.....	79
5.1.7 Gestión de riesgos.....	79
5.1.8 Distribución de responsabilidades.....	80
5.1.9 Participación en la Feria de Proyectos.....	81
5.1.10 Resultados de la gestión del proyecto.....	82
5.2 Listado de entregables del proyecto.....	82
6. Pruebas y Validación.....	83
6.1 Estrategia de pruebas y criterios de éxito.....	83
6.1.1 Criterios de éxito.....	84
6.2 Descripción de la simulación del proyecto de automatización.....	84

6.3 Procedimientos de validación y resultados esperados	86
6.3.1 Validación de la clasificación de piezas	86
6.3.2 Validación de la estación de serigrafía	86
6.3.3 Validación de las comunicaciones MQTT	86
6.3.4 Validación del sensor ultrasónico	87
6.3.5 Validación del sensor LDR.....	87
6.3.6 Validación de la base de datos PostgreSQL	87
6.3.7 Validación global del sistema	87
6.4 Conclusión de las pruebas.....	88
7. Costes y Beneficios	88
7.1 Presupuesto detallado.....	88
7.1.1 Equipamiento principal.....	89
7.1.2 Sistemas de seguridad industrial.....	89
7.1.3 Software y comunicaciones	90
7.1.4 Ingeniería, instalación y formación	90
7.2 Análisis coste-beneficio.....	91
7.3 Indicadores financieros del proyecto.....	93
7.4 Conclusión económica	95
8. Normativa y Regulación. Seguridad	96
8.1 Impacto en los puestos de trabajo	96
8.2 Consideraciones de seguridad	96
8.3 Cumplimiento de regulaciones y estándares	97
8.4 Conclusión	99
9. Desarrollo de Software y Algoritmos.....	99
9.1 Listado tecnológico.....	99
9.2 Arquitectura software y estructura general de los programas	101
9.3 Descripción de la implementación	104
9.3.1 Módulos de simulación y control robótico	104
9.3.2 Módulos de integración y comunicaciones	109

9.3.3 Módulos de almacenamiento de información	110
9.3.4 Software de la ESP32-S3	110
9.3.5 Interacción entre componentes.....	113
9.4 Algoritmos de programación avanzada y estructuras de datos	113
9.4.1 Buffer circular en la ESP32-S3.....	113
9.4.2 Programación multitarea con FreeRTOS.....	114
9.4.3 Gestión de eventos mediante MQTT	114
9.4.4 Máquina de estados distribuida.....	115
9.4.5 Listas y colecciones en Python	115
9.4.6 Justificación de interés	115
9.5 Esquema de la base de datos utilizada	115
9.6 Repositorio de código en GitHub	117
10. Conclusiones y recomendaciones.....	118
10.1 Conclusiones	118
10.2 Lecciones aprendidas	121
10.3 Recomendaciones y líneas futuras de trabajo.....	121
11. Referencias.....	122
12. Anexos	123
12.1 Manual de instalación y funcionamiento	123
12.1.1 Requisitos previos	123
12.1.2 Instalación de RoboDK.....	126
12.1.3 Instalación de Python.....	127
12.1.4 Instalación de PostgreSQL y pgAdmin	129
12.1.5 Instalación y configuración de MQTTX	132
12.1.6 Instalación y configuración de Arduino IDE y la ESP32-S3.....	134
12.1.7 Configuración del proyecto.....	137
12.1.8 Puesta en marcha y ejecución del sistema	141
12.1.9 Uso de la aplicación desarrollada	143
12.1.10 Reinicio y parada del sistema	146



12.1.11 Resolución de incidencias frecuentes.....	148
12.2 Manual del programador	151
12.2.1 Arquitectura general del software	152
12.2.2 Estructura de directorios del proyecto	153
12.2.3 Estructura interna de los módulos software.....	155
12.2.4 Firmware de la ESP32-S3.....	158
12.2.4.1 Estructura general del firmware	158
12.2.4.2 Archivo de configuración (Config.h)	159
12.2.4.3 Gestión de sensores.....	162
12.2.4.4 Gestión de actuadores.....	165
12.2.4.5 Gestión del pulsador	168
12.2.5 Base de datos PostgreSQL	170
12.2.6 Sistema MQTT	174

1. Resumen

Nuestro proyecto consiste en el diseño e implementación de un sistema de automatización industrial que conecta distintos dispositivos y programas para mejorar el proceso de serigrafía de palillos de madera para helados. Para ello, utilizamos una ESP32-S3, programación en Python, simulaciones robóticas en RoboDK y una base de datos relacional, además de la programación en Arduino IDE para la ESP32-S3.

La comunicación entre todos los componentes se realiza mediante el protocolo MQTT, lo que permite que la información se envíe y reciba en tiempo real. La ESP32-S3 se encarga de controlar eventos físicos y funciones de emergencia, mientras que Python coordina el sistema, actualiza la base de datos y controla la simulación en RoboDK.

El proceso automatizado se divide en tres partes principales. Primero, se realiza una clasificación de los palillos utilizando visión artificial para detectar posibles defectos. Después, un robot Delta ABB IRB 360-1/800 4D se encarga de distribuir el material de manera rápida y precisa. Finalmente, unos cobots UR3e realizan el proceso de serigrafiado utilizando un laser de precisión.

El principal objetivo del proyecto es reducir los errores humanos y conseguir una producción más constante y organizada. Además, este sistema permitiría aumentar la productividad,



disminuir pérdidas y recopilar datos importantes sobre el rendimiento de la producción mediante indicadores almacenados en la base de datos.

Durante el desarrollo del proyecto, es posible que surjan algunas dificultades técnicas, como retrasos en la comunicación MQTT o problemas de sincronización entre la simulación y el sistema físico. También será necesario optimizar el código para coordinar correctamente la velocidad de los robots con la actualización de la base de datos.

Por último, agradecemos al profesorado por proporcionar los conocimientos y herramientas necesarias para llevar a cabo este proyecto.

2. Introducción

2.1. Contexto del proyecto

El proyecto se enmarca en una empresa llamada *StickPrint*, especializada en la fabricación y serigrafía de palillos de madera utilizados principalmente en el sector alimentario, especialmente en productos como helados y repostería industrial.

La empresa suministra sus productos a fabricantes de helados, empresas de restauración y distribuidores de consumibles desechables, por lo que requiere altos volúmenes de producción con una calidad constante y trazable.

Actualmente, el proceso de inspección, clasificación y serigrafía de los palillos presenta limitaciones al realizarse parcialmente de forma manual, lo que provoca variaciones en la calidad, menor velocidad de producción y falta de control detallado del proceso. Por ello, la empresa necesita modernizar su línea de producción mediante la incorporación de sistemas automatizados que permitan mejorar la eficiencia y la precisión del marcado de los palillos.

En los últimos años, la industria manufacturera ha experimentado una transformación significativa gracias a la incorporación de tecnologías relacionadas con la Industria 4.0. La digitalización de procesos, el uso de sensores inteligentes y la integración de sistemas de control avanzados permiten optimizar la producción y mejorar la calidad de los productos fabricados.

Dentro de este contexto, StickPrint busca modernizar sus instalaciones mediante la automatización de aquellas tareas que actualmente requieren una elevada intervención humana. El proceso de clasificación y serigrafía de palillos presenta características ideales para ser automatizado, ya que se trata de operaciones repetitivas, con un alto volumen de producción y que requieren precisión constante.

La implantación de un sistema automatizado permitirá a la empresa adaptarse a futuras demandas del mercado, aumentar su capacidad productiva y mejorar la trazabilidad de los productos fabricados. Asimismo, facilitará el cumplimiento de estándares de calidad exigidos por los clientes del sector alimentario, donde la uniformidad y la fiabilidad son aspectos fundamentales.

2.2. Rol del grupo de trabajo

Nuestro grupo de trabajo actúa como un equipo externo de consultoría especializado en automatización industrial. Su función principal es analizar el proceso productivo de la empresa, identificar sus limitaciones y proponer una solución.

Además, el equipo se encarga del diseño de la arquitectura del sistema, la integración de los distintos componentes hardware y software, y la definición del flujo de comunicación entre dispositivos. Esto incluye la coordinación entre el sistema de visión artificial, los robots industriales y colaborativos, la ESP32-S3 como sistema de control físico y el servidor de base de datos.

Para llevar a cabo el desarrollo del proyecto, el grupo de trabajo ha adoptado una metodología colaborativa basada en la división de tareas según las competencias técnicas necesarias en cada fase. De esta forma, se distribuyen responsabilidades relacionadas con la programación, la simulación robótica, el diseño de la base de datos, la configuración de comunicaciones y la integración de hardware.

El equipo también realiza labores de planificación y seguimiento del proyecto, estableciendo objetivos parciales y verificando el correcto funcionamiento de cada módulo antes de proceder a su integración. Esta metodología permite detectar errores de forma temprana y garantizar que todos los componentes funcionen de manera coordinada.

Además, el grupo actúa como intermediario entre las necesidades de la empresa y las soluciones tecnológicas implementadas, evaluando continuamente el impacto de cada decisión sobre la eficiencia, el coste y la viabilidad del sistema final.

Finalmente, el grupo desarrolla la lógica de control en Python, la simulación del proceso en RoboDK y la implementación de la comunicación mediante MQTT, asegurando la correcta interacción y sincronización entre todos los elementos del sistema automatizado.

2.3. Justificación de la automatización

La transición hacia un sistema automatizado se justifica por los siguientes factores clave:

- **Aumento de la productividad:** La automatización permite mantener una velocidad de producción constante durante toda la jornada laboral, eliminando las variaciones asociadas al trabajo manual. Los robots industriales pueden operar de forma continua con elevados niveles de precisión y repetibilidad, aumentando significativamente el número de piezas procesadas por hora.

Además, la coordinación automática entre los diferentes elementos de la línea reduce los tiempos muertos y minimiza la aparición de cuellos de botella, mejorando el aprovechamiento de los recursos disponibles.

- **Control de calidad riguroso:** Los sistemas de visión artificial permiten detectar defectos que podrían pasar desapercibidos durante una inspección manual. La utilización de algoritmos de procesamiento de imágenes garantiza criterios de evaluación uniformes para todas las piezas producidas.

Gracias a esta capacidad de inspección automática, se reduce la probabilidad de que productos defectuosos lleguen al cliente final, mejorando la imagen de la empresa y reduciendo costes derivados de reclamaciones o devoluciones.

- **Trazabilidad y gestión de stock:** La integración con una base de datos relacional permite registrar automáticamente toda la información relevante del proceso productivo. Cada pieza procesada genera datos que pueden ser almacenados y consultados posteriormente para realizar análisis estadísticos o auditorías de calidad.

Esta trazabilidad facilita el control del inventario y permite conocer en todo momento el estado de la producción, el número de piezas válidas y defectuosas, así como el rendimiento de cada estación de trabajo.

- **Reducción de riesgos laborales:** La manipulación continua de pequeñas piezas puede provocar fatiga física y lesiones derivadas de movimientos repetitivos. La incorporación de robots industriales y colaborativos elimina gran parte de estas tareas, mejorando las condiciones ergonómicas de los operarios.

Además, los sistemas automatizados reducen la exposición de los trabajadores a zonas de riesgo y contribuyen a crear un entorno laboral más seguro y eficiente.

- **Optimización de recursos y reducción de costes:** La detección temprana de defectos permite evitar que piezas incorrectas continúen avanzando por la línea de producción, reduciendo el desperdicio de material y energía. Del mismo modo, la automatización disminuye los costes asociados a reprocesos y correcciones posteriores.

A largo plazo, la inversión inicial en automatización se traduce en una reducción significativa de costes operativos y un aumento de la rentabilidad de la empresa.

2.4. Descripción detallada del proceso a automatizar

Antes de la implantación de la propuesta de automatización, el proceso de serigrafía de los palillos se realizaba mediante maquinaria específica destinada al marcado de piezas de madera. Este tipo de equipos requiere normalmente la intervención continua de operarios encargados de alimentar la máquina, posicionar correctamente los palillos y supervisar el proceso de impresión.

El funcionamiento habitual consiste en introducir manualmente los palillos en una zona de trabajo donde la máquina realiza el grabado o serigrafía del diseño solicitado. Una vez completada la operación, los operarios retiran las piezas procesadas y preparan un nuevo lote para continuar la producción.

Este método presenta diversas limitaciones. En primer lugar, la velocidad de producción depende en gran medida de la capacidad de trabajo de los operarios, pudiendo producirse interrupciones o variaciones en el ritmo de fabricación. Además, la manipulación manual de los palillos aumenta la probabilidad de errores de posicionamiento, lo que puede provocar defectos en el grabado o la necesidad de repetir determinadas operaciones.

Otro inconveniente importante es la realización de tareas repetitivas por parte de los trabajadores, especialmente durante la alimentación continua de la máquina de serigrafía. Estas actividades pueden generar fatiga, disminuir la productividad y aumentar el riesgo de errores humanos a medida que transcurre la jornada laboral.

Asimismo, la supervisión manual dificulta la obtención de datos precisos sobre la producción y el control de calidad, ya que gran parte de la información depende de registros realizados por los propios operarios. Como consecuencia, resulta más complicado conocer en tiempo real el estado de la producción o detectar incidencias de forma temprana.

Debido a estas limitaciones, surge la necesidad de desarrollar una solución automatizada capaz de realizar las tareas de clasificación, manipulación y serigrafía de forma autónoma, manteniendo una calidad constante, reduciendo la intervención humana y aumentando la eficiencia global del proceso productivo.

El flujo de trabajo diseñado comprende las siguientes etapas secuenciales:

El proceso productivo objeto de este proyecto corresponde a la fabricación y serigrafía de palillos de madera destinados principalmente al sector alimentario. Estos productos requieren un elevado nivel de calidad debido a que serán utilizados en contacto directo con alimentos, por lo que resulta imprescindible garantizar que todas las unidades fabricadas cumplan los estándares establecidos por la empresa.

El proceso comienza con la recepción y alimentación de los palillos de madera. Los palillos son almacenados inicialmente en una tolva de alimentación que permite suministrar de forma continua el material necesario para mantener la producción. Desde esta tolva, los palillos son introducidos progresivamente en la línea de producción, donde avanzan agrupados hacia las siguientes etapas del proceso.

Una vez incorporados a la línea, los palillos pasan por una fase de inspección de calidad. Durante esta etapa se verifica que las piezas presentan las características adecuadas para continuar el proceso productivo. La inspección tiene como objetivo detectar posibles defectos que puedan comprometer la calidad del producto final, tales como grietas, roturas, deformaciones, irregularidades superficiales o desviaciones en las dimensiones de los palillos.

Tras la inspección, se realiza una clasificación de las piezas en función de su estado. Los palillos que cumplen los criterios de calidad son considerados aptos para continuar el proceso, mientras que aquellos que presentan algún tipo de defecto son separados del flujo principal de producción. Esta clasificación resulta fundamental para garantizar que únicamente los productos conformes lleguen a las fases finales de fabricación.

Los palillos defectuosos son trasladados a una zona específica de rechazo donde permanecen almacenados temporalmente para su posterior gestión. Esta separación evita que productos no conformes puedan mezclarse con los productos válidos y permite llevar un control más preciso de la calidad global de la producción.

Por otro lado, los palillos clasificados como correctos continúan avanzando hacia la etapa de serigrafía. En esta fase se realiza el marcado o personalización del producto mediante la incorporación de elementos gráficos definidos según las necesidades del cliente. Dependiendo

del pedido, el marcado puede incluir textos, logotipos, imágenes corporativas, códigos identificativos u otros diseños personalizados.

La etapa de serigrafía constituye una de las fases más importantes del proceso productivo, ya que aporta valor añadido al producto final y permite adaptar los palillos a los requisitos específicos de cada cliente. Para garantizar un resultado uniforme, es necesario mantener un posicionamiento adecuado de las piezas y asegurar una correcta ejecución del proceso de marcado.

Una vez finalizada la serigrafía, los palillos son agrupados y transportados hacia la zona final de la línea de producción. En esta etapa se realiza el almacenamiento temporal de las piezas terminadas, que quedan preparadas para su posterior manipulación, distribución o incorporación a etapas posteriores del proceso logístico de la empresa.

Todo el proceso requiere una coordinación adecuada entre las distintas fases de alimentación, inspección, clasificación, marcado y almacenamiento. Cualquier incidencia producida en una de estas etapas puede afectar al rendimiento global de la línea, provocando retrasos, pérdidas de producción o disminuciones en la calidad del producto final. Por este motivo, la empresa busca implantar una solución de automatización que permita optimizar el funcionamiento de la línea, mejorar la trazabilidad del proceso y aumentar la productividad manteniendo elevados estándares de calidad.

3. Propuesta de Automatización

3.1 Análisis de alternativas

Antes de definir la solución final propuesta para la automatización de la línea de clasificación y serigrafía de palillos de madera, se realizó un análisis de diferentes alternativas tecnológicas y organizativas. El objetivo de este estudio fue identificar la opción que mejor se adaptaba a las necesidades de la empresa StickPrint, teniendo en cuenta factores como la productividad, la calidad del producto final, la trazabilidad del proceso, la escalabilidad futura y el coste de implantación.

La automatización de un proceso industrial puede abordarse desde distintos niveles de complejidad, por lo que se consideraron varias alternativas antes de seleccionar la solución definitiva.

3.1.1 Alternativa 1: Mantenimiento del proceso manual

La primera alternativa considerada consistía en mantener el proceso productivo actual sin introducir cambios significativos en la línea de fabricación. En este escenario, las tareas de inspección, clasificación y alimentación de la máquina de serigrafía continuarían siendo realizadas por operarios.

La inspección visual de los palillos sería llevada a cabo manualmente por trabajadores encargados de identificar posibles defectos de fabricación. Posteriormente, los palillos correctos serían introducidos manualmente en la máquina de serigrafía, donde se realizaría el marcado correspondiente. Una vez finalizado el proceso, los operarios retirarían los productos terminados y prepararían nuevos lotes para continuar la producción.

Esta alternativa presenta como principal ventaja su reducido coste de implantación, ya que no requiere la adquisición de nuevos equipos ni el desarrollo de sistemas de control complejos. Además, permite aprovechar completamente la infraestructura existente sin necesidad de realizar modificaciones significativas en la planta de producción.

Sin embargo, esta solución presenta importantes limitaciones. La productividad depende directamente del ritmo de trabajo de los operarios, lo que genera variaciones en la velocidad de producción y dificulta el cumplimiento de grandes volúmenes de fabricación. Asimismo, la inspección manual está expuesta a errores humanos y a variaciones en los criterios de evaluación de la calidad.

Otro inconveniente importante es la falta de trazabilidad. La información relativa a la producción suele registrarse manualmente o, en muchos casos, no se registra de forma sistemática, dificultando el análisis posterior del rendimiento de la línea y la detección de incidencias.

Por estos motivos, se consideró que mantener el proceso completamente manual no resultaba una solución adecuada para los objetivos de modernización y mejora de la competitividad perseguidos por la empresa.

3.1.2 Alternativa 2: Automatización parcial del proceso

La segunda alternativa analizada consistía en implementar una automatización parcial, manteniendo la intervención humana en determinadas fases del proceso.

En esta propuesta, se incorporarían elementos automáticos para el transporte de los palillos y para determinadas operaciones de manipulación, mientras que la clasificación de defectos y parte del proceso de serigrafía seguirían dependiendo de operarios.

Esta solución permitiría reducir parcialmente la carga de trabajo manual y aumentar moderadamente la capacidad de producción. Además, requeriría una inversión económica inferior a la necesaria para una automatización completa.

No obstante, seguirían existiendo importantes dependencias de la actuación humana. La calidad del producto continuaría estando condicionada por la capacidad de los operarios para detectar defectos y realizar correctamente las tareas de alimentación y supervisión de la maquinaria.

Asimismo, la coexistencia de procesos manuales y automáticos dificulta la integración de sistemas de trazabilidad y monitorización en tiempo real. Como consecuencia, las mejoras obtenidas en productividad y calidad serían limitadas en comparación con una solución completamente automatizada.

Por estas razones, aunque esta alternativa supone una mejora respecto al proceso manual, no permite alcanzar plenamente los objetivos planteados para el proyecto.

3.1.3 Alternativa 3: Automatización completa mediante robótica industrial y comunicaciones IoT

La tercera alternativa estudiada consiste en la automatización integral del proceso mediante la incorporación de sistemas de visión artificial, robótica industrial, comunicaciones MQTT, dispositivos IoT y almacenamiento centralizado de información.

En esta propuesta, los palillos son alimentados automáticamente desde una tolva y transportados mediante cintas transportadoras a lo largo de las diferentes estaciones de trabajo. Un sistema de inspección determina si cada pieza cumple los requisitos de calidad establecidos.

Posteriormente, un robot industrial Delta se encarga de clasificar los palillos y distribuirlos hacia la línea de producción principal o hacia la línea de rechazo.

Los palillos válidos son enviados a una estación de serigrafía atendida por robots colaborativos, capaces de ejecutar de forma automática las operaciones de marcado sobre las piezas.

Paralelamente, toda la información generada durante el proceso es gestionada mediante comunicaciones MQTT y almacenada en una base de datos para garantizar la trazabilidad de la producción.

La principal ventaja de esta alternativa es la reducción drástica de la intervención humana en tareas repetitivas. Esto permite aumentar significativamente la productividad, mejorar la uniformidad del producto final y minimizar los errores derivados de la manipulación manual.

Además, la integración de tecnologías propias de la Industria 4.0 facilita la supervisión del proceso en tiempo real y permite disponer de información detallada sobre el estado de la producción, favoreciendo la toma de decisiones basada en datos.



Entre los inconvenientes de esta alternativa destacan la mayor complejidad técnica y la inversión inicial necesaria para desarrollar e implantar la solución. Asimismo, se requiere personal con conocimientos especializados para la programación, mantenimiento y supervisión de los sistemas automatizados.

Sin embargo, estos inconvenientes quedan compensados por los beneficios obtenidos a medio y largo plazo en términos de productividad, calidad, seguridad y capacidad de crecimiento.

3.1.4 Justificación de la alternativa seleccionada

Tras analizar las diferentes opciones disponibles, se decidió seleccionar la alternativa basada en una automatización completa del proceso productivo.

Esta solución es la que mejor responde a las necesidades actuales y futuras de la empresa StickPrint, ya que permite aumentar la capacidad de producción, mejorar la calidad de los productos fabricados y disponer de una mayor trazabilidad de la información generada durante el proceso.

Además, la utilización de sistemas de visión artificial, robótica industrial, comunicaciones MQTT y almacenamiento de datos proporciona una arquitectura flexible y escalable que puede ampliarse en el futuro mediante la incorporación de nuevas estaciones, sensores o funcionalidades sin necesidad de rediseñar completamente el sistema.

Por todo ello, la automatización integral se considera la alternativa más adecuada para alcanzar los objetivos planteados en el proyecto y garantizar la competitividad de la empresa en un entorno industrial cada vez más digitalizado.

3.2 Objetivos específicos

El objetivo general del proyecto consiste en diseñar e implementar una solución de automatización industrial para mejorar el proceso de clasificación y serigrafía de palillos de madera en la empresa StickPrint. Para alcanzar dicho objetivo general se han definido una serie de objetivos específicos que permiten estructurar y desarrollar el proyecto de forma organizada.

3.2.1 Automatizar la clasificación de los palillos

Uno de los principales objetivos del proyecto es automatizar el proceso de clasificación de los palillos de madera. Actualmente, la detección de defectos puede requerir supervisión humana, lo que incrementa el tiempo necesario para procesar cada lote y aumenta la probabilidad de errores. Mediante la automatización de esta etapa se pretende identificar



de forma rápida y consistente las piezas correctas y defectuosas, garantizando que únicamente los productos que cumplen los estándares de calidad continúen avanzando por la línea de producción.

3.2.2 Automatizar la manipulación y distribución de las piezas

Otro objetivo fundamental consiste en automatizar el transporte y la distribución de los palillos entre las distintas estaciones de trabajo. Para ello, se busca eliminar la necesidad de realizar movimientos manuales repetitivos y sustituirlos por sistemas robotizados capaces de manipular las piezas con mayor rapidez, precisión y repetibilidad. De esta forma se mejora la eficiencia global del proceso y se reducen los tiempos improductivos.

3.2.3 Optimizar el proceso de serigrafía

El proyecto pretende mejorar el proceso de marcado de los palillos mediante la incorporación de sistemas robotizados capaces de ejecutar trayectorias precisas y repetitivas. Esto permitirá obtener resultados homogéneos en todas las piezas procesadas, independientemente del volumen de producción. Además, la automatización facilitará la personalización de los productos mediante diferentes diseños, textos o logotipos según las necesidades de cada cliente.

3.2.4 Integrar todos los dispositivos mediante una arquitectura de comunicación común

Se pretende desarrollar una arquitectura de comunicación que permita la interacción coordinada entre todos los elementos que forman parte de la estación de trabajo. La utilización de protocolos de comunicación industrial facilitará el intercambio de información en tiempo real entre sensores, actuadores, robots, sistemas de control y dispositivos IoT, garantizando una sincronización adecuada de todas las operaciones.

3.2.5 Implementar un sistema de supervisión basado en IoT

Otro de los objetivos específicos consiste en desarrollar un sistema de supervisión capaz de monitorizar el estado de la instalación y proporcionar información relevante al operario. Para ello se utilizarán sensores y dispositivos electrónicos que permitan detectar eventos como paradas de emergencia, cambios en las condiciones de iluminación o el estado de llenado del contenedor de rechazo. Esta información contribuirá a mejorar la capacidad de respuesta ante incidencias y facilitará el control del proceso.

3.2.6 Desarrollar un sistema de almacenamiento y gestión de datos

La recopilación de información constituye un aspecto clave en los entornos industriales modernos. Por ello, el proyecto contempla la implementación de una base de datos que permita registrar información relacionada con los procesos productivos y los palillos procesados. Esta información podrá utilizarse para realizar análisis posteriores, mejorar la trazabilidad y facilitar la toma de decisiones basada en datos.

3.2.7 Incrementar la productividad de la línea de producción

Uno de los beneficios esperados de la automatización es el aumento de la productividad. Mediante la reducción de tiempos muertos, la eliminación de tareas manuales repetitivas y la coordinación automática de los distintos elementos de la instalación, se espera conseguir un proceso más eficiente y capaz de mantener un ritmo de producción constante.

3.2.8 Reducir la presencia de errores humanos

La automatización de tareas críticas como la clasificación, manipulación y serigrafía de los palillos permitirá disminuir la influencia de factores humanos que pueden afectar a la calidad del producto final. De esta forma se pretende reducir el número de errores de producción y aumentar la fiabilidad del sistema.

3.2.9 Mejorar la seguridad y las condiciones de trabajo

El proyecto también persigue mejorar las condiciones de trabajo de los operarios mediante la reducción de tareas repetitivas y de bajo valor añadido. La automatización de estas operaciones contribuye a disminuir la fatiga física, mejora la ergonomía del puesto de trabajo y reduce la exposición de los trabajadores a determinadas situaciones de riesgo.

3.2.10 Validar la solución mediante simulación virtual

Finalmente, se pretende verificar el correcto funcionamiento de la solución desarrollada mediante una simulación completa en RoboDK. Esta simulación permitirá comprobar la coordinación entre los distintos componentes del sistema, detectar posibles errores antes de una implantación real y evaluar el comportamiento general de la instalación en condiciones de funcionamiento representativas.

3.3 Funcionamiento del sistema tras la automatización

3.3.1 Alimentación de los palillos

El proceso productivo comienza con la alimentación de los palillos de madera a la línea de producción. Para ello, la instalación dispone de una tolva de almacenamiento cuya función es suministrar de forma continua los palillos a la estación. La tolva permite mantener una reserva de material suficiente para evitar interrupciones durante la producción y garantizar un flujo constante de piezas.

Los palillos son depositados sobre la cinta transportadora de entrada en grupos de tres unidades. Esta agrupación facilita la manipulación posterior por parte del robot Delta y permite mantener una organización adecuada de las piezas a lo largo de todo el proceso. La cinta de entrada constituye el primer elemento de transporte de la instalación y se encarga de desplazar los palillos desde la tolva hasta la zona de inspección visual.

El movimiento continuo de la cinta permite que los grupos de palillos lleguen de forma ordenada a la estación de clasificación, donde serán evaluados para determinar si cumplen los criterios de calidad establecidos por la empresa.

3.3.2 Sistema de inspección y clasificación

Una vez que los palillos alcanzan la zona de inspección, interviene el sistema de visión artificial. La solución propuesta utiliza una cámara inteligente OAK-D compatible con OpenCV, capaz de realizar tareas de detección y clasificación de objetos.

En una instalación industrial real, esta cámara permitiría identificar defectos como grietas, roturas, deformaciones, manchas superficiales, irregularidades dimensionales o cualquier otra anomalía que pudiera afectar a la calidad del producto final. Sin embargo, debido al carácter académico y de simulación del proyecto, la identificación de defectos se ha simplificado mediante un sistema basado en colores.

Los palillos correctos se representan mediante una tonalidad marrón clara, mientras que los palillos defectuosos se representan mediante una tonalidad marrón oscura. Gracias a esta diferenciación visual, el sistema es capaz de simular el comportamiento de un algoritmo de clasificación automática sin necesidad de implementar un sistema completo de procesamiento de imágenes.

La información obtenida durante esta fase permite decidir qué piezas continuarán el proceso productivo y cuáles serán retiradas de la línea por presentar defectos.

3.3.3 Manipulación mediante robot Delta ABB IRB 360-1/800 4D

Tras la clasificación, entra en funcionamiento el robot industrial ABB IRB 360 FlexPicker, también conocido como robot Delta. Este robot ha sido seleccionado debido a su elevada velocidad de trabajo y a su precisión en tareas de manipulación de objetos ligeros.

El robot recoge los grupos de palillos directamente desde la cinta transportadora de entrada utilizando una herramienta terminal basada en una ventosa. Este sistema de agarre permite manipular las piezas de forma rápida y segura, evitando daños sobre la superficie de la madera.

Una vez recogidos los palillos, el robot decide su destino en función del resultado de la clasificación. Los palillos considerados correctos son depositados sobre la cinta transportadora principal o cinta central, mientras que los palillos defectuosos son enviados a la cinta de salida destinada al rechazo de material.

Esta etapa constituye uno de los elementos fundamentales del sistema automatizado, ya que permite separar automáticamente los productos válidos de aquellos que no cumplen los requisitos de calidad.

3.3.4 Gestión de piezas defectuosas

Los palillos identificados como defectuosos son transportados mediante una cinta específica de rechazo. Esta cinta conduce las piezas hasta un contenedor destinado a almacenar temporalmente los productos descartados.

La cinta de salida puede albergar grupos de hasta cuatro palillos defectuosos antes de que estos sean depositados en el contenedor final. Este procedimiento permite mantener separadas las piezas incorrectas del resto de la producción, evitando mezclas accidentales y facilitando su posterior gestión.

En una instalación industrial real, este material podría ser enviado a procesos de reciclaje o reutilización de la madera. En la simulación desarrollada, el objetivo principal es representar de forma realista el flujo de rechazo de piezas defectuosas.

3.3.5 Estación de serigrafía robotizada

Los palillos que superan correctamente la fase de inspección son depositados sobre una bandeja situada en la cinta transportadora central. Dicha bandeja tiene capacidad para almacenar nueve palillos distribuidos en tres grupos de tres unidades.

Cuando la bandeja alcanza su capacidad máxima, se inicia la operación de serigrafía. Esta tarea es realizada por dos robots colaborativos Universal Robots UR3e que trabajan de forma coordinada.

El primer robot UR3e se encarga de procesar cinco palillos de la bandeja, mientras que el segundo robot procesa los cuatro restantes. Gracias a esta distribución del trabajo se consigue reducir el tiempo de ciclo y aumentar la productividad de la estación.

La herramienta utilizada por ambos robots simula un sistema de grabado láser. Aunque en la simulación no se realiza ningún marcado físico sobre los palillos, los robots ejecutan todos los movimientos necesarios que serían utilizados en una instalación real.

El sistema está diseñado para permitir la personalización de los productos mediante la incorporación de textos, logotipos, imágenes o cualquier otro elemento gráfico solicitado por el cliente. Esta flexibilidad constituye una de las principales ventajas de utilizar robots colaborativos en procesos de serigrafía industrial.

3.3.6 Sistema de comunicación MQTT

La coordinación entre todos los componentes del sistema se realiza mediante el protocolo MQTT, ampliamente utilizado en aplicaciones de Industria 4.0 e Internet de las Cosas (IoT).

MQTT actúa como elemento de comunicación entre RoboDK, los programas desarrollados en Python y la ESP32-S3 encargada de gestionar los dispositivos físicos de la estación. Gracias a este protocolo, los distintos componentes pueden intercambiar información en tiempo real de forma rápida y eficiente.

El sistema utiliza el broker MQTT EMQX alojado en internet. A través de este broker se publican y reciben mensajes relacionados con el estado de la producción, eventos de emergencia, iluminación de la estación y estado del contenedor de piezas defectuosas.

Entre los principales mensajes intercambiados se encuentran las órdenes de parada e inicio de la estación, las notificaciones de contenedor lleno o vacío, las alertas de iluminación y las órdenes de encendido y apagado de los indicadores luminosos.

3.3.7 Sistema de control basado en ESP32-S3

La ESP32-S3 constituye el núcleo de control físico del sistema. Su función principal es actuar como intermediario entre la simulación desarrollada en RoboDK y los distintos dispositivos electrónicos instalados en la estación.

La placa se conecta a la red WiFi y posteriormente al broker MQTT, permitiendo el intercambio de información con el resto de los elementos del sistema. A través de diferentes topics MQTT recibe órdenes para controlar la pantalla LCD y los indicadores luminosos, además de transmitir la información generada por los sensores.

La instalación dispone de una pantalla LCD I2C de 16 columnas y 2 filas encargada de mostrar información relevante para el operario. En ella aparecen mensajes relacionados con el estado de la producción, las condiciones de funcionamiento de la estación y los distintos eventos generados durante la ejecución del proceso.

Asimismo, la ESP32-S3 controla dos sistemas de señalización luminosa asociados a la cinta de entrada y a la cinta de salida. Estos indicadores permiten visualizar de forma rápida el estado operativo de cada zona de la instalación.

El sistema incorpora también un botón de emergencia que permite detener inmediatamente la estación. Cuando el operario pulsa dicho botón, la ESP32-S3 publica mediante MQTT el mensaje "PARAR". Este mensaje es recibido por el sistema de control desarrollado en Python, que ordena la detención de la simulación en RoboDK. Cuando el botón vuelve a pulsarse, se publica el mensaje "INICIAR", permitiendo reanudar la producción.

Además, la estación dispone de un sensor ultrasónico situado junto al contenedor de rechazo. Este sensor permite determinar si el contenedor se encuentra lleno o vacío. Cuando detecta que el nivel de llenado ha alcanzado el límite establecido, publica el mensaje "LLENO". En caso contrario, publica el mensaje "VACIO". Esta información también es mostrada al operario mediante la pantalla LCD.

Por otra parte, se incorpora una resistencia dependiente de la luz (LDR) encargada de monitorizar la iluminación ambiental de la estación. Cuando la iluminación es insuficiente, el sistema genera una alerta indicando la necesidad de aumentar la luz. De forma similar, cuando el nivel de

iluminación es excesivo, se recomienda reducirla para mantener unas condiciones adecuadas de funcionamiento.

Internamente, el programa de la ESP32-S3 se encuentra dividido en varias tareas concurrentes gestionadas mediante FreeRTOS. Cada tarea se encarga de una función específica, como la gestión de la pantalla LCD, el control de los LEDs, la lectura de sensores o la comunicación MQTT. Esta arquitectura mejora la eficiencia y permite ejecutar varias operaciones simultáneamente.

3.3.8 Gestión de datos y base de datos

Con el objetivo de disponer de un registro permanente de la producción, el sistema incorpora una base de datos PostgreSQL. Esta base de datos almacena información relacionada con los procesos productivos y los palillos procesados por la línea.

Actualmente se registra información asociada a los palillos correctos, permitiendo conocer la cantidad de productos válidos generados durante cada proceso de fabricación. Cada palillo queda asociado al proceso productivo correspondiente mediante identificadores únicos.

La utilización de una base de datos facilita la trazabilidad de la producción y constituye una base sólida para futuras ampliaciones del sistema, como la incorporación de estadísticas avanzadas, análisis de rendimiento o control detallado de defectos.

3.3.9 Salida y almacenamiento del producto final

Una vez completada la operación de serigrafía, los palillos correctos continúan avanzando por la cinta transportadora central hasta alcanzar la zona final de la estación.

En esta posición permanecen organizados sobre la bandeja de salida, preparados para ser retirados por el operario o enviados a procesos posteriores de almacenamiento o distribución.

De esta manera se garantiza que únicamente los productos que han superado correctamente todas las etapas de clasificación y procesamiento llegan al final de la línea de producción.

3.3.10 Simulación integral en RoboDK

Todo el sistema ha sido desarrollado y validado mediante una simulación completa en RoboDK. La simulación reproduce el funcionamiento de la totalidad de la estación industrial, incluyendo las cintas transportadoras, el robot Delta ABB IRB 360-1/800 4D, los robots colaborativos UR3e, los sensores, la comunicación MQTT y la interacción con la ESP32-S3.

Gracias a esta simulación integral es posible verificar el comportamiento de la instalación antes de una posible implementación física, detectar errores de programación, optimizar trayectorias robóticas y comprobar la correcta sincronización entre todos los elementos que componen el sistema.

La utilización de RoboDK reduce los costes de desarrollo y permite validar el diseño de forma segura y eficiente, constituyendo una herramienta fundamental durante todas las fases del proyecto.

3.4 Criterios y metas cuantificables (KPIs)

Con el objetivo de evaluar el rendimiento de la solución propuesta y determinar el grado de éxito alcanzado por la automatización, se han definido una serie de indicadores clave de rendimiento (KPIs). Estos indicadores permiten medir de forma objetiva las mejoras obtenidas respecto al proceso tradicional de serigrafía manual.

3.4.1 Tiempo medio de producción

Uno de los principales objetivos del proyecto es reducir el tiempo necesario para procesar un lote completo de palillos. Para evaluar esta mejora se ha comparado el tiempo de la solución automatizada con el tiempo requerido en un proceso manual convencional.

A partir de la observación de un proceso de serigrafía manual, se ha estimado que cada palillo requiere aproximadamente 10 segundos para ser colocado, procesado y retirado de la máquina. Considerando que cada lote está compuesto por 9 palillos, el tiempo total de producción manual es de aproximadamente 90 segundos.

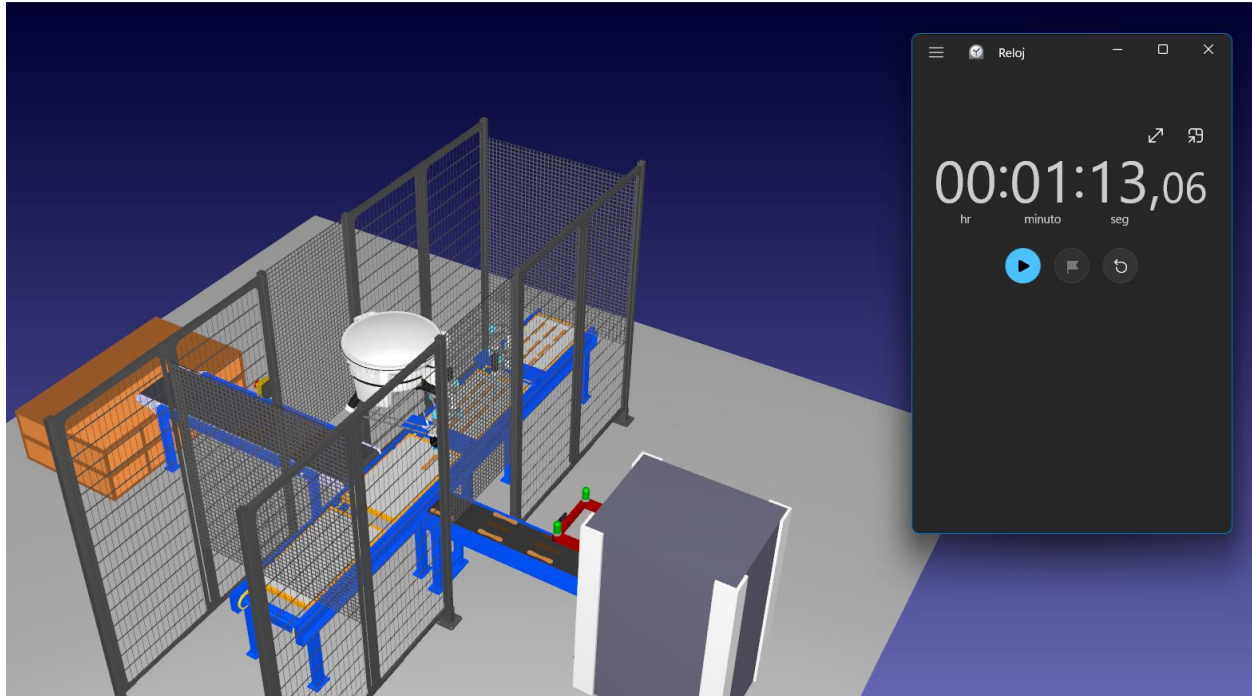
Por otro lado, la simulación desarrollada en RoboDK ha permitido medir un tiempo medio de producción de 55 segundos para procesar un lote completo.

Indicador: Tiempo de producción por lote.

Proceso manual: 90 segundos. [Video de Proceso Manual de Serigrafía](#)

Proceso automatizado: 55 segundos. Proceso automatizado: 55 segundos. Para obtener una medida representativa del ciclo productivo, se ha descontado el tiempo inicial de arranque de la simulación en RoboDK (18 segundos), ya que dicho tiempo no forma parte del proceso de producción propiamente dicho.

Figura 3.4 1



Mejora obtenida:

Reducción del tiempo de producción (%) = $((T_{\text{manual}} - T_{\text{automatizado}}) / T_{\text{manual}}) \times 100$

Donde:

- T_{manual} = tiempo empleado en el proceso manual.
- $T_{\text{automatizado}}$ = tiempo empleado en el proceso automatizado.

Aplicando los valores obtenidos:

Mejora = $((90 - 55) / 90) \times 100 = 38,89\%$

Resultado: reducción del tiempo de producción del 38,89 %.

3.4.2 Productividad de la línea

La reducción del tiempo de ciclo permite aumentar el número de piezas producidas por unidad de tiempo.

Para evaluar la productividad del sistema se ha calculado el número de palillos que pueden procesarse en una hora de funcionamiento continuo.

En el proceso manual, un lote de nueve palillos requiere aproximadamente 90 segundos para completarse. Por tanto, la productividad obtenida es:

En el proceso manual:

Productividad del proceso manual (palillos/hora) = $(3600 / T_{\text{manual}}) \times N$

Donde:

- 3600 = segundos que tiene una hora.
- T_{manual} = tiempo necesario para procesar un lote.
- N = número de palillos por lote.

Aplicando los valores obtenidos:

Productividad manual = $((3600/90) \times 9) = 360$ palillos/hora

Esto significa que un operario podría procesar aproximadamente **360 palillos por hora**.

En el proceso automatizado, la simulación desarrollada en RoboDK completa un lote de nueve palillos en 55 segundos. La productividad resultante es:

En el proceso automatizado:

Productividad del proceso automatizado (palillos/hora) = $(3600 / T_{\text{automatizado}}) \times N$

Donde:

- 3600 = segundos que tiene una hora.
- $T_{\text{automatizado}}$ = tiempo necesario para procesar un lote mediante la solución automatizada.
- N = número de palillos por lote.

Aplicando los valores obtenidos:

Productividad automatizada = $((3600/55) \times 9) = 589$ palillos/hora

Por tanto, el sistema automatizado es capaz de procesar aproximadamente **589 palillos por hora**.

Para determinar la mejora obtenida se ha calculado el incremento porcentual de productividad:

Incremento obtenido:



Incremento de productividad (%) = $((P_{\text{automatizada}} - P_{\text{manual}}) / P_{\text{manual}}) \times 100$

Donde:

- P_{manual} = productividad del proceso manual.
- $P_{\text{automatizada}}$ = productividad del proceso automatizado.

Aplicando los valores obtenidos:

Incremento de productividad = $((589 - 360) / 360) \times 100 = 63,61\%$

Resultado: aumento de la productividad del 63,61 %.

Los resultados muestran que la automatización propuesta incrementa la productividad de la línea en un **63,61 %** respecto al proceso manual, permitiendo producir un mayor número de palillos en el mismo periodo de tiempo y mejorando significativamente la eficiencia global del proceso.

3.4.3 Gestión de piezas defectuosas

La línea debe garantizar que las piezas defectuosas no alcancen la etapa de serigrafía.

Indicador: Porcentaje de palillos defectuosos desviados correctamente.

Meta: 100 % de piezas defectuosas enviadas al contenedor de rechazo.

Resultado esperado: Ninguna pieza defectuosa llega a la zona de serigrafía.

3.4.5 Fiabilidad de las comunicaciones MQTT

El correcto intercambio de información entre RoboDK, Python y la ESP32-S3 depende de la estabilidad de las comunicaciones MQTT.

Indicador: Porcentaje de mensajes transmitidos correctamente.

Meta: 100 % de recepción de mensajes críticos.

Mensajes evaluados:

- PARAR
- INICIAR
- LLENO
- VACIO
- MUCHA_LUZ
- POCA_LUZ

- ENCENDER_LED
- APAGAR_LED

Resultado esperado: Ausencia de pérdidas de comunicación durante la ejecución de la simulación.

3.4.6 Latencia de las comunicaciones MQTT

Además de la fiabilidad de la comunicación, se ha evaluado el tiempo de respuesta entre el envío y la recepción de los mensajes MQTT.

Indicador: Tiempo de transmisión de mensajes.

Meta: Tiempo de respuesta inferior a 1 segundo.

Resultado obtenido: Durante las pruebas realizadas se observó un retardo máximo aproximado de 1 segundo entre la publicación y la recepción de determinados mensajes, manteniéndose dentro de los límites establecidos para el correcto funcionamiento de la estación.

3.4.7 Control del contenedor de rechazo

El sensor ultrasónico debe detectar correctamente el estado del contenedor de piezas defectuosas.

Indicador: Precisión de detección de estados.

Meta: 100 % de detección correcta de los estados LLENO y VACIO.

Resultado esperado: Publicación correcta del estado mediante MQTT y visualización en la pantalla LCD.

3.4.8 Control de iluminación

La estación incorpora un sensor LDR para supervisar las condiciones lumínicas del entorno.

Indicador: Precisión de detección de iluminación.

Meta: 80 % de detección correcta de los estados MUCHA_LUZ y POCA_LUZ.

Resultado esperado: Generación automática de avisos para aumentar o disminuir la iluminación cuando sea necesario.

3.4.9 Trazabilidad y almacenamiento de información

La base de datos PostgreSQL permite registrar los datos generados durante el proceso productivo.

Indicador: Porcentaje de registros almacenados correctamente.

Meta: Registrar el 100 % de los palillos válidos procesados.



Resultado esperado: Durante las pruebas realizadas todos los palillos válidos procesados fueron registrados correctamente en la base de datos PostgreSQL. Los datos almacenados permiten consultar la información asociada a cada proceso productivo y realizar un seguimiento completo de la producción.

Además, la utilización de una base de datos centralizada facilita la supervisión de la actividad de la estación y permite mantener un historial de la producción realizada. Esta información puede utilizarse posteriormente para generar estadísticas de producción, analizar el rendimiento de la línea, detectar posibles incidencias y mejorar la toma de decisiones.

La implementación de este sistema de almacenamiento también proporciona una base sólida para futuras ampliaciones del proyecto, como la incorporación de nuevos indicadores de calidad, registros de piezas defectuosas, tiempos de ciclo, alarmas o sistemas de monitorización remota.

Conclusión: Se alcanzó una tasa de almacenamiento correcta del 100 % de los registros generados, garantizando la trazabilidad completa de los palillos procesados y la disponibilidad permanente de información para futuras consultas, análisis y mejoras del sistema.

3.4.10 Resumen de KPIs

RESUMEN KPIs				
KPI	Indicador	Valor Manual	Valor Automatizado	Resultado / Meta
Tiempo medio de producción	Tiempo por lote (9 palillos)	90 s	55 s	Reducción del 38,89 %
Productividad de la línea	Palillos producidos por hora	360 palillos/h	589 palillos/h	Incremento del 63,61 %
Gestión de piezas defectuosas	Palillos defectuosos desviados correctamente	No automatizado	Automatizado	100 % de piezas defectuosas enviadas al rechazo
Fiabilidad MQTT	Mensajes transmitidos correctamente	No aplicable	100 %	Sin pérdidas de mensajes

RESUMEN KPIs				
KPI	Indicador	Valor Manual	Valor Automatizado	Resultado / Meta
Latencia MQTT	Tiempo de transmisión de mensajes	No aplicable	≤ 1 s	Retardo máximo aproximado de 1 segundo
Control del contenedor de rechazo	Precisión de detección de estados	Manual	Automático	100 % de detección de estados LLENO y VACÍO
Control de iluminación	Precisión de detección	Manual	Automático	80 % de precisión
Trazabilidad y almacenamiento	Registros almacenados correctamente	Parcial	Automatizado	100 % de registros almacenados

En conjunto, estos indicadores permiten evaluar de forma cuantitativa la eficacia de la solución propuesta y demostrar las mejoras obtenidas mediante la automatización del proceso de clasificación y serigrafía de palillos de madera.

Los resultados obtenidos demuestran que la automatización propuesta mejora significativamente la eficiencia del proceso productivo. Destacan especialmente la reducción del tiempo de producción en un 38,89 % y el aumento de la productividad en un 63,61 %. Además, la integración de comunicaciones MQTT, sensores IoT y una base de datos PostgreSQL permite garantizar la trazabilidad de la producción y la supervisión en tiempo real de la estación.

3.5 Mejoras

3.5.1 Impacto sobre los recursos humanos

La propuesta de automatización reduce significativamente la necesidad de realizar tareas repetitivas y manuales, como la inspección visual de los palillos, su manipulación entre estaciones o la alimentación continua de la máquina de serigrafía. Esto permite que los operarios puedan centrar su actividad en tareas de mayor valor añadido, como la supervisión de la producción, el mantenimiento preventivo de los equipos o la gestión de incidencias.

Como consecuencia, se mejora la eficiencia del trabajo y se reduce la probabilidad de errores derivados de la fatiga, la monotonía o la manipulación continua de piezas. La automatización no pretende sustituir completamente al personal humano, sino mejorar sus condiciones de trabajo y permitir que desempeñe funciones de supervisión y control más especializadas.

3.5.2 Impacto sobre el proceso productivo

La automatización mejora la coordinación entre los distintos procesos de la planta, reduciendo tiempos de espera y optimizando el flujo de trabajo entre las etapas de clasificación, transporte y serigrafía.

La utilización de robots industriales y sistemas de comunicación permite sincronizar automáticamente todas las operaciones, evitando cuellos de botella y garantizando una producción más continua y estable. Además, la reducción de los tiempos de ciclo permite aumentar la productividad general de la instalación y mejorar el aprovechamiento de los recursos disponibles.

3.5.3 Mejora de la calidad del producto

La clasificación automática de los palillos permite detectar y separar las piezas defectuosas antes de que lleguen a la etapa de serigrafía, evitando el procesamiento de productos que no cumplen los estándares de calidad establecidos.

Esta mejora contribuye a aumentar la uniformidad del producto final, reducir errores de producción y minimizar la posibilidad de que piezas defectuosas lleguen al cliente final. Asimismo, el uso de robots para las operaciones de manipulación y serigrafía garantiza una mayor repetibilidad y precisión en el proceso.

3.5.4 Gestión de la información y trazabilidad

La incorporación de una base de datos PostgreSQL supone una mejora importante en la gestión de la información generada durante la producción.

Gracias al almacenamiento automático de los datos, es posible disponer de un registro actualizado de los palillos procesados, facilitando la trazabilidad de los productos y permitiendo realizar análisis posteriores sobre el rendimiento de la instalación. Esta información puede utilizarse para obtener estadísticas de producción, detectar incidencias recurrentes o planificar futuras mejoras del sistema.

3.5.5 Supervisión y control de la instalación

La integración de sensores, comunicaciones MQTT y una ESP32-S3 proporciona una mayor capacidad de supervisión y control sobre el estado de la estación.

El operario puede conocer en tiempo real información relacionada con el nivel de llenado del contenedor de rechazo, las condiciones de iluminación del entorno o el estado de determinados actuadores y elementos de señalización. Esta capacidad de monitorización permite reaccionar rápidamente ante posibles incidencias y facilita la toma de decisiones durante el funcionamiento de la instalación.

3.5.6 Optimización de recursos materiales

La automatización contribuye a un mejor aprovechamiento de las materias primas al reducir la posibilidad de procesar piezas defectuosas y minimizar errores durante la manipulación de los palillos.

Como consecuencia, se disminuye el desperdicio de material y se mejora la eficiencia global de la línea de producción. La correcta separación entre piezas válidas y defectuosas también facilita la gestión posterior de los residuos generados durante el proceso.

3.5.7 Posibilidades de ampliación futura

La solución desarrollada ha sido diseñada siguiendo principios propios de la Industria 4.0, utilizando tecnologías como robots industriales, comunicaciones IoT, bases de datos y sistemas de supervisión distribuidos.

Esta arquitectura facilita futuras ampliaciones del proyecto, permitiendo incorporar nuevos sensores, algoritmos avanzados de visión artificial, herramientas de análisis de datos o sistemas de monitorización remota sin necesidad de modificar completamente la instalación existente.

En conjunto, la propuesta de automatización no solo mejora la productividad de la línea de producción, sino que también aporta beneficios en términos de calidad, trazabilidad, supervisión, eficiencia operativa y capacidad de crecimiento futuro.

3.6 Impactos Adicionales

La implantación del sistema automatizado no solo mejora el proceso principal de clasificación y serigrafía de palillos, sino que también genera distintos impactos adicionales a nivel ambiental, tecnológico, logístico y laboral. Estos impactos contribuyen a modernizar la producción y aumentar la competitividad de la empresa.



3.6.1 Impacto ambiental

Desde el punto de vista ambiental, la automatización contribuye a reducir el desperdicio de material gracias a la detección temprana de piezas defectuosas. Al identificar los palillos defectuosos antes de que alcancen la estación de serigrafía, se evita el consumo innecesario de recursos asociados a las etapas posteriores del proceso.

Asimismo, la correcta clasificación de las piezas permite reducir el número de productos que deben ser reprocesados o descartados una vez finalizada la producción. Esto supone un mejor aprovechamiento de las materias primas y una disminución de los residuos generados durante la fabricación.

Aunque el proyecto se ha desarrollado mediante simulación, una posible implantación real permitiría optimizar el consumo de materiales y mejorar la sostenibilidad del proceso productivo.

3.6.2 Impacto tecnológico

La solución desarrollada incorpora tecnologías propias de los entornos industriales modernos y de la Industria 4.0. Entre ellas destacan el uso de robots industriales, robots colaborativos, comunicaciones MQTT, sensores inteligentes, bases de datos y sistemas de supervisión basados en IoT.

La integración entre RoboDK, Python, PostgreSQL y la ESP32-S3 permite crear una arquitectura conectada capaz de intercambiar información en tiempo real entre todos los componentes de la estación. Esta capacidad de comunicación facilita la monitorización del proceso y mejora la coordinación entre los distintos elementos de la instalación.

Además, la utilización de una arquitectura modular facilita futuras ampliaciones tecnológicas, como la incorporación de algoritmos avanzados de visión artificial, nuevos sensores o sistemas de monitorización remota.

3.6.3 Impacto logístico

La automatización mejora la organización del flujo de materiales dentro de la estación de trabajo. La coordinación automática entre cintas transportadoras, robots y sistemas de control reduce los tiempos de espera entre operaciones y evita acumulaciones innecesarias de piezas.

La clasificación automática de los palillos también simplifica la gestión logística del proceso, ya que separa de forma inmediata los productos correctos de aquellos que presentan defectos. Esto permite mantener una mayor organización de la producción y facilita las operaciones posteriores de almacenamiento o distribución.



Asimismo, la monitorización en tiempo real del estado de la instalación permite anticipar posibles incidencias y mejorar la planificación de tareas relacionadas con el mantenimiento o la reposición de materiales.

3.6.4 Impacto sobre el personal

La automatización reduce la necesidad de que los operarios realicen tareas repetitivas como la clasificación manual de los palillos, su manipulación continua o la alimentación de la estación de serigrafía.

Gracias a ello, el personal puede dedicar más tiempo a actividades de supervisión, control de calidad y mantenimiento de los equipos. Esto mejora las condiciones de trabajo, reduce la fatiga asociada a tareas repetitivas y disminuye la probabilidad de errores humanos durante la producción.

Además, la utilización de tecnologías avanzadas favorece el desarrollo de nuevas competencias técnicas relacionadas con la programación, la supervisión de sistemas automatizados y el uso de herramientas de Industria 4.0.

3.6.5 Impacto económico

La mejora de la productividad obtenida mediante la automatización permite aumentar la cantidad de piezas procesadas por unidad de tiempo y reducir el tiempo necesario para completar cada lote de producción.

La disminución de errores y la correcta gestión de las piezas defectuosas contribuyen igualmente a reducir pérdidas asociadas a reprocesos o desperdicios de material. Todo ello puede traducirse en una reducción de los costes operativos y en una mejora de la rentabilidad de la instalación a medio y largo plazo.

3.6.6 Impacto sobre el crecimiento futuro

La arquitectura diseñada permite incorporar nuevas funcionalidades sin necesidad de modificar completamente el sistema existente. Esto facilita la evolución futura de la instalación y su adaptación a nuevas necesidades productivas.

Entre las posibles ampliaciones futuras destacan la incorporación de sistemas reales de visión artificial basados en OpenCV, la integración con plataformas MES o ERP, la monitorización remota mediante aplicaciones web o la ampliación de la base de datos para registrar nuevos indicadores de producción y calidad.

En conjunto, estos impactos muestran que la propuesta de automatización no solo mejora el proceso de clasificación y serigrafía, sino que también aporta beneficios significativos en términos de sostenibilidad, digitalización, organización de la producción, condiciones de trabajo y capacidad de crecimiento futuro.

3.7 Límites y restricciones del proyecto

Aunque la solución propuesta permite automatizar gran parte del proceso de clasificación y serigrafía de palillos, existen una serie de límites y restricciones que deben tenerse en cuenta durante el desarrollo y evaluación del proyecto. Estas limitaciones están relacionadas tanto con el alcance académico de la propuesta como con las características propias de los sistemas utilizados y las posibles dificultades asociadas a una futura implantación industrial.

3.7.1 Naturaleza simulada del proyecto

La principal limitación del proyecto es que la solución ha sido desarrollada y validada mediante simulación en RoboDK. Por tanto, los resultados obtenidos representan una aproximación al comportamiento de una instalación industrial real, pero no contemplan todas las variables presentes en un entorno físico.

Factores como vibraciones mecánicas, desgaste de componentes, tolerancias de fabricación, variaciones en la velocidad de los actuadores, problemas eléctricos o incidencias mecánicas no pueden reproducirse completamente dentro de la simulación. Esto implica que, aunque el comportamiento general del sistema es representativo, los resultados obtenidos podrían diferir de los observados en una instalación real.

3.7.2 Limitaciones del sistema de visión artificial

Aunque la propuesta contempla la utilización de una cámara OAK-D para tareas de inspección, la detección de defectos se ha simplificado en la simulación mediante la diferenciación de colores entre palillos correctos y defectuosos.

Por este motivo, el sistema desarrollado no realiza actualmente una detección real de grietas, roturas, deformaciones, defectos superficiales o anomalías dimensionales complejas. La incorporación de algoritmos avanzados de visión artificial y técnicas de aprendizaje automático constituye una posible línea de mejora futura.

3.7.3 Dependencia de las comunicaciones MQTT

El correcto funcionamiento de la estación depende de la comunicación entre RoboDK, los programas desarrollados en Python y la ESP32-S3 mediante MQTT.

Cualquier fallo en la conexión de red, interrupción del broker MQTT o retraso excesivo en la transmisión de mensajes podría afectar a la sincronización entre los distintos componentes del sistema y provocar comportamientos no deseados durante la producción.

Aunque durante las pruebas realizadas la comunicación ha resultado estable, la dependencia de las comunicaciones constituye uno de los factores críticos de la instalación.

3.7.4 Dependencia de la correcta sincronización entre sistemas

La propuesta integra múltiples tecnologías que deben funcionar de forma coordinada, incluyendo robots industriales, sensores, bases de datos, sistemas de comunicación y dispositivos IoT.

La aparición de errores de sincronización entre estos elementos podría afectar al flujo de producción, provocar bloqueos temporales o generar incidencias durante la ejecución del proceso automatizado.

Por este motivo resulta fundamental mantener una correcta coordinación entre todos los componentes software y hardware que forman parte de la estación.

3.7.5 Limitaciones de los sensores utilizados

Algunos sensores utilizados en el proyecto presentan limitaciones inherentes a su funcionamiento. Un ejemplo es el sensor LDR encargado de supervisar las condiciones de iluminación, cuya precisión presenta un margen de error aproximado del 20 % durante las pruebas realizadas.

Asimismo, la precisión de determinadas medidas puede verse afectada por factores externos como las condiciones ambientales, la posición de los sensores o posibles interferencias presentes en el entorno de trabajo.

Estas limitaciones son habituales en sistemas industriales reales y deben considerarse durante el diseño de estrategias de supervisión y control.

3.7.6 Alcance académico del proyecto

El proyecto ha sido desarrollado con fines académicos y de aprendizaje, por lo que determinados aspectos industriales han sido simplificados para adaptarse al tiempo disponible, a los recursos del equipo y a los objetivos de la asignatura.

Entre estas simplificaciones se encuentran la simulación del proceso de serigrafía, la detección simplificada de defectos, la utilización de un entorno virtual para validar la instalación y la ausencia de determinados sistemas avanzados presentes en líneas de producción industriales completamente operativas.

A pesar de estas limitaciones, la solución desarrollada permite demostrar los principios fundamentales de automatización industrial, integración de sistemas, comunicaciones IoT, robótica industrial y gestión de datos aplicados a un entorno productivo.

3.7.7 Consideraciones para una implantación industrial real

Aunque el proyecto se ha desarrollado en un entorno de simulación, una posible implantación industrial de la solución propuesta implicaría una serie de requisitos y restricciones adicionales.

Entre ellos destaca la inversión económica necesaria para la adquisición de robots industriales ABB IRB 360, robots colaborativos UR3e, sistemas de visión artificial, cintas transportadoras, sensores, equipos de control y sistemas de comunicación. Esta inversión inicial puede representar una barrera importante para determinadas empresas.

Además, sería necesario establecer planes de mantenimiento preventivo y correctivo para garantizar la disponibilidad de los equipos y minimizar posibles paradas de producción. Los robots, sensores y sistemas de transporte requieren inspecciones periódicas, calibraciones y sustitución de componentes cuando sea necesario.

Por otra parte, el personal debería recibir formación específica para la operación, supervisión y mantenimiento de sistemas automatizados. La incorporación de nuevas tecnologías implica la necesidad de adquirir conocimientos relacionados con robótica, comunicaciones industriales, sensores y sistemas de control.

Finalmente, una instalación real dependería de una infraestructura de comunicaciones estable, procedimientos de seguridad adecuados y medidas de protección para garantizar un funcionamiento fiable y seguro de la línea de producción.

En conjunto, estas limitaciones y restricciones no impiden el desarrollo de la propuesta planteada, pero deben ser consideradas tanto durante la fase de diseño como en una posible implantación industrial futura.

4. Diseño de la Solución

4.1 Elementos de la automatización

En este apartado se describen los principales elementos y dispositivos que forman parte de la solución automatizada propuesta para la clasificación y serigrafía de palillos de madera. Cada componente cumple una función específica dentro de la estación y trabaja de forma coordinada con el resto del sistema mediante la integración entre RoboDK, Python, MQTT, la ESP32-S3 y la base de datos PostgreSQL.

La estación está compuesta por elementos mecánicos, robóticos, electrónicos y software. Entre los elementos mecánicos destacan la tolva de alimentación, las cintas transportadoras, el contenedor de rechazo y el vallado de seguridad. En cuanto a los elementos robóticos, se emplea un robot industrial ABB IRB 360 para la clasificación de los palillos y dos robots colaborativos UR3e para la operación de serigrafía. Además, se incorporan sensores, actuadores, indicadores luminosos, una pantalla LCD y un botón de parada de emergencia gestionados mediante la ESP32-S3.

4.1.1 Tabla resumen de dispositivos

Dispositivo / Elemento	Cantidad	Función principal
Tolva de alimentación	1	Almacenar y suministrar los palillos a la cinta de entrada.
Cinta transportadora de entrada	1	Transportar los palillos hasta la zona de clasificación.
Cámara IA OpenCV OAK-D	1	Inspección visual y clasificación de piezas.
ABB IRB 360-1/800 4D	1	Clasificación y distribución de palillos.
RobotiQ EPick Vacuum Gripper	1	Manipulación de piezas mediante vacío.
Cinta transportadora central	1	Transporte de piezas válidas hacia la serigrafía.
Cinta transportadora de rechazo	1	Transporte de piezas defectuosas.
Contenedor de rechazo	1	Almacenamiento de palillos defectuosos.
Robots colaborativos UR3e	2	Simulación del proceso de serigrafía.
Herramienta láser de grabado	2	Herramienta de marcado simulada.
ESP32-S3	1	Gestión de sensores y actuadores.
Sensor ultrasónico	1	Control del nivel del contenedor de rechazo.
Sensor LDR	1	Supervisión de las condiciones de iluminación.
Pantalla LCD I2C	1	Visualización de mensajes y estados.



LEDs RGB indicadores	2	Señalización visual de funcionamiento.
Botón de emergencia	1	Parada y reanudación de la estación.
Vallado de seguridad	1 conjunto	Protección de la zona robotizada.
Broker MQTT	1	Comunicación entre sistemas.
PostgreSQL	1	Almacenamiento de información.
RoboDK	1	Simulación de la estación automatizada.
Python	1	Lógica de control e integración.

4.1.2 Tolva de alimentación y sistema de entrada

La tolva de alimentación constituye el punto inicial de la estación automatizada. Su función es almacenar temporalmente los palillos de madera antes de que entren en la línea de producción y suministrarlos de forma controlada a la cinta transportadora de entrada.

Especificaciones principales

- Tipo: Tolva de alimentación.
- Función: Almacenamiento temporal de materia prima.
- Alimentación: Grupos de tres palillos.
- Integración: Conectada directamente a la cinta de entrada.

Función en el proyecto

Los palillos son introducidos en grupos de tres unidades y depositados sobre la cinta transportadora de entrada. La tolva garantiza un flujo continuo de material y evita interrupciones durante el funcionamiento de la estación.

4.1.3 Sistema de visión artificial OAK-D

La inspección de calidad de los palillos se realiza mediante una cámara OAK-D integrada dentro del entorno de simulación.

Especificaciones principales

- Modelo: Luxonis OAK-D.



- Tipo: Cámara inteligente con capacidades de IA.
- Compatibilidad: OpenCV y Python.
- Función principal: Inspección visual y clasificación.

Función en el proyecto

La cámara identifica los palillos correctos y defectuosos antes de que sean manipulados por el robot Delta. En la simulación, los palillos correctos se representan mediante un color marrón claro, mientras que los defectuosos aparecen en marrón oscuro.

Aunque actualmente la detección se basa en colores, la arquitectura permitiría implementar en el futuro algoritmos capaces de detectar grietas, roturas, deformaciones y otros defectos reales.

4.1.4 Robot Delta ABB IRB 360-1/800 4D

El ABB IRB 360-1/800 4D es un robot industrial de tipo Delta diseñado para aplicaciones Pick & Place de alta velocidad.

Especificaciones principales

- Modelo: ABB IRB 360-1/800 4D.
- Tipo: Robot Delta de 4 ejes.
- Alcance máximo: 800 mm.
- Aplicación: Manipulación rápida de piezas.
- Herramienta utilizada: RobotiQ EPick Vacuum Gripper.

Función en el proyecto

El robot recoge los palillos procedentes de la cinta de entrada y los clasifica según el resultado obtenido por el sistema de visión artificial.

Los palillos válidos son depositados sobre la cinta central para continuar el proceso, mientras que los defectuosos son enviados a la cinta de rechazo.

4.1.5 RobotiQ EPick Vacuum Gripper

Para realizar la manipulación de los palillos, el ABB IRB 360 incorpora una herramienta RobotiQ EPick Vacuum Gripper.

Especificaciones principales

- Tipo: Ventosa eléctrica de vacío.
- Tecnología: Sujeción mediante vacío.



- Aplicación: Manipulación de objetos ligeros.
- Compatibilidad: Robots industriales ABB.

Función en el proyecto

La herramienta permite recoger los palillos de forma segura y transportarlos entre las diferentes zonas de la estación sin provocar daños ni pérdidas de material.

4.1.6 Sistema de transporte

La estación incorpora tres cintas transportadoras que permiten el desplazamiento automático de los palillos entre las distintas etapas del proceso.

Cinta de entrada

Especificaciones principales

- Función: Transporte desde la tolva.
- Dirección: Hacia la zona de clasificación.

Función en el proyecto

Transporta los palillos hasta la posición donde son inspeccionados y clasificados.

Cinta central

Especificaciones principales

- Función: Transporte de productos válidos.
- Destino: Estación de serigrafía.

Función en el proyecto

Recibe los palillos correctos clasificados por el ABB IRB 360. Los productos se organizan en bandejas con capacidad para nueve palillos distribuidos en tres grupos de tres unidades.

Cinta de rechazo

Especificaciones principales

- Función: Transporte de productos defectuosos.
- Destino: Contenedor de rechazo.

Función en el proyecto



Recoge los palillos defectuosos detectados por el sistema de visión artificial y los conduce hasta el contenedor de descarte.

4.1.7 Robots colaborativos UR3e y estación de serigrafía

La estación de serigrafía está formada por dos robots colaborativos UR3e.

Especificaciones principales

- Modelo: UR3e.
- Tipo: Robot colaborativo de 6 ejes.
- Cantidad: 2 unidades.
- Aplicación: Simulación de serigrafía.

Función en el proyecto

Los robots realizan la simulación del proceso de serigrafía sobre los palillos válidos.

El primer robot procesa cinco palillos de cada lote, mientras que el segundo procesa los cuatro restantes, completando los nueve palillos contenidos en cada bandeja.

Aunque no se realiza un grabado real, ambos robots reproducen las trayectorias necesarias para simular una operación industrial de marcado o grabado láser.

4.1.8 Herramienta láser de serigrafía

La estación de serigrafía incorpora dos herramientas láser montadas sobre los robots colaborativos UR3e. Estas herramientas representan el sistema encargado de realizar el marcado o grabado de los palillos una vez han sido clasificados como válidos por el sistema de visión artificial y transportados hasta la estación de serigrafía.

Especificaciones principales

- Tipo: Herramienta láser de grabado.
- Cantidad: 2 unidades.
- Integración: Acopladas a los robots colaborativos UR3e.
- Aplicación: Marcado y personalización de productos.
- Funcionamiento: Simulado dentro del entorno RoboDK.

Función en el proyecto

La función principal de las herramientas láser es simular el proceso de serigrafía sobre los palillos correctos que llegan a la estación de marcado. Para ello, cada herramienta sigue una



trayectoria programada que representa el movimiento que realizaría un sistema real de grabado láser durante el marcado de una pieza.

En la solución desarrollada, el primer robot UR3e realiza el grabado de cinco palillos de la bandeja, mientras que el segundo robot completa la operación sobre los cuatro palillos restantes. De esta forma se procesan los nueve palillos que componen cada lote de producción.

Aunque la simulación no genera un grabado físico sobre la superficie de los palillos, sí reproduce con precisión los movimientos necesarios para representar una operación industrial real de marcado. Esto permite validar la coordinación entre robots, tiempos de ciclo y secuencias de trabajo antes de una posible implantación física.

La utilización de herramientas láser robotizadas ofrece importantes ventajas en aplicaciones industriales, ya que permite personalizar productos de forma automática, mantener una elevada precisión en los grabados y garantizar una gran repetibilidad entre piezas. Además, la automatización de este proceso reduce la necesidad de intervención manual y mejora la productividad general de la línea.

Tareas realizadas

- Simulación del proceso de serigrafía sobre los palillos válidos.
- Ejecución de trayectorias de marcado programadas.
- Personalización automática de los productos.
- Coordinación con los robots UR3e durante el proceso de grabado.
- Procesamiento completo de los nueve palillos contenidos en cada bandeja de producción.

La incorporación de estas herramientas permite representar de forma realista la fase final de personalización del producto dentro de la estación automatizada, completando así el ciclo de clasificación, manipulación y serigrafía desarrollado en el proyecto.

4.1.9 Sistema de control basado en ESP32-S3

La ESP32-S3 constituye el núcleo de control de los dispositivos auxiliares de la estación.

Especificaciones principales

- Microcontrolador de 32 bits.
- Conectividad WiFi integrada.
- Compatibilidad con MQTT.
- Múltiples entradas y salidas digitales.



Función en el proyecto

La ESP32-S3 actúa como intermediario entre la simulación desarrollada en RoboDK y los diferentes sensores y actuadores físicos.

Gestiona la recepción y envío de mensajes MQTT, controla la pantalla LCD, los LEDs indicadores y el botón de emergencia, además de supervisar los sensores conectados.

4.1.10 Sensores y actuadores auxiliares

La estación incorpora diversos sensores y actuadores auxiliares gestionados por la ESP32-S3. Estos dispositivos permiten supervisar el estado de la instalación, proporcionar información al operario y generar eventos que son transmitidos mediante MQTT al resto de componentes del sistema.

Sensor Ultrasónico

Especificaciones principales

- Tipo: Sensor de distancia por ultrasonidos.
- Aplicación: Detección de presencia y nivel.

Función en el proyecto

Se encuentra situado en el contenedor de rechazo y permite determinar si este se encuentra lleno o vacío, publicando la información correspondiente mediante MQTT.

Sensor LDR

Especificaciones principales

- Tipo: Resistencia dependiente de la luz.
- Salida: Analógica.

Función en el proyecto

Permite supervisar las condiciones de iluminación de la estación y generar avisos cuando exista demasiada o insuficiente luz.

LEDs indicadores

Especificaciones principales

- Cantidad: 2.
- Función: Señalización visual.

Función en el proyecto

Indican el estado de funcionamiento de la cinta de entrada y de la cinta de salida mediante órdenes recibidas a través de MQTT.

Pantalla LCD I2C

Especificaciones principales

- Tipo: Pantalla LCD 16x2.
- Comunicación: I2C.

Función en el proyecto

Muestra mensajes relacionados con el estado de la estación, alarmas y eventos generados por los sensores o la simulación.

Botón de emergencia

Especificaciones principales

- Tipo: Pulsador digital.
- Función: Parada de emergencia.

Función en el proyecto

Permite detener o reanudar la simulación mediante el envío de los mensajes MQTT correspondientes.

4.1.11 Vallado de seguridad

La estación incorpora un vallado perimetral alrededor de la zona de trabajo robotizada.

Especificaciones principales

- Tipo: Vallado industrial de protección.
- Ubicación: Zona central de la estación.

Función en el proyecto

Su objetivo es representar las medidas de seguridad que existirían en una instalación industrial real, evitando el acceso accidental de operarios a las zonas de movimiento de los robots.

4.1.12 Sistema de comunicaciones MQTT

Las comunicaciones entre RoboDK, Python y la ESP32-S3 se realizan mediante MQTT.

Especificaciones principales



- Protocolo: MQTT.
- Broker utilizado: EMQX.
- Comunicación: Publicación y suscripción.

Función en el proyecto

Permite el intercambio de información en tiempo real entre todos los componentes de la estación, incluyendo sensores, actuadores y sistemas de supervisión.

Además de actuar como protocolo de comunicación, MQTT permite desacoplar los distintos elementos de la estación, facilitando el intercambio de información entre dispositivos heterogéneos. Gracias a este sistema, la ESP32-S3, RoboDK y los programas desarrollados en Python pueden intercambiar eventos en tiempo real sin necesidad de conexiones directas entre todos los componentes.

Entre los principales mensajes utilizados se encuentran PARAR, INICIAR, LLENO, VACIO, MUCHA_LUZ, POCA_LUZ, ENCENDER_LED y APAGAR_LED.

4.1.13 Base de datos PostgreSQL

La información generada durante el proceso productivo se almacena en una base de datos PostgreSQL.

Especificaciones principales

- Tipo: Base de datos relacional.
- Sistema gestor: PostgreSQL.

Función en el proyecto

Registrar los palillos válidos procesados y mantener información histórica sobre la producción realizada.

La utilización de una base de datos relacional permite mantener un registro persistente de la producción realizada. Esta información puede emplearse para realizar consultas históricas, obtener estadísticas de producción y mejorar la trazabilidad de los productos procesados.

La base de datos constituye además una base para futuras ampliaciones del sistema, como la incorporación de registros de defectos, tiempos de ciclo, alarmas o indicadores avanzados de rendimiento.

4.1.14 RoboDK y Python

RoboDK constituye el entorno principal de simulación de la estación automatizada, mientras que Python se utiliza para implementar la lógica de control y coordinar las comunicaciones entre los distintos componentes.

Especificaciones principales

- RoboDK: Simulación robótica industrial.
- Python: Lenguaje de programación de alto nivel.

Función en el proyecto

Ambas herramientas permiten integrar robots, sensores, comunicaciones MQTT y bases de datos dentro de una única solución automatizada, facilitando el desarrollo, prueba y validación del sistema antes de una posible implantación industrial.

RoboDK permite validar el comportamiento de los robots, las cintas transportadoras y el flujo general de producción antes de una posible implantación física. Por su parte, Python actúa como elemento de integración entre los distintos sistemas, gestionando las comunicaciones MQTT, la interacción con la base de datos y la coordinación general de la estación.

La combinación de ambas herramientas proporciona un entorno flexible para el desarrollo, prueba y validación de soluciones de automatización industrial.

4.1.15 Diagramas de flujo

Con el objetivo de representar de forma visual el funcionamiento de la estación automatizada, se han elaborado diversos diagramas de flujo que describen tanto el proceso productivo como las comunicaciones y la lógica de control implementada.

4.1.15.1 Diagrama de flujo general del proceso

El siguiente diagrama muestra la secuencia completa de funcionamiento de la estación automatizada, desde la alimentación de los palillos mediante la tolva hasta la finalización del proceso de serigrafía. En él se representan las etapas de inspección, clasificación, transporte y marcado de los productos.

Figura 4.1 1

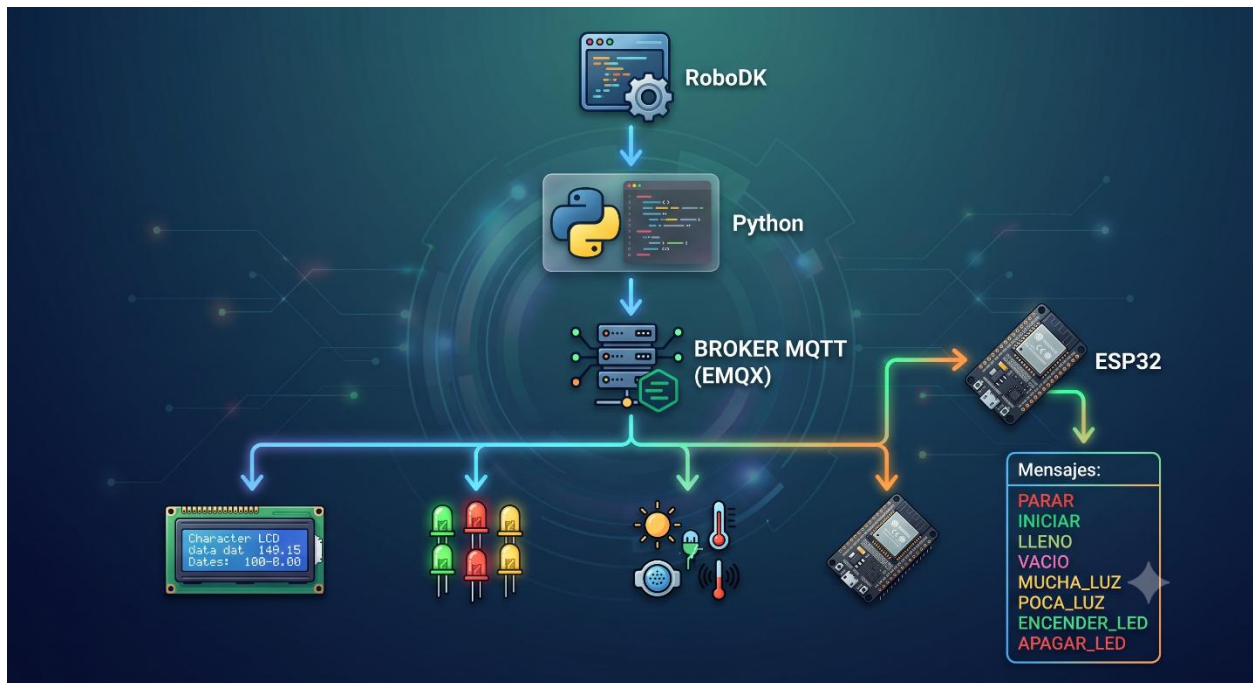


Como puede observarse, los palillos son inspeccionados por el sistema de visión artificial antes de ser clasificados por el robot ABB IRB 360. Las piezas válidas continúan hacia la estación de serigrafía, mientras que las defectuosas son enviadas al contenedor de rechazo.

4.1.15.2 Diagrama de flujo de comunicaciones MQTT

El siguiente diagrama representa la arquitectura de comunicaciones implementada en el proyecto mediante el protocolo MQTT. Se muestran los intercambios de información entre RoboDK, Python, el broker MQTT y la ESP32-S3.

Figura 4.1 2

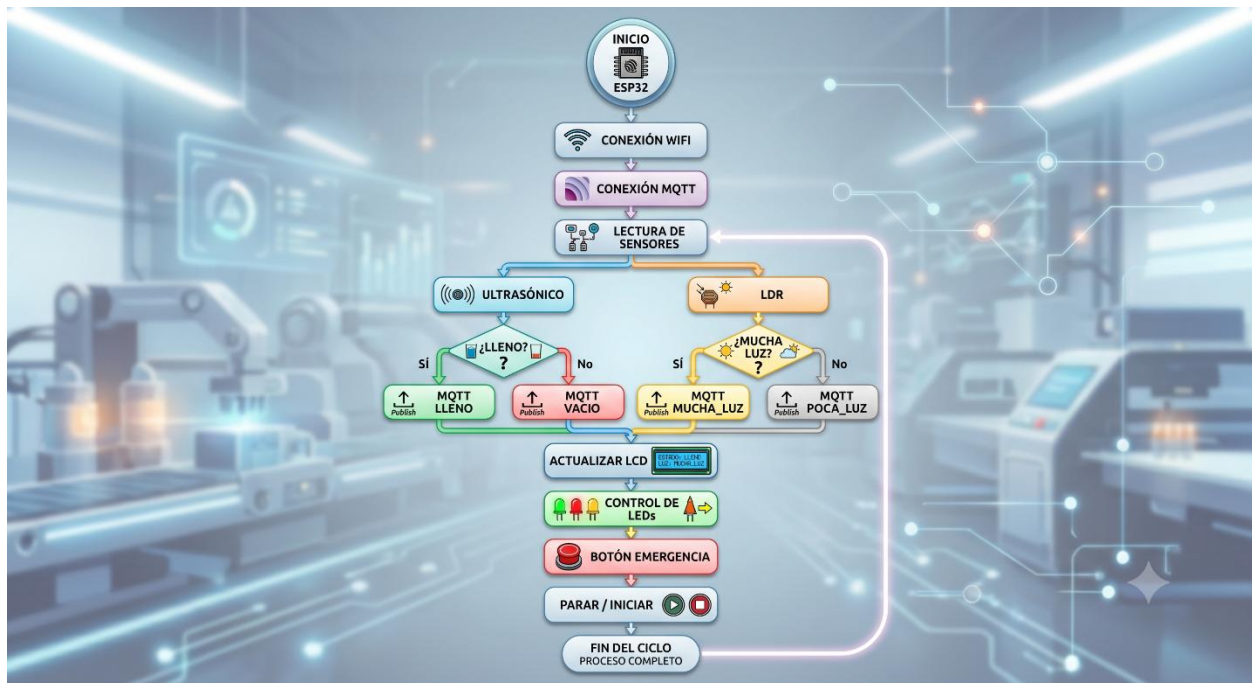


La utilización de MQTT permite desacoplar los distintos componentes de la estación y facilitar el intercambio de mensajes en tiempo real entre sensores, actuadores y sistemas de supervisión.

4.1.15.3 Diagrama de funcionamiento de la ESP32-S3

El siguiente diagrama muestra la lógica de funcionamiento de la ESP32-S3 y la interacción con los sensores y actuadores conectados a la estación.

Figura 4.1 3



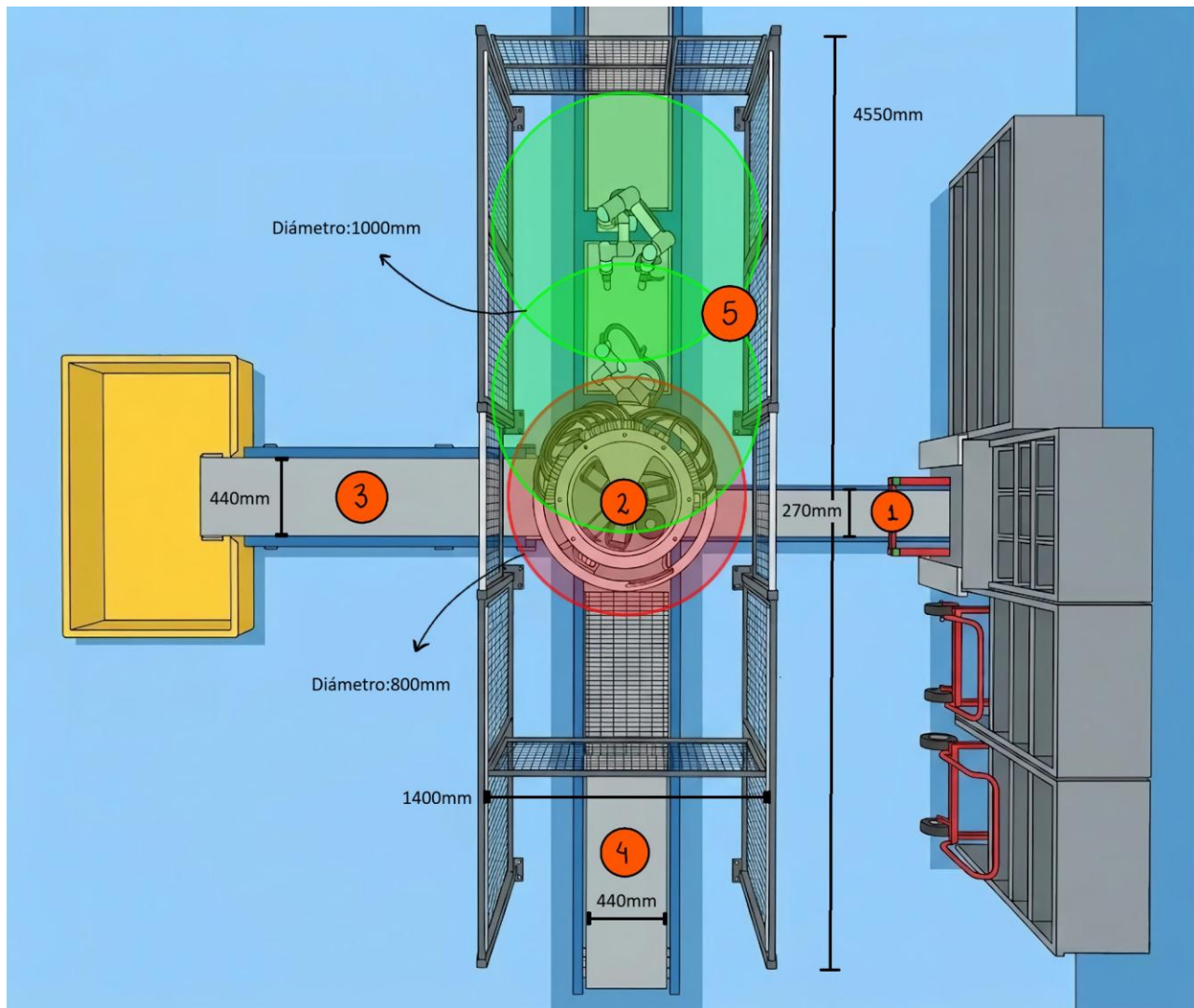
La ESP32-S3 supervisa continuamente el estado de los sensores, procesa los eventos generados y publica la información correspondiente mediante MQTT. Asimismo, controla los LEDs, la pantalla LCD y el botón de emergencia.

4.2 Distribución en Planta (Layout)

La distribución del espacio de trabajo ha sido diseñada con el objetivo de garantizar un flujo continuo de producción, minimizar los desplazamientos innecesarios de los robots y facilitar la supervisión de la instalación. La estación automatizada integra en una única célula de trabajo los sistemas de alimentación, clasificación, transporte, serigrafía y rechazo de piezas defectuosas.

La disposición general de la estación se ha desarrollado en RoboDK siguiendo criterios habituales de diseño industrial, organizando los distintos elementos de acuerdo con la secuencia lógica del proceso productivo.

Figura 4.2 1



Como puede observarse en la figura anterior, la estación presenta una longitud aproximada de 4550 mm y está dividida en varias zonas funcionales claramente diferenciadas.

4.2.1 Zona de alimentación

La zona de alimentación se encuentra situada en el extremo izquierdo de la instalación. Está formada por la tolva de alimentación y la cinta transportadora de entrada.

Su función es suministrar los palillos a la línea de producción de forma continua y transportarlos hasta la zona de inspección y clasificación.

4.2.2 Zona de clasificación

La zona de clasificación constituye el núcleo principal de la estación y está formada por la cámara OAK-D, el robot Delta ABB IRB 360 y la herramienta de vacío RobotiQ EPick.

En esta área se realiza la inspección visual de los palillos y su posterior clasificación entre productos válidos y defectuosos. El robot ABB recoge los palillos procedentes de la cinta de entrada y los distribuye entre la cinta central y la cinta de rechazo según el resultado de la inspección.

4.2.3 Zona de rechazo

La zona de rechazo está formada por la cinta transportadora de salida y el contenedor de descarte.

Los palillos clasificados como defectuosos son transportados automáticamente hasta esta zona, donde quedan almacenados para su posterior retirada. El estado del contenedor es supervisado mediante un sensor ultrasónico conectado a la ESP32-S3.

4.2.4 Zona de carga de bandejas

La zona de carga de bandejas constituye el área de interacción entre el operario y la estación automatizada. Su función es permitir la colocación y retirada de las bandejas utilizadas para transportar los palillos válidos hacia la estación de serigrafía.

Una vez que los palillos han sido clasificados correctamente por el robot ABB IRB 360, estos son depositados sobre la cinta central y organizados en bandejas con capacidad para nueve unidades distribuidas en tres grupos de tres palillos.

El operario puede introducir nuevas bandejas vacías en esta zona para que sean utilizadas durante el proceso o retirar las bandejas ya procesadas una vez finalizada la operación de grabado. De esta forma se garantiza la continuidad de la producción y se facilita la manipulación de los productos terminados.

La existencia de una zona específica para la gestión de bandejas permite separar las tareas manuales de las operaciones robotizadas, mejorando la organización del espacio de trabajo y reduciendo posibles interferencias con el funcionamiento de la estación.

4.2.5 Zona de serigrafía

La zona de serigrafía se encuentra situada en el extremo derecho de la estación y está compuesta por la cinta central, las bandejas de almacenamiento temporal y los dos robots colaborativos UR3e equipados con herramientas láser de grabado.

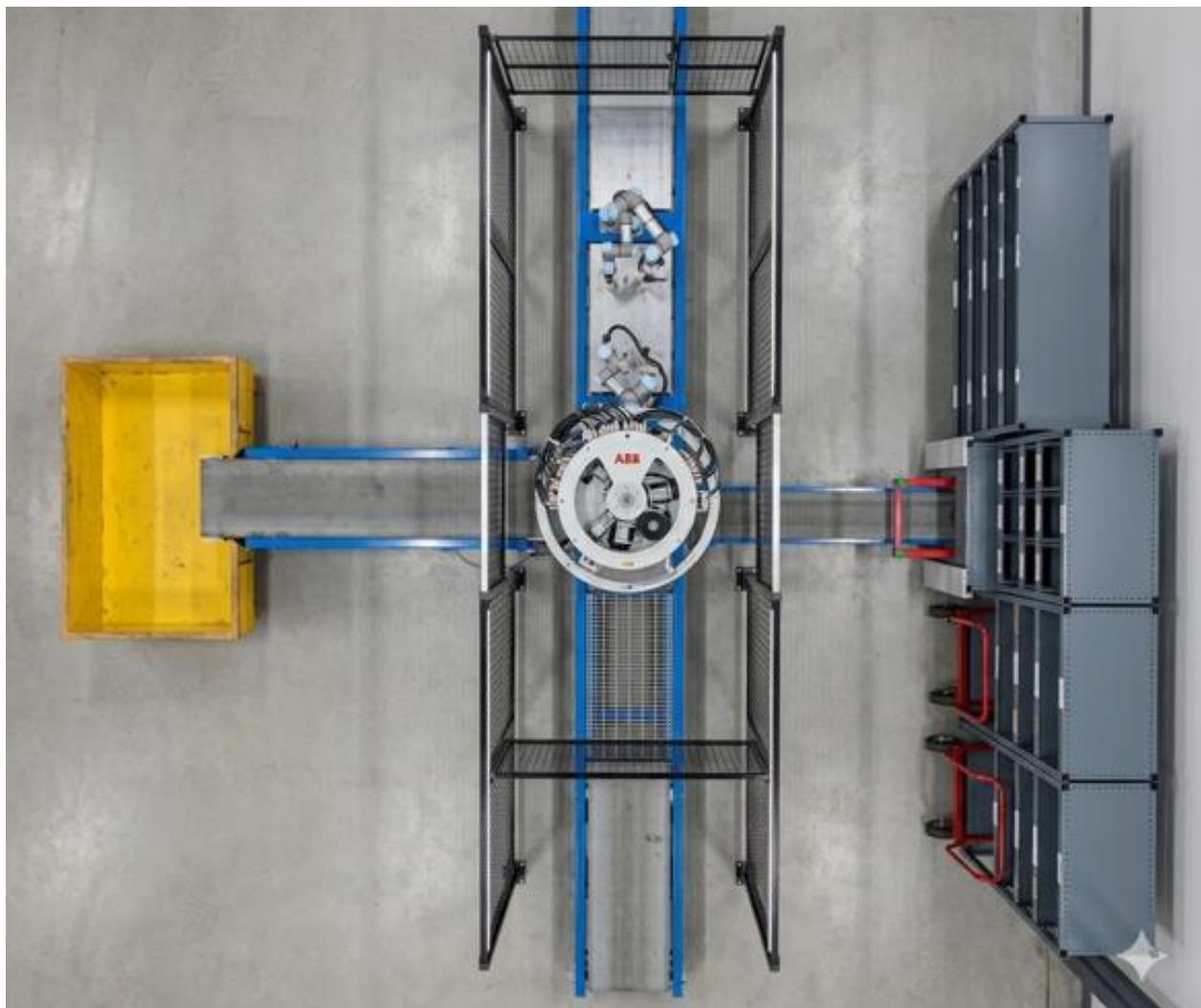
En esta zona se realiza la simulación del proceso de marcado de los palillos válidos. Los productos llegan organizados en bandejas con capacidad para nueve unidades, permitiendo que ambos robots trabajen de forma coordinada para completar el proceso de serigrafía.

4.2.6 Seguridad de la instalación

La zona robotizada se encuentra delimitada mediante un vallado de seguridad que rodea los elementos con movimiento automático. Aunque el proyecto ha sido desarrollado mediante simulación, la incorporación de este vallado permite representar una instalación industrial real y considerar aspectos relacionados con la seguridad de los operarios durante la operación de la célula robotizada.

La distribución adoptada permite mantener una secuencia de producción lineal, facilita la coordinación entre los distintos dispositivos y optimiza el flujo de materiales desde la alimentación inicial hasta la finalización del proceso de serigrafía.

Figura 4.2 2



En esta vista puede apreciarse la ubicación del robot ABB IRB 360 en la zona central de la instalación, las cintas transportadoras de entrada y rechazo, la zona de carga de bandejas y la estación de serigrafía formada por los dos robots colaborativos UR3e. Asimismo, se observa el vallado de seguridad que delimita el área de trabajo robotizada.

4.3 Estrategia de Persistencia y Datos

4.3.1 Utilización de buffers para la gestión de eventos

Durante el desarrollo del sistema se implementó un buffer en la ESP32-S3 con el objetivo de gestionar de forma ordenada los eventos generados por los sensores, actuadores y mensajes MQTT.

Este buffer permite almacenar temporalmente la información recibida antes de ser procesada por el programa principal, evitando pérdidas de datos cuando varios eventos se producen de forma simultánea o en intervalos muy cortos de tiempo.

La utilización de esta estructura de datos mejora la sincronización entre las diferentes tareas ejecutadas por la ESP32-S3 y garantiza una gestión más eficiente de los mensajes intercambiados con RoboDK y el broker MQTT.

Además, el uso de un buffer facilita la escalabilidad del sistema, ya que permite incorporar nuevos eventos o dispositivos sin modificar significativamente la lógica general de funcionamiento.

4.3.2 Programación orientada a objetos

La lógica de funcionamiento de la estación se basa principalmente en un modelo de programación orientado a eventos. En este enfoque, las acciones del sistema se ejecutan como respuesta a sucesos concretos generados por sensores, pulsadores o mensajes MQTT.

Por ejemplo, la pulsación del botón de emergencia genera un evento que provoca la publicación del mensaje PARAR, mientras que los sensores pueden generar eventos asociados a los estados LLENO, VACIO, MUCHA_LUZ o POCA_LUZ.

Esta estrategia permite que el sistema responda en tiempo real a los cambios producidos durante la ejecución de la estación y facilita la coordinación entre los diferentes componentes que intervienen en el proceso.

4.4 Persistencia de Datos y Base de Datos Relacional

Para garantizar la trazabilidad del proceso productivo, la solución desarrollada incorpora una base de datos relacional implementada en PostgreSQL. Esta base de datos permite almacenar información relacionada con los palillos procesados, los procesos de producción, los robots que intervienen, los diseños de serigrafía, el stock, los clientes, los proveedores y las compras.

La utilización de una base de datos relacional resulta adecuada para este proyecto, ya que permite organizar la información en tablas relacionadas entre sí mediante claves primarias y claves ajenas. De esta forma se garantiza la integridad de los datos y se facilita la consulta posterior de la información almacenada.

4.4.1 Modelo relacional utilizado

El modelo relacional diseñado contempla diferentes entidades relacionadas con el proceso de fabricación y serigrafía de palillos. Las tablas principales son las siguientes:

Tabla	Función principal
PROVEEDORES	Almacena información de los proveedores de materia prima.
TELEFONOS_PROV	Registra los teléfonos asociados a cada proveedor.
COMPRA	Almacena información sobre las compras realizadas.
CANTIDAD_COMPRA	Registra las cantidades asociadas a cada compra.
PALILLO	Almacena los datos de cada palillo procesado.
STOCK	Gestiona la cantidad de unidades disponibles.
SERIGRAFIADO	Almacena información sobre el diseño o marcado aplicado.
CLIENTES	Registra información de los clientes.
TELEFONOS_CLIEN	Registra los teléfonos asociados a cada cliente.
PRODUCCIÓN	Almacena los procesos productivos realizados.
ROBOT	Registra información sobre los robots utilizados.
INTERVENCION_ROBOT	Relaciona robots con procesos y tareas realizadas.

4.4.2 Tabla PALILLO

La tabla **PALILLO** es una de las tablas centrales del modelo, ya que almacena la información correspondiente a los palillos procesados en la línea. Cada palillo cuenta con un identificador único y queda asociado a un proceso de producción.

Campo	Descripción
código	Identificador único del palillo.
forma	Forma del palillo: alargado, ovalado o pequeño.
material_madera	Tipo de madera: reciclado, roble, pino o cerezo.
grabado	Tipo de grabado aplicado: estándar o decorativo.
cód_compra	Compra asociada al palillo.
cód_stock	Stock asociado.
cód_serigrafiado	Diseño de serigrafía asociado.
cód_proceso	Proceso de producción al que pertenece.

Esta tabla permite relacionar cada palillo con su origen, su proceso de fabricación y su posible diseño de serigrafía.

4.4.3 Tabla PRODUCCIÓN

La tabla **PRODUCCIÓN** almacena la información relacionada con cada proceso productivo ejecutado por la estación automatizada.

Campo	Descripción
cód_proceso	Identificador único del proceso.
fecha_inicio	Fecha de inicio del proceso.
fecha_fin	Fecha de finalización.
estado	Estado del proceso: en_curso, finalizado o error.

Esta tabla permite registrar los distintos ciclos de producción y conocer el estado asociado a cada uno de ellos.

4.4.4 Tabla ROBOT e INTERVENCION_ROBOT

La tabla **ROBOT** almacena información sobre los robots que intervienen en la línea automatizada.



Campo	Descripción
cód_robot	Identificador del robot.
tipo	Tipo de robot: delta o ur3e.
modelo	Modelo del robot utilizado.
estado	Estado del robot: activo, mantenimiento o error.
fecha_mantenimiento	Fecha de la última revisión.

Por su parte, la tabla **INTERVENCION_ROBOT** permite relacionar los robots con los procesos productivos en los que participan.

Campo	Descripción
cód_proceso	Identificador de la intervención.
tiempo_inicio	Inicio de la intervención.
tiempo_fin	Fin de la intervención.
tarea	Tarea realizada: clasificar, transportar, serigrafiar o desechar.
cód_producción	Proceso de producción asociado.
cód_robot	Robot que realiza la intervención.

Gracias a estas tablas es posible conocer qué robot ha intervenido en cada proceso y qué tarea ha realizado.

4.4.5 Tabla SERIGRAFIADO

La tabla **SERIGRAFIADO** almacena la información relacionada con los diseños o marcados aplicados a los palillos.

Campo	Descripción
código	Identificador del diseño.

Campo	Descripción
letra	Tipo de letra utilizado.
tamaño	Tamaño del marcado.
título	Título del diseño.
símbolo	Símbolo o elemento gráfico.
cód_cliente	Cliente asociado.
fecha_venta	Fecha de venta.
cantidad	Cantidad vendida.
precio	Precio de venta.
valoración	Valoración del cliente.
cód_robot	Robot asociado al proceso de serigrafía.

Esta tabla permite relacionar el diseño aplicado con el cliente, la venta y el robot que interviene en el proceso.

4.4.6 Tablas de clientes, proveedores, compras y stock

Además de las tablas directamente relacionadas con la producción, el modelo incluye tablas destinadas a la gestión comercial y logística.

La tabla **CLIENTES** almacena los datos de los clientes, mientras que **TELEFONOS_CLIEN** permite registrar varios teléfonos asociados a un mismo cliente.

De forma similar, la tabla **PROVEEDORES** contiene información sobre los proveedores de materia prima y **TELEFONOS_PROV** almacena sus números de contacto.

Las tablas **COMPRA** y **CANTIDAD_COMPRA** permiten gestionar las compras realizadas, mientras que la tabla **STOCK** registra la cantidad de unidades disponibles y la fecha de actualización.

Estas tablas permiten ampliar el sistema más allá del proceso automatizado, incorporando información útil para la gestión global de la empresa.

4.4.7 Funcionamiento de la persistencia desde Python

La conexión con la base de datos se realiza desde Python mediante la librería **psycopgp**. El programa establece conexión con la base de datos PostgreSQL, selecciona el esquema correspondiente y realiza inserciones en las tablas necesarias.

En la implementación actual, el programa crea un nuevo proceso productivo en la tabla **PRODUCCIÓN** y posteriormente inserta un nuevo palillo asociado a dicho proceso en la tabla **PALILLO**.

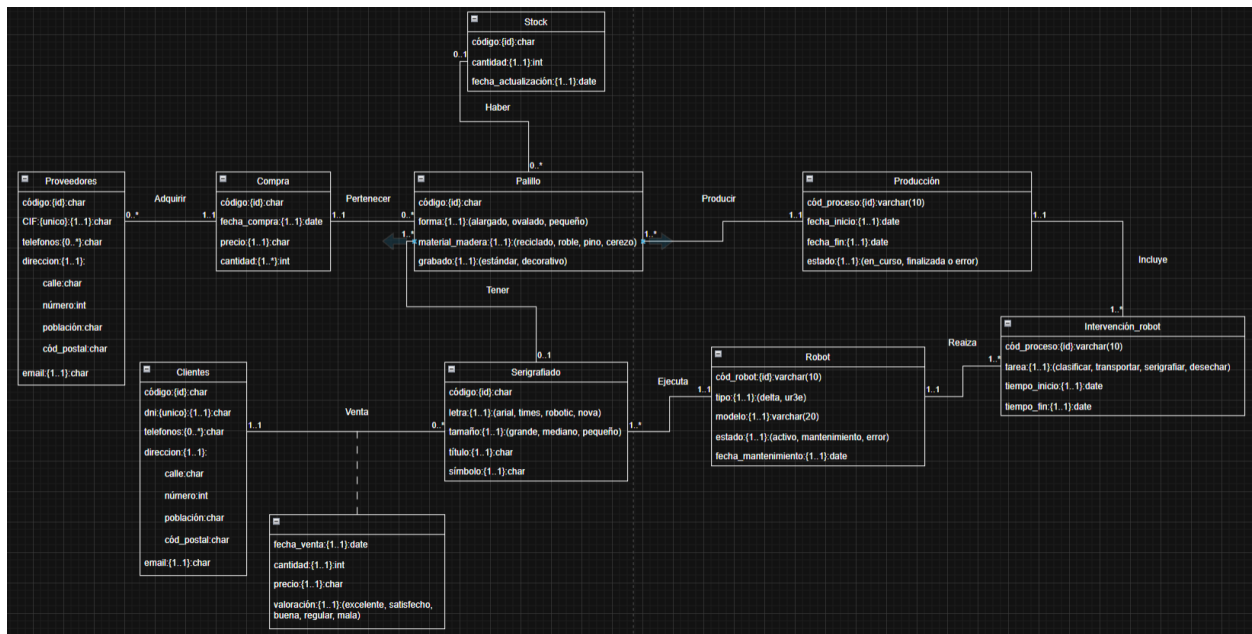
Para generar identificadores únicos se emplea la librería **uuid**, permitiendo crear códigos de proceso y códigos de palillo no repetidos. Además, algunos valores como la forma del palillo, el material de madera y el estado del proceso se generan de forma aleatoria para simular diferentes situaciones de producción.

Este funcionamiento permite comprobar que la base de datos almacena correctamente la información generada durante la simulación y que la relación entre procesos y palillos se mantiene de forma adecuada.

4.4.8 Diagrama del modelo relacional

El modelo de datos puede representarse mediante un diagrama entidad-relación en el que se muestran las tablas principales y las relaciones existentes entre ellas.

Figura 4.4 1



[Enlace Imagen Diagrama Base de Datos Relacional](#)

Como puede observarse en la figura anterior, la entidad **PALILLO** constituye uno de los elementos centrales del modelo de datos, ya que se relaciona con los procesos de producción, las operaciones de serigrafía y la gestión de stock. Del mismo modo, las entidades **ROBOT** e **INTERVENCION_ROBOT** permiten registrar la participación de los distintos robots en cada fase del proceso productivo.

La utilización de un modelo relacional facilita la integridad de los datos mediante el uso de claves primarias y claves ajenas, garantizando la consistencia de la información almacenada.

4.4.9 Posibles ampliaciones futuras

Aunque el modelo diseñado ya permite registrar información relevante del proceso, en futuras versiones podría ampliarse para incorporar nuevos datos, como tiempos de ciclo, número de piezas defectuosas, alarmas generadas, consumo energético o estadísticas de rendimiento.

Estas ampliaciones permitirían mejorar el análisis de la producción y facilitar la toma de decisiones basada en datos, acercando el sistema a un entorno de Industria 4.0 más completo.

4.5 Arquitectura de Integración

La solución desarrollada utiliza una arquitectura distribuida basada en el protocolo MQTT, que permite integrar la simulación robótica realizada en RoboDK, el sistema físico basado en ESP32-S3 y la base de datos PostgreSQL mediante un mecanismo de comunicación indirecta.

La comunicación se centraliza en un broker MQTT, encargado de recibir, gestionar y distribuir los mensajes intercambiados entre los distintos componentes del sistema. Gracias a esta arquitectura, los dispositivos no necesitan comunicarse directamente entre sí, sino que publican y reciben información a través de topics específicos.

Python actúa como elemento central de integración, funcionando como intermediario entre RoboDK y MQTT. Este componente se encarga de procesar los mensajes recibidos, actualizar el estado de la simulación y registrar información relevante del proceso productivo.

La arquitectura desarrollada permite desacoplar los distintos módulos de la aplicación, facilitando el mantenimiento, la escalabilidad y la incorporación de nuevos dispositivos en futuras ampliaciones del proyecto.

4.5.1 Componentes de la arquitectura de integración

La arquitectura de integración está formada por los siguientes componentes:

Componente	Función dentro de la arquitectura
ESP32-S3	Gestiona sensores, actuadores, pantalla LCD, LEDs y botón de emergencia.

Broker MQTT	Centraliza la publicación y recepción de mensajes entre dispositivos.
Python	Actúa como backend de integración entre RoboDK, MQTT y la base de datos.
RoboDK	Ejecuta la simulación robótica y publica eventos de la estación.
PostgreSQL	Almacena información del proceso productivo.
Sensores	Generan eventos físicos como nivel de contenedor o iluminación.
Actuadores	Ejecutan acciones físicas como encendido de LEDs o visualización en LCD.

Esta división permite organizar el sistema de forma modular. Cada componente cumple una función específica y se comunica con el resto mediante mensajes MQTT.

4.5.2 Flujo general de comunicación

El flujo general de comunicación del sistema se puede resumir en los siguientes pasos:

1. Los sensores conectados a la ESP32-S3 detectan eventos físicos.
2. La ESP32-S3 publica mensajes MQTT en el topic correspondiente.
3. El broker MQTT recibe y distribuye los mensajes.
4. Python recibe los mensajes y ejecuta la lógica correspondiente.
5. RoboDK actualiza la simulación en función del evento recibido.
6. RoboDK puede publicar nuevos eventos hacia MQTT.
7. La ESP32-S3 recibe comandos y actúa sobre LEDs, pantalla LCD o estados internos.

Este flujo permite establecer una comunicación bidireccional entre la parte física y la simulación virtual, consiguiendo que los eventos físicos puedan afectar al comportamiento de RoboDK y que los eventos de la simulación puedan generar acciones sobre los dispositivos conectados a la ESP32-S3.

4.5.3 Topics MQTT utilizados

Para organizar la comunicación se han definido diferentes topics MQTT. Cada topic tiene una función concreta dentro del sistema y permite separar los mensajes según su destino o finalidad.

Topic MQTT	Descripción / Proceso
------------	-----------------------

giirob/pr2/devices/input/lcd	Topic en el cual nos suscribimos desde la ESP32-S3 para recibir el mensaje que debe mostrarse en la pantalla LCD. El contenido es enviado desde RoboDK mediante los scripts desarrollados en Python.
giirob/pr2/devices/input/led	Topic utilizado para controlar el encendido y apagado de los LEDs indicadores de la cinta transportadora entrada. La ESP32-S3 recibe los mensajes ENCENDER_LED y APAGAR_LED publicados desde RoboDK.
giirob/pr2/devices/input/led2	Topic utilizado para controlar el encendido y apagado de los LEDs indicadores de la cinta transportadora salida. La ESP32-S3 recibe los mensajes ENCENDER_LED y APAGAR_LED publicados desde RoboDK.
giirob/pr2/devices/output	Topic utilizado por la ESP32-S3 para publicar los eventos generados por el sensor LDR, el sensor ultrasónico y el botón de emergencia. Python y RoboDK se encuentran suscritos a este topic.

La utilización de topics independientes permite organizar las comunicaciones y facilita la ampliación futura del sistema mediante nuevos sensores, actuadores o dispositivos conectados al broker MQTT.

4.5.4 Mensajes intercambiados

Los mensajes enviados mediante MQTT representan eventos o acciones específicas dentro del sistema automatizado.

Mensaje	Función
PARAR	Detiene completamente la estación mediante el botón de emergencia.
INICIAR	Reanuda el funcionamiento de la estación tras una parada.
LLENO	Indica que el contenedor de rechazo ha alcanzado su capacidad máxima.
VACIO	Indica que el contenedor de rechazo dispone de capacidad libre.
MUCHA_LUZ	Señala un nivel elevado de iluminación detectado por el sensor LDR.

POCA_LUZ	Señala una iluminación insuficiente para el correcto funcionamiento de la cámara de visión artificial.
ENCENDER_LED	Activa los LEDs indicadores asociados a las cintas transportadoras.
APAGAR_LED	Desactiva los LEDs indicadores.
Texto LCD	Mensaje mostrado en la pantalla LCD de la estación.

Estos mensajes permiten coordinar el funcionamiento de la instalación y sincronizar el comportamiento de los dispositivos físicos con la simulación desarrollada en RoboDK.

4.5.5 Publicadores y suscriptores

El sistema sigue el modelo publicación/suscripción característica de MQTT.

Dispositivo	Rol MQTT
ESP32-S3	Publicador y suscriptor.
Python	Publicador y suscriptor.
RoboDK	Publicador indirecto mediante Python.
Broker MQTT	Intermediario de comunicaciones.

La ESP32-S3 publica los eventos generados por sensores y pulsadores, mientras que RoboDK publica órdenes destinadas a los actuadores. Python actúa como intermediario procesando la información y coordinando la comunicación entre ambos entornos.

Gracias a esta arquitectura ningún componente necesita conocer directamente la implementación interna del resto, ya que toda la comunicación se realiza a través del broker MQTT.

4.5.6 Integración RoboDK, Python y MQTT

La integración entre RoboDK y MQTT se realiza mediante scripts desarrollados en Python. Estos scripts permiten conectar la simulación con el broker MQTT, recibir mensajes procedentes de la ESP32-S3 y modificar el estado de la estación en RoboDK.

Por ejemplo, cuando Python recibe el mensaje PARAR, actualiza el parámetro correspondiente en RoboDK para detener la estación. Si recibe el mensaje INICIAR, la simulación vuelve a ejecutarse de forma normal.



De forma similar, mensajes como LLENO, VACIO, MUCHA_LUZ o POCA_LUZ generan avisos relacionados con el estado de la estación y permiten supervisar las condiciones de funcionamiento durante la ejecución de la simulación.

Python también se encarga de publicar mensajes hacia la ESP32-S3 para controlar elementos físicos como la pantalla LCD o los LEDs indicadores.

4.5.7 Integración ESP32-S3 y MQTT

La ESP32-S3 establece conexión con la red WiFi y posteriormente con el broker MQTT. Una vez conectada, se suscribe a los topics destinados al control de LEDs y pantalla LCD.

Cuando recibe mensajes como ENCENDER_LED o APAGAR_LED, activa o desactiva los LEDs correspondientes. Cuando recibe un mensaje destinado a la pantalla LCD, este se muestra automáticamente al operario.

Además, la ESP32-S3 monitoriza continuamente el estado de sus sensores. Cuando detecta un cambio relevante publica el mensaje correspondiente mediante MQTT. El botón de emergencia genera los mensajes PARAR e INICIAR, el sensor ultrasónico genera los estados LLENO y VACIO, mientras que el sensor LDR publica los eventos MUCHA_LUZ y POCA_LUZ.

4.5.8 Gestión de errores y casos especiales

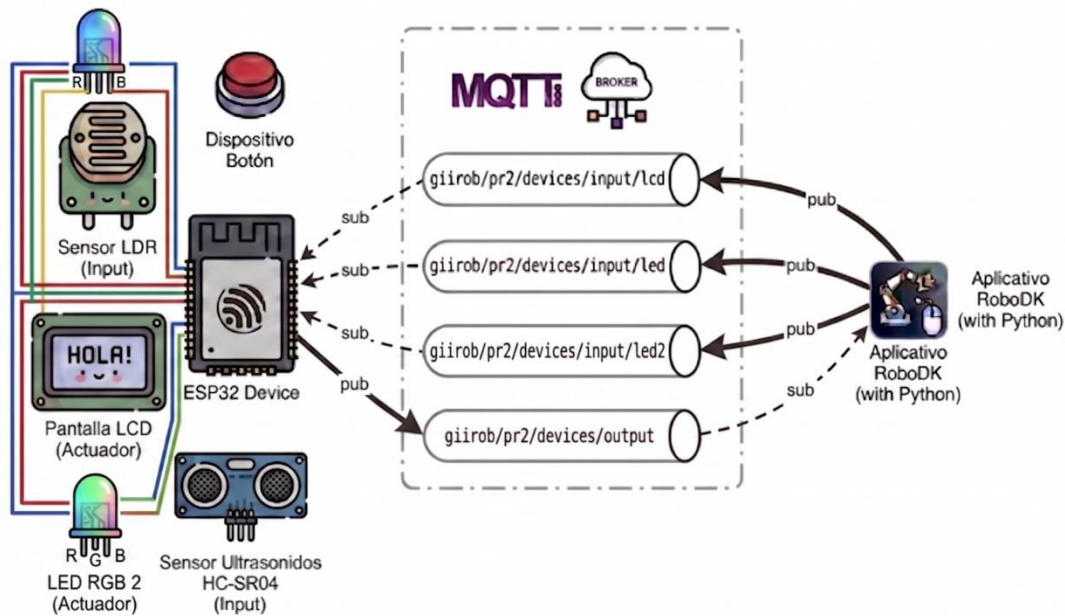
La arquitectura contempla diferentes situaciones que pueden producirse durante el funcionamiento del sistema.

En caso de pérdida de conexión WiFi, la ESP32-S3 intenta reconectarse automáticamente al broker MQTT. Del mismo modo, si se produce una interrupción temporal del servicio MQTT, el sistema intenta restablecer la comunicación para recuperar el funcionamiento normal de la estación.

La arquitectura también contempla eventos relacionados con la producción, como el llenado del contenedor de rechazo, condiciones inadecuadas de iluminación o la activación del botón de emergencia. Estos eventos son transmitidos inmediatamente mediante MQTT para permitir una respuesta rápida por parte del sistema.

4.5.9 Diagrama de arquitectura de integración

Figura 4.5 1



Como puede observarse en la figura anterior, la ESP32-S3, RoboDK y Python intercambian información a través del broker MQTT utilizando una arquitectura de publicación y suscripción. Este enfoque permite desacoplar los distintos componentes y facilita la incorporación de nuevos dispositivos en futuras ampliaciones del proyecto.

4.5.10 Justificación de la arquitectura seleccionada

La arquitectura basada en MQTT ha sido seleccionada por su simplicidad, modularidad y facilidad de integración entre dispositivos heterogéneos. Este protocolo permite conectar sistemas embebidos, aplicaciones software y simulaciones robóticas sin necesidad de establecer conexiones directas entre todos ellos.

Además, el modelo publicación/suscripción facilita la escalabilidad de la solución, ya que permite incorporar nuevos sensores, actuadores o estaciones de trabajo mediante la creación de nuevos topics sin modificar la arquitectura existente.

Por todo ello, la solución propuesta resulta adecuada para entornos de automatización industrial e Industria 4.0, donde es necesario integrar dispositivos físicos y virtuales de forma flexible y eficiente.

4.6 Diseño eléctrico y conexionado

Con el objetivo de integrar los sensores y actuadores utilizados en el proyecto, se ha diseñado un sistema electrónico basado en una placa ESP32-S3. Este dispositivo actúa como nodo de control

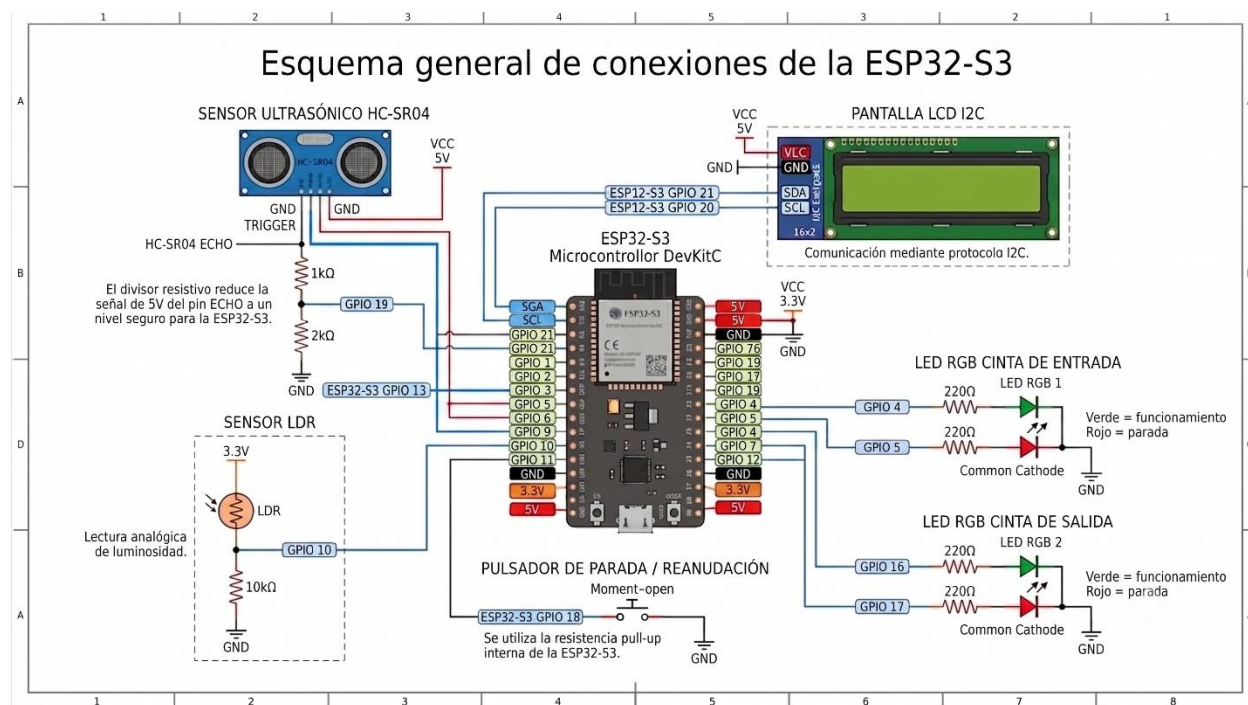
físico e interfaz IoT de la estación automatizada, permitiendo la lectura de sensores, el control de actuadores y el intercambio de información mediante MQTT.

La ESP32-S3 se encarga de gestionar los dispositivos auxiliares de la instalación, como la pantalla LCD, los LEDs indicadores, el sensor ultrasónico, el sensor LDR y el pulsador de parada de emergencia. Estos elementos permiten supervisar el estado de la estación, mostrar información al operario y generar eventos que son enviados al resto del sistema mediante comunicación MQTT.

En los siguientes apartados se describe el conexionado eléctrico de los principales dispositivos conectados a la ESP32-S3.

4.6.1 Esquema general de conexiones

Figura 4.6 1



[Enlace Esquema Eléctrico ESP32S3 General](#)

Nota: El esquema mostrado tiene carácter orientativo y representa la distribución funcional de las conexiones empleadas en el proyecto.

El esquema general muestra la interconexión entre la placa ESP32-S3 y los distintos dispositivos electrónicos utilizados para la supervisión y control de la estación automatizada. La ESP32-S3 actúa como nodo central del sistema IoT, siendo responsable de la adquisición de señales procedentes de los sensores y del control de los actuadores conectados.

Entre los dispositivos integrados se encuentran una pantalla LCD con interfaz I2C utilizada para mostrar información al operario, un sensor ultrasónico HC-SR04 destinado a la detección del nivel de llenado del contenedor de rechazo, un sensor LDR encargado de medir las condiciones de iluminación ambiental, dos LEDs RGB empleados para indicar visualmente el estado de las cintas transportadoras y un pulsador utilizado para la función de parada y reanudación de la estación.

Asimismo, el esquema incorpora los elementos electrónicos auxiliares necesarios para garantizar el correcto funcionamiento del sistema, como resistencias limitadoras de corriente para los LEDs, un divisor resistivo asociado al pin ECHO del sensor ultrasónico y la resistencia utilizada en el divisor de tensión del sensor LDR.

Toda la información adquirida por la ESP32-S3 es procesada local y posteriormente intercambiada con RoboDK y los módulos software desarrollados en Python mediante el protocolo MQTT, permitiendo la integración entre el entorno físico y el entorno virtual de la estación automatizada.

Esta arquitectura centralizada simplifica el conexionado del sistema, facilita las tareas de mantenimiento y permite ampliar la instalación incorporando nuevos sensores o actuadores sin modificar significativamente la estructura general de comunicaciones.

4.6.2 Pantalla LCD I2C

La estación automatizada incorpora una pantalla LCD 16x2 con interfaz I2C conectada a la ESP32-S3. Este dispositivo se utiliza como elemento de interacción visual con el operario, permitiendo mostrar información relevante sobre el estado del sistema y los procesos ejecutados en la estación.

La información mostrada en la pantalla es enviada desde RoboDK a través de los scripts desarrollados en Python mediante mensajes MQTT. Una vez recibidos por la ESP32-S3, estos mensajes son procesados y mostrados en tiempo real en la pantalla LCD.

La comunicación entre la ESP32-S3 y la pantalla se realiza mediante el protocolo I2C, utilizando únicamente dos líneas de comunicación, SDA (datos) y SCL (reloj), lo que reduce el número de pines necesarios respecto a una conexión paralela convencional.

Las conexiones empleadas son las siguientes:

Señal LCD	ESP32-S3
VCC	5V
GND	GND

SDA	GPIO 21
SCL	GPIO 20

La pantalla LCD recibe mensajes procedentes del sistema de control mediante MQTT, permitiendo mostrar información relevante para el operario durante la ejecución de la estación automatizada.

Los principales eventos MQTT utilizados son los siguientes:

Evento MQTT	Descripción
MENSAJE_LCD	Texto enviado desde RoboDK para ser mostrado en la pantalla LCD.

Estos mensajes permiten informar al operario sobre el estado de la producción y mostrar el contenido que posteriormente será grabado sobre los palillos durante el proceso de marcado.

Gracias a esta configuración es posible actualizar dinámicamente el contenido mostrado en la pantalla sin afectar al funcionamiento del resto de tareas ejecutadas por la ESP32-S3. Entre los mensajes mostrados se incluyen textos relacionados con el estado de la producción, información enviada desde RoboDK y avisos generados durante la ejecución de la estación automatizada.

La utilización de una interfaz I2C simplifica el cableado del sistema, mejora la organización del montaje electrónico y facilita futuras ampliaciones o sustituciones del dispositivo de visualización.

4.6.3 Sensor ultrasónico HC-SR04

La estación automatizada incorpora un sensor ultrasónico HC-SR04 utilizado para supervisar el nivel de llenado del contenedor destinado a almacenar los palillos defectuosos o descartados durante el proceso de clasificación.

El funcionamiento del sensor se basa en la emisión de pulsos ultrasónicos y la medición del tiempo necesario para recibir el eco reflejado por los objetos presentes frente al sensor. A partir de este tiempo es posible calcular la distancia existente entre el sensor y el contenido del contenedor.

La ESP32-S3 realiza periódicamente las mediciones de distancia y, en función del valor obtenido, determina si el contenedor dispone de capacidad disponible o si ha alcanzado su nivel máximo de llenado. Esta información se transmite posteriormente mediante MQTT al resto de componentes del sistema.

Debido a que el pin ECHO del HC-SR04 puede generar señales de tensión superiores a las admitidas por la ESP32-S3, se emplea un divisor resistivo formado por resistencias de 1 kΩ y 2 kΩ para adaptar el nivel de tensión y garantizar una comunicación segura entre ambos dispositivos.

Las conexiones utilizadas son las siguientes:

Señal HC-SR04	ESP32-S3
VCC	5V
GND	GND
TRIGGER	GPIO 13
ECHO	GPIO 19 (mediante divisor resistivo)

La ESP32-S3 procesa las mediciones obtenidas por el sensor ultrasónico y genera distintos eventos MQTT en función del nivel de llenado detectado en el contenedor de rechazo.

Los principales eventos MQTT utilizados son los siguientes:

Evento MQTT	Descripción
LLENO	Contenedor lleno
VACIO	Contenedor vacío

Estos eventos permiten supervisar automáticamente el estado del contenedor y generar avisos cuando sea necesario realizar su vaciado.

La utilización de este sensor permite supervisar automáticamente el estado del contenedor de rechazo y generar avisos cuando sea necesario vaciarlo, evitando interrupciones no planificadas durante el funcionamiento de la estación automatizada.

4.6.4 Sensor LDR

La estación automatizada incorpora un sensor LDR (Light Dependent Resistor) cuya función es medir el nivel de iluminación existente en el entorno de trabajo. Este sensor permite simular sistemas de supervisión ambiental utilizados habitualmente en instalaciones industriales para detectar condiciones anómalas de iluminación.



El funcionamiento del dispositivo se basa en la variación de su resistencia eléctrica en función de la cantidad de luz recibida. Cuando la iluminación aumenta, la resistencia de la LDR disminuye; por el contrario, cuando la iluminación disminuye, su resistencia aumenta.

Para poder realizar la lectura mediante la ESP32-S3, la LDR se conecta formando un divisor de tensión junto con una resistencia de 10 kΩ. De esta forma, la variación de resistencia se transforma en una señal analógica que puede ser leída por el microcontrolador.

Las conexiones utilizadas son las siguientes:

Elemento	ESP32-S3
Salida divisor LDR	GPIO 10
Alimentación	3.3 V
Resistencia asociada	10 kΩ

La ESP32-S3 realiza lecturas periódicas del sensor y compara los valores obtenidos con unos umbrales previamente definidos. En función de estos valores, el sistema puede generar distintos eventos MQTT que informan sobre las condiciones de iluminación detectadas.

Los principales mensajes generados son:

Evento MQTT	Descripción
MUCHA_LUZ	Disminuir luz
POCA_LUZ	Aumentar luz

Estos eventos son posteriormente recibidos por los módulos software desarrollados en Python, permitiendo registrar información sobre las condiciones ambientales y adaptar el comportamiento del sistema cuando sea necesario.

La incorporación de este sensor demuestra la capacidad de la arquitectura desarrollada para integrar dispositivos de adquisición de datos y transmitir información en tiempo real mediante tecnologías IoT.

4.6.5 LEDs RGB indicadores

La estación automatizada incorpora dos LEDs RGB utilizados como elementos de señalización visual. Estos dispositivos permiten representar de forma sencilla el estado de funcionamiento de las cintas transportadoras presentes en el sistema, facilitando la supervisión del proceso por parte del operario.

Se emplea un LED RGB para la cinta de entrada y otro para la cinta de salida. En ambos casos únicamente se utilizan los canales rojo y verde, permitiendo indicar diferentes estados de operación mediante cambios de color.

Las conexiones utilizadas son las siguientes:

LED RGB	Función	ESP32-S3
Canal verde entrada	Estado de funcionamiento	GPIO 4
Canal rojo-entrada	Estado de parada	GPIO 5
Canal verde salida	Estado de funcionamiento	GPIO 16
Canal rojo-salida	Estado de parada	GPIO 17

Cada canal LED incorpora una resistencia limitadora de corriente de 220 Ω con el objetivo de proteger tanto el LED como los pines de salida de la ESP32-S3.

La activación de los LEDs se realiza mediante mensajes MQTT enviados desde RoboDK. La ESP32-S3 recibe dichos mensajes y modifica el estado de los LEDs en función de la información recibida.

Los principales eventos MQTT utilizados son los siguientes:

Evento MQTT	Descripción
ENCENDER_LED	Activa el color verde del LED correspondiente, indicando funcionamiento normal.
APAGAR_LED	Activa el color rojo del LED correspondiente, indicando parada o estado inactivo.

Gracias a este sistema de señalización visual, el operario puede identificar rápidamente el estado de las distintas zonas de la estación automatizada sin necesidad de consultar otros dispositivos de supervisión.

4.6.6 Pulsador de parada y reanudación

La estación automatizada incorpora un pulsador conectado a la ESP32-S3 que permite controlar el estado de funcionamiento del sistema. Este dispositivo actúa como elemento de interacción directa con el operario, permitiendo detener temporalmente la estación o reanudar su funcionamiento cuando sea necesario.

El pulsador se encuentra conectado a una entrada digital de la ESP32-S3 y utiliza la resistencia pull-up interna del microcontrolador, por lo que no es necesario incorporar resistencias externas adicionales en el circuito. En condiciones normales el pin permanece en estado lógico alto y, cuando el pulsador es accionado, pasa temporalmente a estado lógico bajo.

Las conexiones utilizadas son las siguientes:

Elemento	ESP32-S3
Pulsador	GPIO 18
Referencia	GND

La ESP32-S3 supervisa continuamente el estado del pulsador y genera distintos eventos MQTT en función de la acción realizada por el operario.

Los principales eventos MQTT utilizados son los siguientes:

Evento MQTT	Descripción
PARAR	Solicita la detención temporal de la estación automatizada.
INICIAR	Permite reanudar el funcionamiento de la estación tras una parada previa.

Estos eventos son procesados por los módulos software desarrollados en Python y por los scripts de RoboDK, modificando el estado global del sistema y permitiendo detener o reanudar la ejecución de los diferentes procesos de forma controlada.

La incorporación de este pulsador proporciona una interfaz sencilla para el control de la estación y permite simular mecanismos de interacción habituales en instalaciones industriales automatizadas.

5. Gestión del Proyecto

5.1 Gestión del Proyecto

Para la planificación, organización y seguimiento del proyecto se utilizó la plataforma Trello, una herramienta de gestión colaborativa basada en tableros que permite organizar tareas, asignar responsabilidades, establecer fechas de entrega y supervisar el progreso de las distintas actividades.

La utilización de Trello permitió coordinar el trabajo entre los miembros del equipo y estructurar el desarrollo del proyecto en diferentes fases. Esta planificación facilitó el control de los plazos establecidos por la asignatura y permitió realizar un seguimiento continuo del avance del proyecto.

La gestión se desarrolló siguiendo una metodología incremental, donde cada fase se apoyaba en los resultados obtenidos en la anterior. De esta forma fue posible evolucionar progresivamente desde la definición inicial de la propuesta hasta la integración completa del sistema y su posterior presentación en la Feria de Proyectos.

5.1.1 Herramienta de gestión utilizada

La herramienta seleccionada para la gestión del proyecto fue Trello, utilizando un tablero compartido por todos los integrantes del equipo.

Esta plataforma permitió:

- Organizar el trabajo en fases diferenciadas.
- Asignar responsabilidades a cada integrante.
- Realizar el seguimiento del estado de las tareas.
- Controlar los plazos de entrega.
- Coordinar el desarrollo simultáneo de distintas partes del proyecto.
- Mantener una visión global del progreso del trabajo.

La utilización de una herramienta colaborativa resultó especialmente útil debido a la naturaleza multidisciplinar del proyecto, que combina robótica, programación, electrónica, comunicaciones industriales y bases de datos.

5.1.2 Fases del proyecto

El proyecto se dividió en seis fases principales.

Fase	Objetivo principal	Periodo
Definición	Definir la idea y los requisitos del proyecto	Marzo
Equipamiento	Seleccionar los dispositivos y diseñar la estación	Abril
Implementación	Desarrollar e integrar la solución	Abril - Mayo
Documentación	Elaborar la memoria técnica	Mayo
Presentación	Preparar la exposición pública	Mayo
Feria de Proyectos	Presentar el proyecto final	Finales de Mayo

Fase 1. Definición

Durante esta fase se realizó una lluvia de ideas para seleccionar el proyecto a desarrollar y se definieron los objetivos generales y específicos de la solución. Además, se redactó la propuesta inicial y se establecieron los requisitos funcionales que debía cumplir la estación automatizada.

Fase 2. Equipamiento

En esta fase se seleccionaron los elementos hardware y software que formarían parte de la estación. Se eligieron el robot ABB IRB 360 para la clasificación de piezas, los robots colaborativos UR3e para la serigrafía, la cámara OAK-D, la ESP32-S3, los sensores auxiliares y el resto de los componentes necesarios. También se diseñó el layout general de la estación.

Fase 3. Implementación

La fase de implementación concentró la mayor parte del trabajo técnico. Durante esta etapa se desarrolló la simulación en RoboDK, se programó la ESP32-S3, se implementó la comunicación MQTT, se diseñó la base de datos PostgreSQL y se desarrollaron los scripts en Python encargados de integrar todos los componentes del sistema.

Fase 4. Documentación

Una vez finalizada la implementación técnica se elaboró la documentación del proyecto. Se redactó la memoria, se desarrollaron los diagramas de flujo, el modelo relacional de la base de datos, la arquitectura MQTT y el resto de documentación requerida.

Fase 5. Presentación



Durante esta fase se preparó el material necesario para la exposición pública del proyecto, incluyendo el cartel informativo, el material visual de apoyo y las demostraciones de funcionamiento de la estación automatizada.

Fase 6. Feria de Proyectos

La última fase estuvo orientada a la participación en la Feria de Proyectos. Se preparó el video demostrativo, se realizó el montaje del stand y se presentó el proyecto a profesores y estudiantes.

5.1.3 Desglose de tareas realizadas

Cada una de las tareas principales registradas en Trello agrupó diversas subtareas técnicas desarrolladas durante el proyecto.

Área	Actividades realizadas
RoboDK	Configuración del ABB IRB 360, programación de trayectorias, configuración de los UR3e, diseño de cintas transportadoras, implementación de bandejas, simulación de visión artificial y simulación de serigrafía láser.
ESP32-S3	Configuración WiFi, implementación MQTT, programación LCD, LEDs, sensor ultrasónico, sensor LDR, botón de emergencia e implementación del buffer.
MQTT	Definición de topics, programación de publicadores y suscriptores, validación de mensajes y pruebas de comunicación.
Python	Integración entre RoboDK, MQTT y PostgreSQL, gestión de eventos y sincronización de dispositivos.
PostgreSQL	Diseño del modelo relacional, creación de tablas, inserción de registros y validación de consultas.
Documentación	Elaboración de diagramas, memoria técnica, análisis financiero, normativas y material para la presentación.

5.1.4 Hitos del proyecto

Durante el desarrollo se establecieron diferentes hitos que permitieron evaluar el avance del proyecto.



Hito	Descripción	Resultado
H1	Definición del proyecto	Propuesta aprobada
H2	Diseño conceptual de la estación	Layout completado
H3	Selección de robots y componentes	Arquitectura física definida
H4	Primera simulación funcional en RoboDK	Clasificación operativa
H5	Programación de la ESP32-S3	Dispositivos operativos
H6	Integración MQTT	Comunicación funcional
H7	Integración PostgreSQL	Persistencia de datos operativa
H8	Integración completa del sistema	Sistema funcional
H9	Finalización de la memoria	Documentación completada
H10	Participación en la Feria de Proyectos	Presentación pública

5.1.5 Planificación temporal

La planificación temporal se organizó a lo largo del cuatrimestre distribuyendo las diferentes actividades según su complejidad y dependencias.

Mes	Actividades principales
Marzo	Definición del proyecto y propuesta inicial
Abril	Diseño conceptual, selección de dispositivos y desarrollo inicial
Mayo	Implementación, integración y validación del sistema
Finales de Mayo	Elaboración de la documentación y preparación de la presentación
Finales de Mayo	Participación en la Feria de Proyectos

Esta organización permitió desarrollar el proyecto de forma progresiva, evitando solapamientos innecesarios y facilitando la coordinación entre los miembros del equipo.



5.1.6 Recursos empleados

Recursos hardware

- Robot ABB IRB 360.
- Robots colaborativos UR3e.
- Cámara OAK-D.
- ESP32-S3.
- Sensor ultrasónico.
- Sensor LDR.
- Pantalla LCD.
- LEDs indicadores.
- Pulsador de emergencia.
- Equipos informáticos de desarrollo.

Recursos software

- RoboDK.
- Python.
- PostgreSQL.
- Arduino IDE.
- MQTT (EMQX).
- Trello.
- Draw.io.
- Microsoft Word.

5.1.7 Gestión de riesgos

Durante el desarrollo del proyecto se identificaron diferentes riesgos que podían afectar al correcto funcionamiento de la solución.

Riesgo	Impacto	Medidas adoptadas
Fallos en la comunicación MQTT	Alto	Realización de pruebas periódicas de comunicación.
Problemas de integración entre componentes	Alto	Desarrollo modular y pruebas por etapas.
Errores de programación	Medio	Validación continua y depuración del código.
Retrasos en la documentación	Medio	Distribución equilibrada de tareas.
Problemas en la simulación	Medio	Pruebas iterativas en RoboDK.
Pérdida de datos	Bajo	Uso de PostgreSQL para almacenamiento persistente.

La identificación temprana de estos riesgos permitió minimizar su impacto durante el desarrollo del proyecto.

5.1.8 Distribución de responsabilidades

Las tareas fueron distribuidas entre los integrantes del grupo según sus conocimientos y disponibilidad.

Integrante	Responsabilidades
Francisco Lorente Ruzickova	ESP32-S3, MQTT, Python, integración de sistemas y documentación
Jesús Mochales Domingo	Base de datos PostgreSQL, documentación y presentación.
Andrea Quintanilla Torrez	RoboDK, ABB IRB 360 y simulación de clasificación.
María Belén Marca Ábalos	Elaboración de memoria, cumplimiento de plazos y simulación RoboDK
Rida Ballaoui Bouddih	ESP32-S3, MQTT, Python, integración de sistemas y documentación

Esta distribución permitió desarrollar diferentes partes del proyecto de forma simultánea y optimizar el tiempo disponible.

5.1.9 Participación en la Feria de Proyectos

La participación en la Feria de Proyectos constituyó la fase final del trabajo realizado y tuvo como objetivo presentar públicamente los resultados obtenidos durante el desarrollo de la asignatura.

Para ello se preparó una demostración funcional de la estación automatizada, permitiendo mostrar la integración entre RoboDK, MQTT, Python, PostgreSQL y la ESP32-S3. Durante la demostración fue posible observar el funcionamiento de los robots, la comunicación entre los distintos componentes del sistema y el intercambio de información en tiempo real mediante MQTT.

Además de la demostración técnica, se elaboró material visual de apoyo, incluyendo un cartel explicativo y un vídeo demostrativo que resumía el funcionamiento general del sistema, la arquitectura desarrollada y las principales tecnologías empleadas durante el proyecto.

La preparación de la feria requirió la realización de diversas tareas previas, entre las que destacan la elaboración del material divulgativo, la preparación de la simulación en RoboDK, la comprobación del correcto funcionamiento de las comunicaciones MQTT, la validación de la integración con la ESP32-S3 y la base de datos PostgreSQL, así como la organización del material necesario para la exposición.

Durante la jornada, los integrantes del grupo se encargaron de presentar el proyecto, realizar demostraciones de funcionamiento y responder a las preguntas formuladas por profesores, estudiantes y visitantes interesados en la solución desarrollada.

La feria permitió presentar los resultados obtenidos, validar el funcionamiento global de la estación automatizada y difundir las tecnologías empleadas durante el proyecto. Asimismo, constituyó una experiencia muy enriquecedora para los miembros del equipo, ya que permitió desarrollar habilidades de comunicación técnica, trabajo en equipo y capacidad de exposición pública. Además, ofreció la oportunidad de recibir comentarios y sugerencias por parte de otros estudiantes y profesores, proporcionando una visión externa del trabajo realizado y permitiendo valorar la solución desarrollada desde diferentes perspectivas.

En conjunto, la participación en la Feria de Proyectos supuso el cierre del ciclo de desarrollo de la estación automatizada, permitiendo mostrar de forma práctica los resultados alcanzados y poner en valor los conocimientos adquiridos en áreas como robótica industrial, programación, comunicaciones industriales, bases de datos e integración de sistemas.

5.1.10 Resultados de la gestión del proyecto

La planificación realizada permitió completar todas las fases previstas dentro de los plazos establecidos, obteniendo una solución funcional capaz de integrar simulación robótica, comunicaciones MQTT, electrónica embebida y almacenamiento de datos.

La utilización de Trello facilitó el seguimiento continuo de las tareas y permitió coordinar eficazmente el trabajo realizado por los distintos integrantes del equipo. Además, la organización por fases permitió identificar claramente los hitos alcanzados y garantizar una evolución ordenada desde la idea inicial hasta la presentación final del proyecto.

[Enlace tablero Trello](#)

5.2 Listado de entregables del proyecto

Como resultado del desarrollo de la propuesta de automatización se han generado diferentes documentos, modelos y desarrollos software que constituyen los entregables principales del proyecto.

La documentación y materiales que serían entregados a la empresa cliente son los siguientes:

Entregable	Descripción
Memoria técnica del proyecto	Documento completo que recoge el análisis, diseño, implementación y validación de la solución automatizada.
Layout de la estación	Plano de distribución en planta donde se muestra la ubicación de robots, cintas transportadoras, sensores, actuadores y zonas de trabajo.
Simulación RoboDK	Modelo completo de la estación automatizada desarrollado en RoboDK, incluyendo robots, cintas transportadoras y procesos de clasificación y serigrafía.
Código fuente Python	Scripts desarrollados para la integración entre RoboDK, MQTT y PostgreSQL.
Código fuente ESP32-S3	Programa desarrollado para la gestión de sensores, actuadores, pantalla LCD y comunicaciones MQTT.
Base de datos PostgreSQL	Modelo relacional y scripts utilizados para la creación de la base de datos y almacenamiento de información del proceso productivo.

Entregable	Descripción
Documentación MQTT	Descripción de la arquitectura de comunicaciones, topics utilizados y mensajes intercambiados entre los distintos componentes del sistema.
Diagramas de flujo	Diagramas que describen el funcionamiento de los procesos principales y la comunicación entre dispositivos.
Diagramas de arquitectura	Diagramas de integración entre RoboDK, Python, MQTT, PostgreSQL y la ESP32-S3.
Cartel del proyecto	Material divulgativo preparado para la presentación en la Feria de Proyectos.
Vídeo demostrativo	Vídeo que muestra el funcionamiento general de la estación automatizada y la integración de sus componentes.

Estos entregables permiten documentar completamente la solución propuesta y facilitan tanto su comprensión como posibles futuras ampliaciones o modificaciones del sistema.

6. Pruebas y Validación

6.1 Estrategia de pruebas y criterios de éxito

Con el objetivo de verificar el correcto funcionamiento de la solución desarrollada, se definió una estrategia de pruebas basada en la validación progresiva de los distintos subsistemas que componen la estación automatizada. Esta estrategia permite comprobar individualmente cada componente antes de realizar la validación global del sistema integrado.

Las pruebas se realizaron principalmente mediante la simulación desarrollada en RoboDK y mediante la ejecución de los programas implementados en Python y la ESP32-S3. De esta forma fue posible evaluar tanto el comportamiento de los robots como las comunicaciones MQTT, el funcionamiento de los sensores y el almacenamiento de información en la base de datos PostgreSQL.

La estrategia seguida se dividió en cuatro niveles:

- Validación de la simulación robótica.
- Validación de las comunicaciones MQTT.

- Validación de la base de datos PostgreSQL.
- Validación global del sistema integrado.

Este enfoque permitió detectar posibles errores de forma progresiva y garantizar que todos los componentes funcionaran correctamente antes de su integración final.

6.1.1 Criterios de éxito

Para considerar satisfactoria la solución desarrollada se establecieron los siguientes criterios de éxito:

Elemento validado	Criterio de éxito
Simulación RoboDK	Ejecución completa del proceso sin errores de trayectoria ni colisiones.
Clasificación de piezas	Clasificación correcta de palillos válidos y defectuosos.
Comunicación MQTT	Recepción correcta de todos los mensajes intercambiados.
Sensor ultrasónico	Detección correcta de los estados LLENO y VACIO.
Sensor LDR	Detección de condiciones de iluminación con un error máximo del 20 %.
Base de datos PostgreSQL	Registro correcto de la información generada durante el proceso.
Sistema integrado	Funcionamiento continuo sin interrupciones durante la simulación.

El cumplimiento de estos criterios permite verificar que la solución propuesta satisface los objetivos definidos durante la fase de diseño.

6.2 Descripción de la simulación del proyecto de automatización

La validación principal del proyecto se realiza mediante una simulación desarrollada en RoboDK que reproduce el funcionamiento completo de la estación automatizada propuesta.

La simulación incorpora todos los elementos principales definidos durante la fase de diseño:

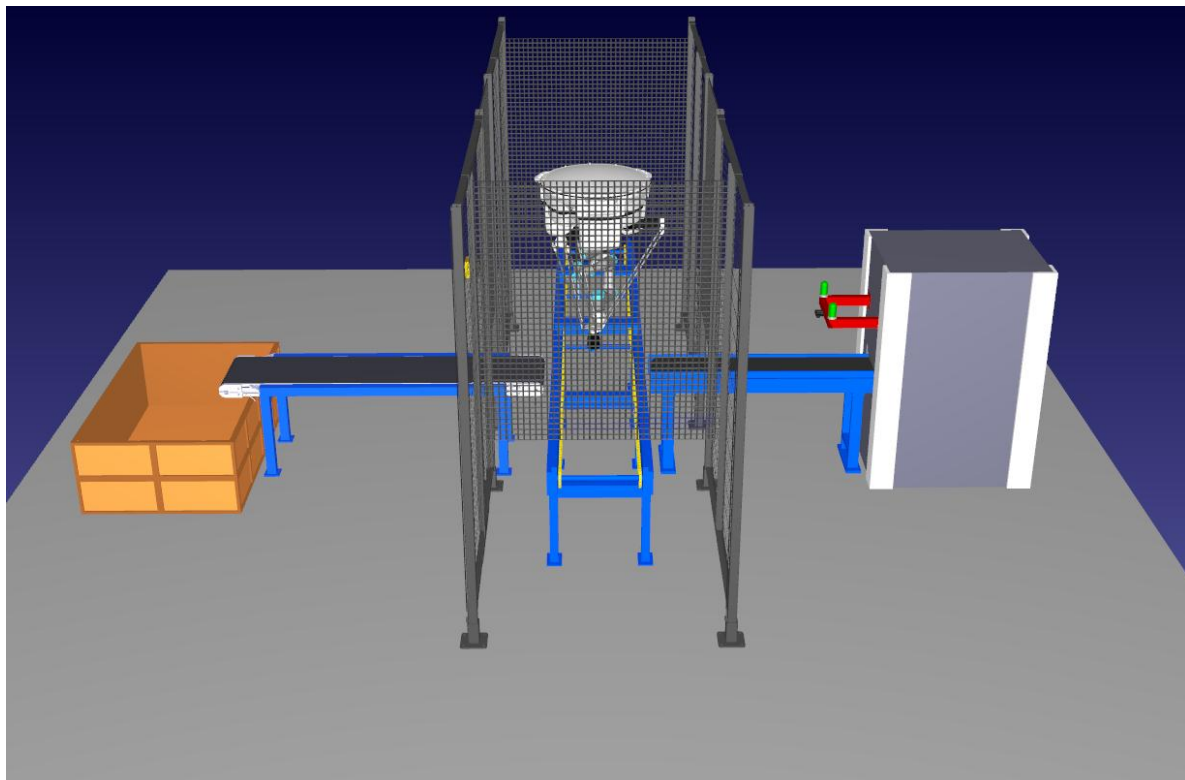
- Tolva de alimentación de palillos.
- Cintas transportadoras de entrada, clasificación y salida.

- Robot ABB IRB 360 encargado de la clasificación de piezas.
- Sistema de visión artificial para la inspección de calidad.
- Robots colaborativos UR3e encargados del proceso de serigrafía.
- Herramienta láser para el marcado de los palillos.
- Bandejas de almacenamiento.
- Sistema de comunicaciones MQTT.
- Integración con Python y PostgreSQL.

El proceso comienza con la alimentación de los palillos desde la tolva de entrada. A continuación, el robot ABB IRB 360 recoge cada pieza y la clasifica según la información proporcionada por el sistema de visión artificial. Los palillos válidos continúan hacia la estación de serigrafía, donde los robots UR3e realizan el marcado correspondiente mediante la herramienta láser. Finalmente, las piezas terminadas son depositadas en las bandejas de salida.

Además del proceso productivo, la simulación permite validar la integración con la ESP32-S3 mediante MQTT, reproduciendo eventos relacionados con el sensor ultrasónico, el sensor LDR, la pantalla LCD, los LEDs indicadores y el botón de emergencia.

Figura 6.2 1



6.3 Procedimientos de validación y resultados esperados

Con el fin de comprobar el correcto funcionamiento del sistema se definieron diferentes procedimientos de validación asociados a los principales componentes de la solución.

6.3.1 Validación de la clasificación de piezas

Procedimiento

- Ejecutar la simulación con diferentes tipos de palillos.
- Verificar la detección de piezas válidas y defectuosas.
- Comprobar el depósito correcto de cada pieza en su ubicación correspondiente.

Resultado esperado

- Clasificación correcta de todas las piezas procesadas.
- Ausencia de errores de posicionamiento o depósito.

6.3.2 Validación de la estación de serigrafía

Procedimiento

- Ejecutar el proceso de marcado sobre los palillos clasificados como válidos.
- Verificar el seguimiento correcto de las trayectorias programadas para los robots UR3e.
- Comprobar la sincronización entre los robots y las cintas transportadoras.

Resultado esperado

- Ejecución correcta del proceso de marcado láser.
- Ausencia de colisiones o errores de trayectoria.

6.3.3 Validación de las comunicaciones MQTT

Procedimiento

- Publicar y recibir mensajes entre RoboDK, Python y la ESP32-S3.
- Comprobar la recepción de mensajes relacionados con sensores y actuadores.
- Verificar el intercambio de información durante la ejecución de la simulación.

Resultado esperado

- Recepción correcta de todos los mensajes publicados.
- Ausencia de pérdidas de comunicación durante la ejecución de la simulación.



6.3.4 Validación del sensor ultrasónico

Procedimiento

- Simular diferentes niveles de llenado del contenedor de rechazo.
- Verificar la publicación de los mensajes correspondientes mediante MQTT.
- Comprobar la visualización de los estados generados.

Resultado esperado

- Detección correcta de los estados LLENO y VACIO.
- Generación correcta de los mensajes MQTT asociados.

6.3.5 Validación del sensor LDR

Procedimiento

- Simular diferentes condiciones de iluminación.
- Verificar la generación de los estados MUCHA_LUZ y POCA_LUZ.
- Comprobar la transmisión de la información mediante MQTT.

Resultado esperado

- Detección de condiciones lumínicas con un error máximo del 20 %.
- Generación correcta de avisos relacionados con la iluminación.

6.3.6 Validación de la base de datos PostgreSQL

Procedimiento

- Ejecutar la simulación completa.
- Registrar información generada durante el proceso.
- Verificar la inserción correcta de registros en las tablas correspondientes.
- Comprobar la integridad de las relaciones existentes entre tablas.

Resultado esperado

- Almacenamiento correcto del 100 % de los registros generados.
- Disponibilidad de la información para consultas posteriores.

6.3.7 Validación global del sistema

Procedimiento



- Ejecutar la simulación completa durante varios ciclos consecutivos.
- Comprobar el funcionamiento simultáneo de robots, sensores, MQTT, Python y PostgreSQL.
- Verificar la correcta sincronización entre todos los componentes.

Resultado esperado

- Funcionamiento estable del sistema integrado.
- Ausencia de errores críticos durante la ejecución.
- Correcta coordinación entre todos los elementos de la estación automatizada.

6.4 Conclusión de las pruebas

La estrategia de pruebas definida permite verificar el correcto funcionamiento de todos los componentes que forman parte de la solución desarrollada. La utilización de RoboDK como entorno de simulación facilita la validación previa del sistema antes de una posible implantación física, reduciendo riesgos y permitiendo detectar errores de diseño o integración en fases tempranas del proyecto.

Los procedimientos de validación establecidos permiten comprobar tanto el comportamiento individual de cada subsistema como el funcionamiento global de la estación automatizada, garantizando que la solución propuesta cumple los requisitos definidos durante la fase de diseño.

7. Costes y Beneficios

7.1 Presupuesto detallado

La implantación de la estación automatizada requiere la adquisición e integración de diversos elementos hardware y software destinados a garantizar el correcto funcionamiento del sistema. Para estimar la inversión necesaria se ha realizado una valoración aproximada basada en precios de mercado de equipos industriales equivalentes a los utilizados en la propuesta.

El presupuesto incluye los robots industriales, los sistemas de transporte, los dispositivos de visión artificial, los elementos de comunicación, los sistemas de seguridad industrial, las licencias software y los costes asociados a la ingeniería, instalación y puesta en marcha de la solución.

7.1.1 Equipamiento principal

Categoría	Concepto	Unidades	Coste (€)
Conjunto robótico	Robot Delta ABB IRB 360	1	30.000 €
Conjunto robótico	Robot colaborativo UR3e	2	45.000 €
Logística	Cintas transportadoras motorizadas	3	12.000 €
Visión artificial	Cámara OAK-D Pro PoE Industrial	1	400 €
Comunicaciones	Gateway IoT Industrial / Edge Computer	1	1.000 €
Comunicaciones	Módulo E/S industriales IP67	1	250 €
Instrumentación	Pantalla HMI industrial	1	500 €
Instrumentación	Sensor ultrasónico industrial	1	250 €
Instrumentación	Fotocélula industrial	1	150 €
Instrumentación	Torre luminosa LED industrial	1	200 €
Instrumentación	Borneras, carril DIN y canaletas	1	500 €

7.1.2 Sistemas de seguridad industrial

La seguridad constituye uno de los aspectos más importantes en cualquier instalación robotizada. Debido a la presencia de robots industriales de alta velocidad, es necesario incorporar medidas de protección que garanticen la seguridad de los operarios y el cumplimiento de la normativa vigente.

Categoría	Concepto	Unidades	Coste (€)
Seguridad	Vallado perimetral de seguridad	1	12.000 €
Seguridad	Barreras fotoeléctricas	1	2.000 €
Seguridad	Relés de seguridad	1	800 €
Seguridad	Pulsadores de emergencia	2	300 €

Categoría	Concepto	Unidades	Coste (€)
Seguridad	Sistema de parada de emergencia integrado	1	2.000 €

7.1.3 Software y comunicaciones

Para la simulación, programación e integración de la estación automatizada es necesario disponer de diferentes herramientas software y servicios de comunicación.

Categoría	Concepto	Unidades	Coste (€)
Software	Licencia RoboDK Profesional	1	2.500 €
Software	Servidor PostgreSQL y configuración	1	800 €
Software	Librerías y herramientas de desarrollo	1	200 €

7.1.4 Ingeniería, instalación y formación

Además de los costes asociados al equipamiento, es necesario considerar los trabajos de ingeniería, integración, instalación y formación del personal encargado de operar y mantener el sistema.

Categoría	Concepto	Coste (€)
Ingeniería	Diseño e integración del sistema	15.000 €
Instalación	Montaje mecánico y eléctrico	8.000 €
Instalación	Puesta en marcha y validación	4.000 €
Formación	Capacitación de operarios y personal de mantenimiento	3.000 €

RESUMEN ECONÓMICO	
Concepto	Coste (€)
Equipamiento principal	90.250 €
Seguridad industrial	17.100 €

RESUMEN ECONÓMICO	
Concepto	Coste (€)
Software y comunicaciones	3.500 €
Ingeniería, instalación y formación	30.000 €
TOTAL ESTIMADO	140.850 €

Por tanto, la inversión total estimada para la implantación de la estación automatizada asciende a **140.850 €**, incluyendo todos los elementos necesarios para su funcionamiento, así como los costes de ingeniería, integración, seguridad y formación del personal.

7.2 Análisis coste-beneficio

El análisis coste-beneficio permite evaluar la viabilidad económica de la propuesta de automatización comparando la inversión inicial necesaria con los beneficios obtenidos durante la explotación de la instalación.

Aunque la implantación de una estación automatizada requiere una inversión significativa, la incorporación de robots industriales, sistemas de visión artificial y tecnologías de comunicación aporta numerosas ventajas que permiten mejorar la eficiencia global del proceso productivo.

Uno de los beneficios más importantes es el incremento de la productividad. Los resultados obtenidos durante la simulación muestran una reducción del tiempo de ciclo desde los 90 segundos del proceso manual hasta los 55 segundos del proceso automatizado, lo que supone una mejora del 38,89 %. Esta reducción permite aumentar significativamente la capacidad productiva de la instalación y procesar un mayor número de piezas durante la misma jornada de trabajo.

Otro aspecto relevante es la disminución del desperdicio de material. Gracias a la clasificación automática de las piezas y al uso de sistemas de inspección, es posible detectar productos defectuosos antes de que continúen avanzando por la línea de producción, evitando operaciones innecesarias y reduciendo el consumo de recursos.

La automatización también contribuye a disminuir errores operativos y paradas de producción. Los robots industriales ofrecen elevados niveles de precisión y repetibilidad, permitiendo ejecutar tareas repetitivas de forma constante y minimizando errores derivados de la intervención manual.

Asimismo, la incorporación de una base de datos PostgreSQL y de comunicaciones MQTT permite mejorar la trazabilidad del proceso productivo. La disponibilidad de información en



tiempo real facilita las tareas de supervisión, mantenimiento y análisis posterior de la producción, contribuyendo a una gestión más eficiente de la instalación.

BENEFICIOS ANUALES ECONÓMICOS	
Concepto	Beneficio anual (€)
Reducción de desperdicio de material	12.000 €
Incremento de productividad	35.000 €
Reducción de errores y paradas	8.000 €
Mejora de la trazabilidad y supervisión del proceso	5.000 €
TOTAL BENEFICIOS ANUALES	60.000 €

COSTE MANTENIMIENTO	
Concepto	Coste anual (€)
Mantenimiento de robots y cintas	8.000 €
Consumo eléctrico	3.500 €
Actualización de software y licencias	2.000 €
Sustitución de sensores y elementos de desgaste	1.500 €
Mantenimiento de sistemas de seguridad	1.500 €
Soporte hardware y comunicaciones	500 €
TOTAL COSTES ANUALES	17.000 €

BENEFICIO NETO ANUAL	
Concepto	Valor (€)
Beneficios anuales	60.000 €
Costes operativos anuales	-17.000 €
BENEFICIO NETO ANUAL	43.000 €

FLUJO DE CAJA ANUAL		
Año	Flujo de caja (€)	Flujo acumulado (€)
0	-140.850 €	-140.850 €
1	43.000 €	-97.850 €
2	43.000 €	-54.850 €
3	43.000 €	-11.850 €
4	43.000 €	31.150 €
5	43.000 €	74.150 €

La inversión inicial se recupera aproximadamente entre el tercer y cuarto año de funcionamiento de la instalación.

7.3 Indicadores financieros del proyecto

Con el objetivo de complementar el análisis económico realizado, se han calculado diversos indicadores financieros que permiten evaluar la rentabilidad y viabilidad de la inversión propuesta.

Retorno de la inversión (ROI)

El retorno de la inversión (ROI) mide la rentabilidad obtenida respecto al capital invertido.

La expresión utilizada es:

$$ROI = (((\text{Beneficio neto anual} \times \text{Horizonte de análisis}) - \text{Inversión inicial}) / \text{Inversión inicial}) \times 100$$

$$ROI = (((43.000 \times 5) - 140.850) / 140.850) \times 100 = 52.6\%$$



Considerando:

- Inversión inicial: 140.850 €
- Beneficio neto anual: 43.000 €
- Horizonte de análisis: 5 años

Resultado: El proyecto presenta un ROI aproximado del **52,6 %**, indicando que la inversión genera una rentabilidad positiva durante el periodo analizado.

Valor Actual Neto (VAN)

El VAN permite determinar el valor generado por la inversión teniendo en cuenta el valor temporal del dinero.

Fórmula general:

$$VAN = -I + \sum_{t=1}^n \frac{F_t}{(1+r)^t}$$

Donde:

- I : Inversión inicial.
- F_t : Flujo de caja del año t .
- r : Tasa de descuento.
- n : Número de años analizados.

Considerando:

- Inversión inicial: 140.850 €
- Flujo anual: 43.000 €
- Horizonte temporal: 5 años
- Tasa de descuento: 8 %

Resultado:

$$VAN \approx 30.900 \text{ €}$$

Resultado: El VAN es positivo, indicando que el proyecto genera valor económico y resulta financieramente viable.

Tasa Interna de Retorno (TIR)



La TIR representa la rentabilidad interna del proyecto y corresponde a la tasa de descuento que hace que el VAN sea igual a cero.

Para los flujos de caja considerados:

Fórmula:

$$0 = -I + \sum_{t=1}^n \frac{F_t}{(1 + TIR)^t}$$

Donde:

- I : Inversión inicial.
- F_t : Flujo de caja anual.
- n : Horizonte temporal del proyecto.

Resultado obtenido:

$$TIR \approx 15,8\%$$

Resultado: La rentabilidad obtenida es superior a la tasa de descuento utilizada, por lo que la inversión resulta económicamente atractiva.

Periodo de recuperación (Payback)

El periodo de recuperación indica el tiempo necesario para recuperar la inversión inicial realizada.

Payback = Inversión inicial / Beneficio neto anual

$$\text{Payback} = 140.850 / 43.000 = 3.28 \text{ años}$$

Resultado: La inversión se recupera aproximadamente en **3,3 años**, equivalentes a unos **39 meses de funcionamiento**.

7.4 Conclusión económica

El análisis económico realizado demuestra que la propuesta de automatización es viable desde el punto de vista financiero. Aunque la inversión inicial es elevada debido a la incorporación de robots industriales, sistemas de visión artificial, dispositivos de comunicación y elementos de seguridad industrial, los beneficios obtenidos permiten recuperar la inversión en un plazo razonable.

La mejora de la productividad, la reducción de desperdicios, la disminución de errores y el aumento de la trazabilidad contribuyen a obtener una rentabilidad positiva a medio plazo. Los indicadores financieros calculados muestran un VAN positivo, una TIR superior a la tasa de



descuento considerada y un ROI favorable, confirmando la viabilidad económica de la solución propuesta.

En conjunto, la estación automatizada representa una inversión justificada para entornos industriales que buscan incrementar su eficiencia productiva, mejorar la calidad del proceso y avanzar hacia modelos de fabricación más digitalizados e integrados.

8. Normativa y Regulación. Seguridad

8.1 Impacto en los puestos de trabajo

La implantación de la estación automatizada supone una transformación de las tareas realizadas por los operarios dentro del proceso productivo. Actividades repetitivas como la manipulación de piezas, la clasificación visual o determinadas operaciones de marcado pasan a ser ejecutadas por robots industriales y sistemas automáticos de inspección.

Sin embargo, la automatización no implica la eliminación completa de la intervención humana. Los operarios continúan participando en tareas de supervisión, alimentación de material, control de la producción, mantenimiento preventivo y resolución de incidencias.

Esta evolución de los puestos de trabajo permite reducir la exposición de los trabajadores a tareas repetitivas y de bajo valor añadido, mejorando además las condiciones ergonómicas del entorno laboral. La automatización contribuye a disminuir la fatiga asociada a operaciones repetitivas y favorece que el personal pueda desempeñar funciones de mayor cualificación técnica.

Por otra parte, la implantación del sistema requiere una formación específica del personal en aspectos relacionados con la operación segura de robots, utilización de sistemas de supervisión, procedimientos de parada de emergencia y mantenimiento básico de los equipos instalados.

8.2 Consideraciones de seguridad

La seguridad constituye uno de los aspectos más importantes en el diseño de cualquier instalación automatizada. Debido a la presencia de robots industriales, elementos móviles y herramientas de marcado láser, se han incorporado diferentes medidas destinadas a minimizar los riesgos para los operarios y garantizar un funcionamiento seguro de la instalación.

Seguridad del robot ABB IRB 360

El robot Delta ABB IRB 360 trabaja a alta velocidad realizando operaciones de clasificación de piezas. Debido a los riesgos mecánicos asociados a sus movimientos, se ha previsto la instalación de un vallado perimetral que delimita completamente su zona de trabajo e impide el acceso accidental de personas durante el funcionamiento normal de la línea.

Asimismo, se incorporan dispositivos de parada de emergencia accesibles desde distintos puntos de la instalación para permitir la detención inmediata del sistema en caso de incidencia.

Seguridad de los robots colaborativos UR3e

Los robots UR3e utilizados en la estación de marcado han sido diseñados para aplicaciones colaborativas. No obstante, su utilización sigue requiriendo una evaluación de riesgos específica y la aplicación de medidas de seguridad adecuadas.

Durante el funcionamiento se contempla la limitación de velocidades y trayectorias, así como la supervisión continua del entorno de trabajo. Estas medidas reducen la probabilidad de colisiones y permiten una interacción segura con los operarios cuando sea necesario.

Seguridad de la herramienta láser

La estación de marcado incorpora una herramienta láser que requiere medidas específicas de protección debido a los riesgos asociados a la radiación luminosa.

Para minimizar dichos riesgos se contempla el encapsulado de la zona de marcado, la señalización de advertencia correspondiente y la limitación de acceso durante el funcionamiento del sistema. Además, el láser únicamente podrá activarse cuando se cumplan las condiciones de seguridad establecidas por el sistema de control.

Seguridad eléctrica y de comunicaciones

La instalación incorpora armarios eléctricos, canalizaciones protegidas y sistemas de puesta a tierra adecuados para reducir los riesgos eléctricos asociados al funcionamiento de la maquinaria.

Respecto a las comunicaciones, la arquitectura MQTT implementada debe garantizar la integridad de los mensajes intercambiados entre la ESP32-S3, RoboDK y la base de datos, evitando comportamientos inesperados derivados de pérdidas de comunicación.

Evaluación y reducción de riesgos

Siguiendo las recomendaciones de la normativa de seguridad industrial, se realizará una evaluación de riesgos previa a la puesta en marcha de la instalación. Esta evaluación permitirá identificar peligros potenciales, estimar los riesgos asociados y definir las medidas de protección necesarias para reducirlos hasta niveles aceptables.

8.3 Cumplimiento de regulaciones y estándares

El diseño de la estación automatizada se ha desarrollado teniendo en consideración la normativa vigente relacionada con seguridad de maquinaria, robótica industrial y robótica colaborativa.

Directiva de Máquinas 2006/42/CE

La Directiva Europea de Máquinas establece los requisitos esenciales de seguridad que deben cumplir las máquinas comercializadas dentro de la Unión Europea. Entre otros aspectos, exige la identificación y reducción de riesgos durante el diseño de la instalación y constituye la base para la obtención del marcado CE.

En el proyecto, esta directiva se aplica mediante la incorporación de medidas de protección física, dispositivos de parada de emergencia y procedimientos de evaluación de riesgos.

UNE-EN ISO 12100

La norma UNE-EN ISO 12100 define los principios generales para la evaluación y reducción de riesgos en maquinaria industrial. Establece metodologías para identificar peligros, estimar riesgos y seleccionar medidas de protección adecuadas.

Su aplicación en el proyecto se refleja en el análisis de riesgos realizado durante el diseño de la estación automatizada y en la incorporación de elementos de protección específicos.

UNE-EN ISO 10218-1

Esta norma establece los requisitos de seguridad aplicables a los robots industriales. Define criterios de diseño seguro, funciones de parada de emergencia y medidas destinadas a reducir los riesgos asociados al funcionamiento de los robots industriales.

La utilización del ABB IRB 360 dentro de una zona protegida mediante vallado perimetral responde directamente a las recomendaciones de esta norma.

UNE-EN ISO 10218-2

La segunda parte de la norma ISO 10218 se centra en la integración de robots industriales dentro de sistemas robotizados completos. Incluye requisitos relacionados con accesos, espacios de trabajo, ergonomía, dispositivos de seguridad externos e interacción con otros equipos de la instalación.

Esta norma resulta especialmente relevante para la integración conjunta del ABB IRB 360, los UR3e, las cintas transportadoras y los sistemas de visión artificial presentes en la estación automatizada.

ISO/TS 15066

La especificación técnica ISO/TS 15066 complementa la normativa de robótica colaborativa y establece requisitos adicionales para garantizar una interacción segura entre robots y personas. Define aspectos relacionados con limitación de fuerza, velocidad, potencia y distancia de separación.



Esta normativa resulta aplicable a los robots UR3e utilizados en la estación de marcado.

ISO 13849-1

La norma ISO 13849-1 establece requisitos para las partes relacionadas con la seguridad de los sistemas de mando y control. Se utiliza para evaluar la fiabilidad de funciones críticas como paradas de emergencia, sensores de seguridad y dispositivos de protección.

ISO 13857 e ISO 13855

Estas normas definen las distancias mínimas de seguridad y los criterios para el posicionamiento de resguardos y dispositivos de protección respecto a las zonas peligrosas de la instalación.

Su aplicación permite determinar adecuadamente las dimensiones del vallado perimetral y la ubicación de los elementos de seguridad.

8.4 Conclusión

La propuesta de automatización se ha diseñado teniendo en cuenta los principales requisitos normativos aplicables a sistemas robotizados industriales y colaborativos. La incorporación de vallados de seguridad, dispositivos de parada de emergencia, procedimientos de evaluación de riesgos y medidas de protección específicas para los robots y la herramienta láser permite garantizar un funcionamiento seguro de la instalación.

Asimismo, la aplicación de las normas UNE-EN ISO 12100, UNE-EN ISO 10218, ISO/TS 15066 e ISO 13849 proporciona un marco técnico adecuado para asegurar la protección de los trabajadores y el cumplimiento de la normativa vigente en materia de seguridad industrial.

9. Desarrollo de Software y Algoritmos

9.1 Listado tecnológico

El desarrollo software de la estación automatizada se ha realizado mediante la integración de diferentes tecnologías destinadas a la simulación industrial, la comunicación entre dispositivos, la persistencia de datos y la programación de sistemas embebidos.

La combinación de estas herramientas permite implementar una arquitectura distribuida capaz de coordinar robots industriales, sensores, actuadores y sistemas de almacenamiento de información, facilitando la automatización completa del proceso de clasificación y marcado de palillos.

Las principales tecnologías empleadas durante el proyecto se muestran en la siguiente tabla:

Tecnología	Función dentro del proyecto
RoboDK	Simulación y programación de la estación robotizada.
Python	Integración entre RoboDK, MQTT y PostgreSQL.
ESP32-S3	Control de sensores, actuadores y comunicaciones con el sistema.
Arduino IDE	Desarrollo del software embebido de la ESP32-S3.
MQTT	Protocolo de comunicación entre dispositivos y aplicaciones.
Broker EMQX	Gestión del intercambio de mensajes MQTT.
PostgreSQL	Sistema gestor de bases de datos para el almacenamiento de información.
psycopg	Librería utilizada para la conexión entre Python y PostgreSQL.
paho-mqtt	Cliente MQTT utilizado desde Python.
GitHub	Gestión del código fuente y control de versiones.
Draw.io	Diseño de diagramas de arquitectura y bases de datos.

A nivel funcional, el software desarrollado puede dividirse en cuatro grandes bloques. El primero corresponde a la simulación robótica implementada en RoboDK, donde se ejecutan los movimientos de los robots y los distintos elementos de la estación. El segundo bloque está formado por los programas desarrollados en Python, encargados de coordinar el funcionamiento general del sistema y gestionar las comunicaciones. El tercer bloque corresponde al sistema embebido implementado en la ESP32-S3, responsable de la interacción con sensores y actuadores físicos. Finalmente, el cuarto bloque está formado por la base de datos PostgreSQL, utilizada para almacenar la información generada durante la producción.

La integración de todos estos componentes permite construir una solución flexible, escalable y alineada con los principios de la Industria 4.0, facilitando la comunicación entre sistemas físicos y virtuales mediante protocolos estándar y tecnologías de almacenamiento de datos.

9.2 Arquitectura software y estructura general de los programas

El software desarrollado para la estación automatizada sigue una arquitectura modular distribuida, donde cada componente se encarga de una función específica dentro del sistema. Esta organización permite desacoplar las distintas funcionalidades, facilitar el mantenimiento del código y simplificar futuras ampliaciones de la instalación.

La arquitectura global está formada por cuatro bloques principales:

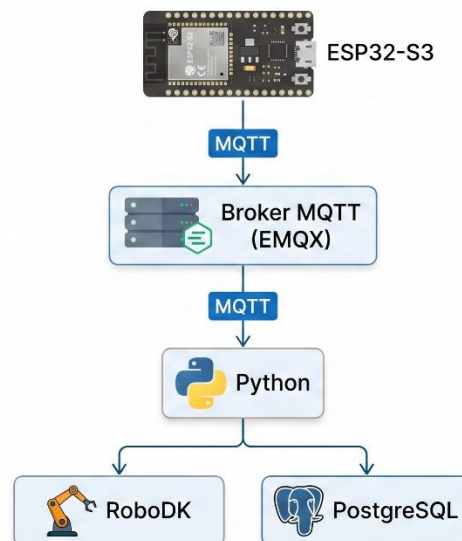
- Simulación y control robótico mediante RoboDK.
- Coordinación e integración mediante Python.
- Interacción física mediante la ESP32-S3.
- Persistencia de información mediante PostgreSQL.

La comunicación entre los diferentes componentes se realiza mediante el protocolo MQTT, utilizando un broker centralizado que permite intercambiar información entre los distintos sistemas de forma asíncrona.

Arquitectura general del sistema

El funcionamiento global del sistema puede representarse mediante la siguiente arquitectura:

IoT System Architecture Diagram: ESP32-S3, EMQX, Python, RoboDK & PostgreSQL



En esta arquitectura, Python actúa como elemento central de coordinación, gestionando la interacción entre la simulación desarrollada en RoboDK, la base de datos PostgreSQL y los dispositivos físicos controlados mediante la ESP32-S3.

División funcional del software

Con el fin de facilitar el desarrollo y mantenimiento del proyecto, el software se ha dividido en diferentes módulos especializados.

Simulación robótica

La simulación implementada en RoboDK constituye el núcleo operativo del sistema. En ella se encuentran definidos los robots industriales, las cintas transportadoras, las herramientas de manipulación y los distintos elementos que intervienen en el proceso de producción.

Dentro de RoboDK se implementan los movimientos del robot ABB IRB 360 encargados de la clasificación de piezas, así como los movimientos de los robots colaborativos UR3e responsables del proceso de marcado láser.

Además, la simulación incorpora la lógica de funcionamiento de las cintas transportadoras, los sistemas de almacenamiento y los mecanismos de sincronización necesarios para coordinar el flujo de producción.

Sistema de integración basado en Python

Python constituye la capa de integración del proyecto.

Su principal función consiste en coordinar todos los componentes que forman parte de la estación automatizada. Para ello se desarrollaron distintos scripts especializados que gestionan:

- El control de los robots.
- La lógica de clasificación.
- El control de las cintas transportadoras.
- La comunicación mediante MQTT.
- La conexión con PostgreSQL.
- La sincronización de los diferentes procesos.

La ejecución del sistema comienza mediante un programa principal encargado de lanzar los distintos módulos y garantizar su correcto funcionamiento.

La organización modular permite que cada proceso pueda ejecutarse de forma independiente, favoreciendo la reutilización del código y simplificando futuras modificaciones.



Sistema embebido basado en ESP32-S3

La ESP32-S3 constituye la interfaz física del sistema.

Su software ha sido desarrollado utilizando Arduino IDE.

Entre sus principales funciones destacan:

- Lectura del sensor de ultrasonidos.
- Lectura del sensor LDR.
- Gestión de la pantalla LCD.
- Control de LEDs indicadores.
- Gestión del pulsador de parada de emergencia.
- Comunicación MQTT con el sistema central.

La utilización de FreeRTOS permite ejecutar varias tareas simultáneamente, garantizando una respuesta rápida ante eventos y evitando bloqueos durante la operación normal del sistema.

Sistema de almacenamiento de información

El almacenamiento de datos se realiza mediante PostgreSQL.

La base de datos permite registrar información relacionada con los procesos de producción, los palillos procesados y otros elementos definidos en el modelo relacional desarrollado para el proyecto.

La persistencia de la información facilita la trazabilidad de la producción y permite realizar consultas posteriores sobre el funcionamiento de la instalación.

Flujo general de funcionamiento

El funcionamiento software de la estación automatizada comienza cuando se inicia la simulación en RoboDK.

A medida que los robots ejecutan las operaciones de clasificación y marcado, los distintos módulos Python supervisan el estado del proceso y gestionan la comunicación con el resto de los componentes.

La ESP32-S3 recibe y envía mensajes mediante MQTT relacionados con sensores, LEDs, pantalla LCD y pulsadores. Estos mensajes son procesados por Python, que actualiza el estado del sistema y registra la información relevante en PostgreSQL.

Finalmente, todos los subsistemas trabajan de forma coordinada para mantener sincronizado el proceso productivo y garantizar el correcto funcionamiento de la estación automatizada.

Esta arquitectura modular permite separar claramente las responsabilidades de cada componente, facilitando el desarrollo, las tareas de mantenimiento y la futura ampliación de la solución propuesta.

9.3 Descripción de la implementación

Con el objetivo de facilitar el desarrollo, mantenimiento y ampliación de la solución propuesta, el software de la estación automatizada se ha diseñado siguiendo una arquitectura modular. Cada programa desarrollado implementa una funcionalidad específica dentro del sistema, permitiendo desacoplar los distintos procesos que intervienen en la producción y simplificando la integración entre componentes.

La implementación software puede dividirse en cuatro grandes bloques funcionales:

- Simulación y control robótico mediante RoboDK.
- Integración y coordinación mediante Python.
- Interacción física mediante la ESP32-S3.
- Persistencia de información mediante PostgreSQL.

Esta organización permite distribuir las responsabilidades entre módulos especializados, favoreciendo la reutilización del código y facilitando futuras ampliaciones del sistema.

9.3.1 Módulos de simulación y control robótico

Los módulos de simulación se ejecutan dentro de RoboDK y son responsables de controlar el comportamiento de los robots industriales, las cintas transportadoras y los distintos elementos que forman parte de la estación automatizada.

ProgramaPrincipal.py

Este programa constituye el punto de entrada del sistema y se encarga de inicializar los distintos componentes necesarios para el funcionamiento de la estación automatizada.

Entre sus funciones principales destacan:

- Inicialización del entorno RoboDK.
- Puesta en marcha de los distintos módulos del sistema.
- Coordinación inicial de la simulación.
- Gestión global de la ejecución.



PararProgramas.py

Este módulo se ha desarrollado para gestionar la detención controlada de todos los programas que se encuentran ejecutándose dentro de la simulación.

Debido a que gran parte de los scripts implementados en RoboDK funcionan mediante bucles de ejecución continua, no es posible reiniciar correctamente la estación mientras dichos programas siguen activos. Por este motivo, antes de ejecutar cualquier procedimiento de reinicio es necesario detener previamente todos los procesos en ejecución.

Las funciones principales de este módulo son:

- Detener la ejecución de los distintos scripts activos.
- Finalizar los bucles de ejecución continua presentes en los programas.
- Restablecer los estados de control necesarios para permitir el reinicio del sistema.
- Preparar la estación para la ejecución de los scripts de restauración.

El procedimiento correcto de reinicio de la simulación es el siguiente:

1. Ejecutar **PararProgramas.py** para detener todos los procesos activos.
2. Ejecutar **ResetGeneral.py** para devolver la estación a su estado inicial.
3. Ejecutar nuevamente **ProgramaPrincipal.py** para iniciar un nuevo ciclo de funcionamiento.

Este mecanismo garantiza que no permanezcan procesos activos de ejecuciones anteriores y evita problemas de sincronización o comportamientos inesperados durante las pruebas y validaciones realizadas sobre la estación automatizada.

Delta.py

Este módulo controla el robot ABB IRB 360 encargado de la clasificación de los palillos.

Sus principales responsabilidades son:

- Recoger los palillos desde la cinta de entrada.
- Clasificar las piezas válidas y defectuosas.
- Depositar cada pieza en la ubicación correspondiente.
- Coordinar el flujo de producción con las cintas transportadoras.

SerigrafiaUR3e1.py



Controla el primer robot colaborativo UR3e utilizado en la estación de marcado.

Sus funciones principales son:

- Posicionamiento para el marcado de las piezas.
- Ejecución de trayectorias de trabajo.
- Coordinación con el segundo robot UR3e.

SerigrafiaUR3e2.py

Controla el segundo robot colaborativo UR3e.

Entre sus funciones destacan:

- Posicionamiento para el marcado de las piezas.
- Ejecución de trayectorias de trabajo.
- Coordinación con el segundo robot UR3e.

MainConveyor.py

Este módulo gestiona el funcionamiento de la cinta transportadora principal de la instalación.

Funciones principales:

- Control del movimiento de la cinta.
- Transporte de piezas entre estaciones.
- Sincronización con los robots.
- Gestión del flujo general de materiales.

StickConveyorMove.py

Controla la generación y alimentación de palillos al sistema.

Entre sus responsabilidades destacan:

- Creación de nuevos grupos de palillos.
- Posicionamiento de las piezas sobre la cinta de entrada.
- Control de la alimentación del proceso.
- Generación de registros asociados a la producción.

StickConveyorDeleteMove.py



Gestiona la retirada de piezas defectuosas detectadas durante la clasificación.

Funciones principales:

- Eliminación de palillos descartados.
- Liberación de espacio en la estación.
- Mantenimiento del flujo continuo de producción.

StopBoton.py

Implementa la lógica asociada al botón de parada de emergencia presente en la simulación.

Su función consiste en supervisar la interacción con el pulsador virtual y activar o desactivar el estado de parada del sistema, permitiendo detener temporalmente el funcionamiento de la estación cuando sea necesario.

WeldOn.py

Este módulo se utiliza para simular el funcionamiento de la herramienta de marcado empleada durante la fase de serigrafía.

Permite representar visualmente el proceso de marcado dentro de RoboDK y facilita la validación de las trayectorias ejecutadas por los robots colaborativos.

Scripts de reinicio y restauración del sistema

Con el fin de facilitar las pruebas y la validación de la estación automatizada, se desarrollaron diversos programas encargados de restablecer el estado inicial de los diferentes subsistemas presentes en la simulación.

Estos scripts permiten eliminar elementos generados durante la ejecución, reinicializar variables de control y devolver los robots y cintas transportadoras a sus posiciones de partida.

ResetGeneral.py

Este programa coordina el reinicio completo de la estación automatizada.

Entre sus funciones principales destacan:

- Reinicialización de variables globales.
- Eliminación de objetos generados durante la simulación.
- Restablecimiento del estado general del sistema.
- Ejecución de los distintos módulos de reinicio.



ResetDelta.py

Encargado de devolver el robot ABB IRB 360 a su posición inicial de trabajo.

Funciones principales:

- Restablecimiento de la posición del robot.
- Reinicialización de variables asociadas a la clasificación.
- Preparación para una nueva ejecución.

ResetMainConveyor.py

Gestiona el reinicio de la cinta transportadora principal.

Funciones principales:

- Restablecimiento de posiciones.
- Eliminación de elementos temporales.
- Reinicio de variables de transporte.

ResetStickConveyor.py

Responsable de reiniciar la cinta de alimentación de palillos.

Funciones principales:

- Eliminación de palillos pendientes.
- Restablecimiento del sistema de alimentación.
- Reinicialización de contadores de producción.

ResetStickConveyorDeleteMove.py

Encargado de reiniciar el sistema de retirada de piezas defectuosas.

Funciones principales:

- Eliminación de elementos descartados.
- Restablecimiento de posiciones.
- Reinicialización de parámetros de funcionamiento.

ResetPalillo.py

Permite eliminar los palillos generados durante la ejecución de la simulación.



Funciones principales:

- Limpieza de objetos creados dinámicamente.
- Preparación del entorno para nuevas pruebas.

ResetSoldar.py

Responsable de reiniciar el estado asociado a la estación de marcado láser.

Funciones principales:

- Eliminación de trazas generadas durante el marcado.
- Reinicialización de parámetros de serigrafía.
- Preparación para nuevos ciclos de producción.

La utilización de estos programas facilita enormemente el proceso de desarrollo y validación del sistema, ya que permite ejecutar múltiples pruebas consecutivas sin necesidad de reconstruir manualmente la simulación.

9.3.2 Módulos de integración y comunicaciones

Los módulos de integración constituyen la capa software encargada de coordinar el intercambio de información entre RoboDK, la ESP32-S3 y la base de datos PostgreSQL.

MqttListener.py

Implementa el cliente MQTT encargado de recibir mensajes procedentes de la ESP32-S3.

Sus funciones principales son:

- Conexión al broker MQTT.
- Suscripción a los topics definidos.
- Recepción de mensajes.
- Procesamiento de eventos externos.

RobotController.py

Actúa como intermediario entre los mensajes MQTT y la lógica de funcionamiento de la estación automatizada.

Funciones principales:

- Interpretación de mensajes recibidos.



- Actualización de estados del sistema.
- Coordinación de acciones dentro de RoboDK.
- Sincronización de procesos.

9.3.3 Módulos de almacenamiento de información

La persistencia de datos se implementa mediante PostgreSQL.

ConexionBasedeDatos.py

Este módulo gestiona la conexión con la base de datos.

Funciones principales:

- Apertura y cierre de conexiones.
- Ejecución de consultas SQL.
- Inserción de registros.
- Gestión de errores de comunicación.

Gracias a esta capa de persistencia es posible almacenar información histórica del funcionamiento de la estación y realizar consultas posteriores para tareas de análisis y supervisión.

9.3.4 Software de la ESP32-S3

La ESP32-S3 constituye la interfaz física de la estación automatizada. Su software ha sido desarrollado utilizando Arduino IDE y una arquitectura modular basada en múltiples archivos especializados.

Esta organización permite separar las distintas responsabilidades del sistema embebido y simplifica el mantenimiento del código.

ESP32_S3_IoT_Device_PR2_ENTREGA.ino

Archivo principal del proyecto.

Actúa como punto de entrada de la aplicación y coordina la inicialización de los distintos módulos que forman parte del sistema.

Config.h

Contiene toda la configuración global del dispositivo.

En este archivo se definen:



- Parámetros de conexión WiFi.
- Configuración MQTT.
- Topics de publicación y suscripción.
- Configuración de pines.
- Prioridades de tareas.
- Variables globales compartidas.
- Estructuras de datos utilizadas por el sistema.

d_wifi_lib_no_tocar.ino

Gestiona la conexión de la ESP32-S3 a la red WiFi.

Funciones principales:

- Inicialización de la interfaz WiFi.
- Conexión a la red inalámbrica.
- Reconexión automática en caso de pérdida de conexión.

e_mqtt_lib_no_tocar.ino

Implementa el cliente MQTT utilizado por la ESP32-S3.

Funciones principales:

- Conexión al broker MQTT.
- Suscripción a topics.
- Publicación de mensajes.
- Gestión de eventos MQTT.

f_funciones.ino

Contiene funciones auxiliares utilizadas por el resto de los módulos del sistema.

Su objetivo es centralizar operaciones comunes y evitar duplicación de código.

g_comunicaciones.ino

Implementa la lógica asociada al intercambio de información mediante MQTT.

Gestiona:



- Recepción de mensajes.
- Procesamiento de comandos.
- Publicación de información generada por sensores.

s_setup.ino

Contiene la fase de inicialización del sistema.

Durante su ejecución se realizan tareas como:

- Configuración de entradas y salidas.
- Inicialización de la pantalla LCD.
- Conexión WiFi.
- Inicialización MQTT.
- Creación de tareas FreeRTOS.

w_loop.ino

Implementa el ciclo principal de ejecución del sistema.

Su función consiste en mantener operativas las comunicaciones y garantizar el funcionamiento continuo de las distintas tareas.

c_logger.ino

Módulo encargado del registro y visualización de mensajes de depuración.

Permite supervisar el funcionamiento interno de la ESP32-S3 y facilita las tareas de desarrollo y mantenimiento.

Funcionalidades implementadas en la ESP32-S3

El software desarrollado proporciona las siguientes funcionalidades:

- Lectura del sensor ultrasónico.
- Lectura del sensor LDR.
- Gestión de la pantalla LCD.
- Control de LEDs indicadores.
- Gestión del pulsador de parada de emergencia.
- Comunicación MQTT.



- Intercambio de datos con RoboDK.
- Generación de eventos y avisos del sistema.

9.3.5 Interacción entre componentes

Todos los módulos descritos trabajan de forma coordinada para garantizar el correcto funcionamiento de la estación automatizada.

Los robots ejecutan las operaciones de clasificación y marcado dentro de RoboDK, mientras que Python coordina las comunicaciones, la lógica de control y el almacenamiento de información. Paralelamente, la ESP32-S3 gestiona los sensores y actuadores físicos, intercambiando información mediante MQTT con el sistema central.

Esta arquitectura modular permite separar claramente las responsabilidades de cada componente, facilita las tareas de mantenimiento y proporciona una base sólida para futuras ampliaciones de la instalación.

9.4 Algoritmos de programación avanzada y estructuras de datos

En el desarrollo del proyecto se han aplicado diferentes técnicas de programación orientadas a mejorar la organización, sincronización y eficiencia del sistema. Aunque no se han utilizado algoritmos clásicos como programación dinámica, sí se han implementado estructuras de datos y estrategias de control propias de sistemas embebidos, automatización industrial e integración de dispositivos.

9.4.1 Buffer circular en la ESP32-S3

Una de las estructuras de datos más importantes implementadas en el proyecto es el buffer circular utilizado en la ESP32-S3. Esta estructura permite almacenar temporalmente mensajes y eventos antes de ser procesados por las distintas tareas del sistema.

El buffer se utiliza para gestionar información relacionada con los LEDs, la pantalla LCD, el sensor ultrasónico y los mensajes enviados mediante MQTT. De esta forma se evita la pérdida de datos cuando varios eventos se producen en intervalos cortos de tiempo.

La estructura implementada contiene:

- Un array para almacenar mensajes.
- Un índice de entrada.
- Un índice de salida.
- Un contador de elementos almacenados.
- Un mecanismo de protección para evitar accesos simultáneos.

El funcionamiento del buffer sigue una lógica FIFO, es decir, el primer mensaje que entra es el primero que se procesa. Esto permite mantener el orden de llegada de los eventos y garantiza una gestión más organizada de la información.

9.4.2 Programación multitarea con FreeRTOS

El software de la ESP32-S3 se ha organizado mediante tareas independientes. Cada tarea se encarga de una función concreta, como la gestión de la pantalla LCD, el control de LEDs, la lectura del sensor LDR, la lectura del sensor ultrasónico o el envío de mensajes MQTT.

Esta división permite que el sistema pueda atender varios procesos de forma simultánea sin que una tarea bloquee completamente al resto. Por ejemplo, el sensor ultrasónico puede seguir tomando medidas mientras la pantalla LCD muestra información o mientras se recibe un mensaje MQTT.

La utilización de tareas mejora la capacidad de respuesta del sistema y facilita el mantenimiento del código, ya que cada funcionalidad se encuentra separada en bloques independientes.

9.4.3 Gestión de eventos mediante MQTT

La comunicación entre la ESP32-S3, Python y RoboDK se basa en una arquitectura orientada a eventos. En este modelo, el sistema responde a sucesos concretos, como la pulsación del botón de emergencia, la detección de un contenedor lleno o la recepción de un comando desde RoboDK.

Los principales eventos utilizados son:

Evento	Acción asociada
PARAR	Detiene la estación.
INICIAR	Reanuda la estación.
LLENO	Indica que el contenedor de rechazo está lleno.
VACIO	Indica que el contenedor tiene capacidad disponible.
MUCHA_LUZ	Señala un exceso de iluminación.
POCA_LUZ	Señala iluminación insuficiente.
ENCENDER_LED	Activa LEDs indicadores.
APAGAR_LED	Desactiva LEDs indicadores.

Este enfoque permite que el sistema no dependa de una ejecución completamente secuencial, sino que pueda reaccionar a los eventos generados durante el proceso.

9.4.4 Máquina de estados distribuida

La coordinación entre los distintos módulos del sistema se realiza mediante estados compartidos. En RoboDK se utilizan parámetros como stop, salir, serigrafia, serigrafia1, groups_created y groups_to_process, que permiten controlar el estado de la simulación y sincronizar robots, cintas y procesos auxiliares.

Por ejemplo, cuando el parámetro stop toma el valor 1, los scripts detienen temporalmente su ejecución. Cuando vuelve a 0, la estación puede continuar funcionando. Del mismo modo, los parámetros serigrafian y serigrafia1 activan los robots UR3e encargados del marcado láser.

Esta estrategia funciona como una máquina de estados distribuida, ya que cada módulo consulta el estado global del sistema y actúa en consecuencia.

9.4.5 Listas y colecciones en Python

En los scripts desarrollados en Python se utilizan listas para gestionar grupos de palillos, posiciones de recogida, posiciones de depósito y trayectorias de los robots.

Estas estructuras permiten almacenar referencias a objetos de RoboDK y recorrerlas de forma ordenada durante la ejecución del proceso. Por ejemplo, las trayectorias de serigrafía de los UR3e se organizan mediante listas de targets, lo que facilita ejecutar movimientos repetitivos de forma estructurada.

El uso de listas simplifica la programación de trayectorias y evita repetir código innecesariamente.

9.4.6 Justificación de interés

La aplicación de estas estructuras y estrategias permite mejorar la robustez y organización del sistema. El buffer circular evita pérdidas de información, la programación multitarea permite atender varios dispositivos de forma simultánea, y la máquina de estados facilita la sincronización entre todos los componentes de la estación.

Estas soluciones son especialmente adecuadas para un sistema de automatización industrial, donde varios procesos deben ejecutarse de forma coordinada y responder rápidamente ante eventos externos.

9.5 Esquema de la base de datos utilizada

La solución desarrollada utiliza una base de datos relacional PostgreSQL para almacenar información relacionada con los procesos de producción y los palillos procesados por la estación automatizada.



Durante el diseño del proyecto se definió un modelo relacional amplio que permite gestionar información sobre producción, palillos, robots, serigrafiado, stock, clientes, proveedores y compras. Este modelo proporciona una base para garantizar la trazabilidad del proceso y facilitar futuras ampliaciones del sistema.

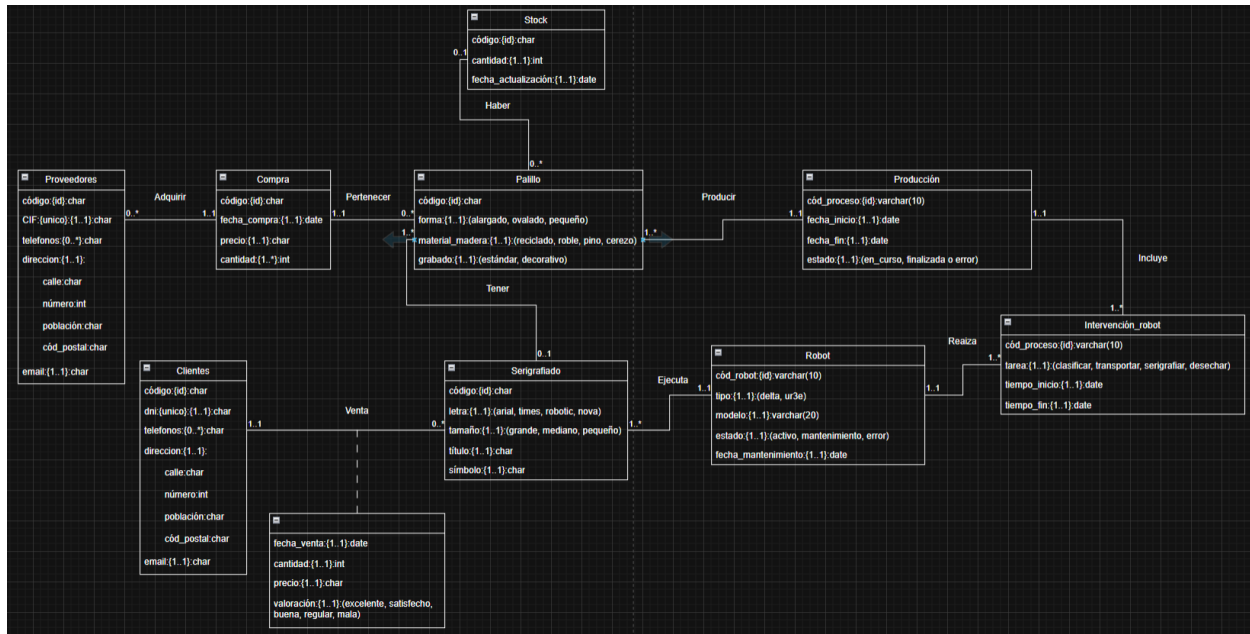
Las principales tablas del modelo son:

Tabla	Función principal
PRODUCCIÓN	Registra los procesos productivos realizados.
PALILLO	Almacena los datos de cada palillo procesado.
ROBOT	Contiene información de los robots utilizados.
INTERVENCION_ROBOT	Relaciona robots con procesos y tareas realizadas.
SERIGRAFIADO	Almacena datos del diseño o marcado aplicado.
STOCK	Gestiona las unidades disponibles.
CLIENTES	Registra información de clientes.
PROVEEDORES	Registra información de proveedores.
COMPRA	Almacena datos de compras de materia prima.

La tabla PALILLO constituye una de las entidades centrales del modelo, ya que permite asociar cada producto con su proceso de producción, el tipo de material, la forma, el grabado y otros elementos relacionados.

La tabla PRODUCCIÓN permite registrar cada proceso ejecutado, almacenando información como el identificador del proceso, fecha de inicio, fecha de fin y estado. Por otro lado, las tablas ROBOT e INTERVENCION_ROBOT permiten conocer qué robot ha intervenido en cada tarea del proceso.

Figura 9.5 1



[Enlace Imagen Diagrama Base de Datos Relacional](#)

En la implementación desarrollada, Python se conecta a PostgreSQL mediante la librería psycopg. Durante la ejecución de la simulación se generan registros asociados a procesos de producción y palillos, permitiendo comprobar la persistencia de la información y la relación entre los datos almacenados.

La utilización de una base de datos relacional permite mantener la integridad de la información mediante claves primarias, claves ajenas y restricciones de dominio. Además, facilita la consulta posterior de los datos y permite ampliar el sistema en futuras versiones para incluir estadísticas de producción, tiempos de ciclo, alarmas o registros de mantenimiento.

9.6 Repositorio de código en GitHub

El código fuente del proyecto se ha organizado en un repositorio de GitHub con el objetivo de facilitar el control de versiones, el trabajo colaborativo y la trazabilidad de las contribuciones realizadas por los distintos miembros del equipo.

El repositorio incluye los principales archivos desarrollados durante el proyecto:

- Scripts Python de RoboDK.
- Código de comunicación MQTT.
- Código de conexión con PostgreSQL.
- Programa de la ESP32-S3.

- Archivos de configuración.
- Scripts SQL de la base de datos.
- Archivos auxiliares de la simulación.

[Github del proyecto](#)

La estructura del repositorio permite separar los distintos bloques del proyecto, facilitando la localización de los archivos y el mantenimiento del sistema.

Carpeta / bloque	Contenido
RoboDK / Python / MQTT	Scripts de control de robots, cintas y simulación.
ESP32-S3	Código Arduino para sensores, actuadores y comunicaciones.
Base de datos	Scripts SQL y documentación del modelo relacional.
Documentación	Diagramas, capturas y material complementario.

El uso de GitHub ha permitido mantener un registro ordenado del desarrollo, reducir el riesgo de pérdida de información y facilitar la integración de los distintos módulos software desarrollados durante el proyecto.

10. Conclusiones y recomendaciones

10.1 Conclusiones

El desarrollo de este proyecto ha permitido diseñar e implementar una propuesta integral de automatización industrial basada en los principios de la Industria 4.0, combinando tecnologías de simulación robótica, comunicaciones industriales, sistemas embebidos y gestión de datos en una única arquitectura de trabajo. El resultado obtenido demuestra la viabilidad de integrar diferentes herramientas software y hardware para crear una estación automatizada capaz de supervisar, coordinar y ejecutar tareas de forma autónoma y eficiente.

La motivación principal para la realización de este proyecto surge de la creciente necesidad de digitalización e interconexión de los procesos productivos industriales. En los entornos de fabricación modernos, la capacidad de adquirir información en tiempo real, procesarla y tomar decisiones basadas en datos constituye uno de los pilares fundamentales para mejorar la productividad, la trazabilidad y la flexibilidad de las líneas de producción. Bajo esta premisa, se ha

planteado una solución que reproduce a pequeña escala muchas de las tecnologías y metodologías utilizadas actualmente en la industria.

La propuesta desarrollada ha permitido integrar una célula robotizada simulada mediante RoboDK con una infraestructura de comunicaciones basada en MQTT, una base de datos PostgreSQL destinada al almacenamiento de información y una plataforma hardware basada en ESP32-S3 encargada de la adquisición y gestión de señales procedentes de sensores. Esta integración ha dado lugar a un sistema distribuido en el que cada componente desempeña una función específica, pero donde todos ellos trabajan de forma coordinada para alcanzar un objetivo común.

Uno de los aspectos más relevantes del proyecto ha sido la creación de una arquitectura modular. Esta filosofía de diseño permite que cada elemento pueda evolucionar de manera independiente sin afectar significativamente al resto del sistema. Gracias a ello, la solución propuesta puede ampliarse fácilmente mediante la incorporación de nuevos sensores, robots, estaciones de trabajo o sistemas de supervisión, adaptándose a futuros requisitos industriales sin necesidad de rediseñar completamente la infraestructura existente.

A lo largo del desarrollo se han alcanzado satisfactoriamente los objetivos inicialmente planteados. En primer lugar, se ha conseguido establecer una comunicación fiable entre los diferentes componentes del sistema utilizando protocolos estándar ampliamente empleados en entornos industriales. En segundo lugar, se ha implementado una base de datos capaz de registrar eventos, estados y parámetros operativos, permitiendo almacenar información histórica del funcionamiento de la estación. Finalmente, se ha desarrollado una lógica de control que coordina las diferentes acciones necesarias para la ejecución de los procesos simulados.

Durante la realización del proyecto también han surgido diversos desafíos técnicos. La integración de tecnologías heterogéneas ha supuesto uno de los mayores retos encontrados. Cada herramienta utilizada dispone de sus propias características, protocolos de comunicación y requisitos de funcionamiento, lo que ha obligado a diseñar mecanismos de interoperabilidad que garantizaran un intercambio correcto de información entre todos los elementos involucrados.

Otro desafío importante ha sido la sincronización de procesos. En sistemas distribuidos resulta fundamental asegurar que las distintas aplicaciones compartan información actualizada y reaccionen adecuadamente ante los eventos generados por otros componentes. La gestión de estados, la coordinación temporal y la detección de incidencias han requerido un análisis detallado para evitar inconsistencias y asegurar un funcionamiento estable del sistema.

Asimismo, el diseño de la estructura de datos y de la lógica de almacenamiento ha requerido una planificación cuidadosa. La utilización de PostgreSQL no solo ha permitido almacenar información

relevante sobre el proceso, sino también sentar las bases para futuras aplicaciones de análisis de datos, generación de informes y monitorización avanzada de la producción.

Desde una perspectiva técnica, el proyecto ha puesto de manifiesto la importancia de la comunicación entre dispositivos en los sistemas industriales modernos. La utilización de MQTT ha demostrado ser una solución eficiente para la transmisión de información en tiempo real gracias a su bajo consumo de recursos, simplicidad de implementación y elevada escalabilidad. Estas características explican su creciente adopción en aplicaciones relacionadas con el Internet Industrial de las Cosas (IIoT).

Por otra parte, el uso de RoboDK como herramienta de simulación ha permitido validar la lógica de funcionamiento de la estación sin necesidad de disponer inicialmente de todos los recursos físicos. Esta aproximación reduce costes de desarrollo, facilita la detección temprana de errores y permite realizar pruebas de funcionamiento antes de la puesta en marcha real del sistema. Además, reproduce una metodología de trabajo ampliamente utilizada en el ámbito industrial actual, donde los gemelos digitales y las simulaciones virtuales adquieren cada vez mayor relevancia.

Desde el punto de vista formativo, este proyecto ha supuesto una oportunidad para aplicar de forma práctica conocimientos adquiridos durante distintas asignaturas del grado. La naturaleza multidisciplinar del trabajo ha permitido combinar conceptos de programación, bases de datos, sistemas empotrados, robótica, comunicaciones industriales y automatización, demostrando cómo todas estas áreas se complementan para resolver problemas reales de ingeniería.

Especialmente relevante ha sido la comprensión de cómo los conceptos estudiados de forma individual durante la carrera convergen en un proyecto real. Aspectos como la programación concurrente, el diseño de arquitecturas software, el modelado de bases de datos, los protocolos de comunicación o el control de sistemas automatizados han dejado de ser conocimientos aislados para convertirse en elementos interdependientes dentro de una solución global.

Además de los conocimientos técnicos adquiridos, el desarrollo del proyecto ha permitido fortalecer competencias transversales fundamentales para el ejercicio profesional de la ingeniería. Entre ellas destacan la capacidad de planificación, la resolución de problemas complejos, el análisis crítico de soluciones técnicas, la búsqueda de documentación especializada y la integración de tecnologías procedentes de distintos ámbitos.

En términos generales, los resultados obtenidos permiten concluir que la propuesta desarrollada cumple satisfactoriamente los objetivos establecidos y constituye una demostración práctica de cómo las tecnologías asociadas a la Industria 4.0 pueden integrarse para crear sistemas automatizados inteligentes, conectados y preparados para futuras ampliaciones. La experiencia

adquirida durante el desarrollo proporciona una base sólida para abordar proyectos de mayor complejidad en entornos industriales reales.

10.2 Lecciones aprendidas

Durante la realización del proyecto se han extraído diversas enseñanzas que resultarán de gran utilidad en futuros desarrollos.

La primera de ellas es la importancia de diseñar arquitecturas modulares desde las fases iniciales del proyecto. Una correcta separación de responsabilidades simplifica el mantenimiento, mejora la escalabilidad y reduce significativamente la complejidad del sistema.

También se ha comprobado la relevancia de la documentación técnica. La utilización de herramientas procedentes de distintos fabricantes y entornos obliga a consultar documentación especializada de manera constante, por lo que la capacidad para interpretar manuales técnicos y documentación oficial resulta fundamental.

Otra lección importante ha sido comprender que la integración entre sistemas suele representar una parte significativa del esfuerzo total de desarrollo. Aunque cada componente funcione correctamente de forma individual, garantizar que todos ellos interactúen de forma coordinada requiere un importante trabajo de diseño, pruebas y validación.

Finalmente, el proyecto ha permitido valorar la importancia creciente de la digitalización industrial y del análisis de datos como herramientas clave para mejorar la eficiencia, la flexibilidad y la capacidad de adaptación de los sistemas productivos modernos.

10.3 Recomendaciones y líneas futuras de trabajo

Aunque el sistema desarrollado cumple los objetivos planteados inicialmente, existen diversas mejoras que podrían implementarse en futuras versiones con el fin de aumentar las capacidades de la estación automatizada y acercarla aún más a un entorno industrial real.

Una de las principales líneas de mejora sería la implementación física completa de la estación automatizada. Aunque la simulación realizada en RoboDK ha permitido validar el funcionamiento general del sistema, la construcción de un prototipo físico permitiría verificar aspectos relacionados con tolerancias mecánicas, tiempos reales de operación, precisión de los robots y comportamiento de los sensores en condiciones reales de trabajo.

Otra posible mejora consistiría en incorporar sistemas de visión artificial industrial para realizar tareas de inspección automática. Mediante cámaras industriales y algoritmos de procesamiento de imagen sería posible detectar defectos, verificar la calidad de los palillos o identificar automáticamente diferentes tipos de productos sin necesidad de intervención humana.

También sería interesante desarrollar una interfaz gráfica de supervisión que permitiera visualizar en tiempo real el estado de la instalación. Esta interfaz podría mostrar información procedente de la base de datos, alarmas generadas por los sensores, estados de los robots, estadísticas de producción y otros indicadores relevantes para la gestión del proceso.

Desde el punto de vista de las comunicaciones industriales, el sistema podría ampliarse incorporando protocolos ampliamente utilizados en entornos industriales reales, como OPC-UA o Modbus TCP, permitiendo una integración más sencilla con sistemas SCADA, PLCs industriales y plataformas de supervisión avanzadas.

Otra línea de trabajo futura sería la aplicación de técnicas de análisis de datos e inteligencia artificial. Gracias a la información almacenada en PostgreSQL podrían desarrollarse modelos capaces de detectar patrones de funcionamiento, optimizar tiempos de ciclo, identificar anomalías o incluso predecir necesidades de mantenimiento antes de que se produzcan fallos en la instalación.

Asimismo, podría ampliarse la funcionalidad de la base de datos incorporando registros históricos de producción, incidencias, mantenimientos realizados y estadísticas de rendimiento, permitiendo realizar análisis más detallados sobre la eficiencia global de la estación automatizada.

Finalmente, se recomienda continuar evolucionando la arquitectura modular desarrollada durante el proyecto, ya que esta facilita la incorporación de nuevos dispositivos y funcionalidades sin necesidad de modificar significativamente el diseño actual del sistema.

11. Referencias

Bibliografía y documentación técnica

- [1] RoboDK Inc. *RoboDK Documentation*. Disponible en: <https://robodk.com/doc>
- [2] PostgreSQL Global Development Group. *PostgreSQL Documentation*. Disponible en: <https://www.postgresql.org/docs/>
- [3] Eclipse Foundation. *MQTT Version 5.0 Specification*. Disponible en: <https://mqtt.org/>
- [4] Espressif Systems. *ESP32-S3 Technical Reference Manual*. Disponible en: <https://docs.espressif.com/>
- [5] Espressif Systems. *ESP-IDF Programming Guide*. Disponible en: <https://docs.espressif.com/>
- [6] Python Software Foundation. *Python Documentation*. Disponible en: <https://docs.python.org/>
- [7] Arduino. *Arduino Language Reference*. Disponible en: <https://www.arduino.cc/reference/>
- [8] Universal Robots. *UR3e User Manual*. Disponible en: <https://www.universal-robots.com/>

- [9] FreeRTOS. *FreeRTOS Documentation*. Disponible en: <https://www.freertos.org/>
- [10] Silberschatz, A.; Korth, H.; Sudarshan, S. *Fundamentos de Bases de Datos*. McGraw-Hill.
- [11] Groover, M. P. *Automation, Production Systems and Computer-Integrated Manufacturing*. Pearson Education.

Uso de herramientas de Inteligencia Artificial

Durante el desarrollo de la memoria y del proyecto se han utilizado herramientas de inteligencia artificial generativa, concretamente ChatGPT (OpenAI), como apoyo para tareas de redacción técnica, revisión de textos, mejora de la estructura documental y generación de propuestas de organización de contenidos. Además se ha utilizado Gemini, también para la generación de imágenes concretas.

Las aportaciones obtenidas mediante estas herramientas han sido utilizadas exclusivamente como apoyo al trabajo realizado por los autores del proyecto. Todas las decisiones técnicas, el diseño de la arquitectura, la implementación del software, la integración de los sistemas y la validación de resultados han sido desarrolladas y verificadas por los integrantes del equipo.

La utilización de herramientas de IA ha permitido mejorar la claridad expositiva de la memoria, revisar la redacción de determinados apartados y proponer estructuras documentales acordes con los requisitos académicos establecidos.

12. Anexos

12.1 Manual de instalación y funcionamiento

El presente manual describe los pasos necesarios para instalar, configurar y ejecutar la solución de automatización desarrollada. Su objetivo es proporcionar una guía completa que permita desplegar el sistema desde cero, incluyendo la instalación del software necesario, la configuración de la base de datos, la puesta en marcha del broker MQTT, la programación de la ESP32-S3 y la ejecución de la simulación desarrollada en RoboDK.

La solución propuesta se compone de diversos elementos hardware y software que trabajan de forma integrada mediante una arquitectura basada en MQTT. Para garantizar el correcto funcionamiento del sistema, es necesario seguir el orden de instalación y configuración descrito en los siguientes apartados.

12.1.1 Requisitos previos

Antes de iniciar la instalación del sistema, es necesario disponer de los recursos hardware y software necesarios para la ejecución de la estación automatizada.



Hardware necesario

La parte física del proyecto está basada en una placa ESP32-S3 conectada a diferentes sensores y actuadores utilizados para supervisar y controlar determinados aspectos de la estación automatizada.

Los elementos hardware utilizados son los siguientes:

Elemento	Función
ESP32-S3	Nodo IoT encargado de gestionar sensores, actuadores y comunicaciones MQTT.
Pantalla LCD 16x2 I2C	Visualización de mensajes y estados del sistema.
Sensor ultrasónico HC-SR04	Supervisión del nivel de llenado del contenedor de rechazo.
Sensor LDR	Medición del nivel de iluminación ambiental.
LED RGB de entrada	Indicación visual del estado de la cinta de entrada.
LED RGB de salida	Indicación visual del estado de la cinta de salida.
Pulsador de control	Permite detener y reanudar el funcionamiento de la estación.
Resistencias de 220 Ω	Limitación de corriente para los LEDs RGB.
Resistencias de 1 k Ω y 2 k Ω	Adaptación de nivel del pin ECHO del sensor HC-SR04.
Resistencia de 10 k Ω	Divisor de tensión utilizado por el sensor LDR.
Cable USB	Programación y alimentación de la ESP32-S3.

Software necesario

La solución integra diferentes herramientas software que permiten desarrollar la simulación, gestionar las comunicaciones, almacenar datos y programar los dispositivos electrónicos.

Los programas necesarios son los siguientes:

Software	Función
RoboDK	Simulación de la estación automatizada y programación de los robots.
Python 3.x	Ejecución de los scripts de integración y control.
PostgreSQL	Sistema gestor de bases de datos utilizado para almacenar la información generada durante la producción.
pgAdmin 4	Herramienta gráfica para la administración de PostgreSQL.
EMQX Broker	Broker MQTT utilizado para la comunicación entre los diferentes componentes del sistema.
Arduino IDE	Desarrollo y programación de la ESP32-S3.

Librerías Python

Los scripts desarrollados en Python requieren la instalación de las dependencias necesarias para la comunicación MQTT, el acceso a la base de datos y la interacción con RoboDK.

Las librerías utilizadas pueden variar ligeramente en función del entorno de instalación, aunque las principales dependencias empleadas en el proyecto son las siguientes:

Librería	Función
paho-mqtt	Comunicación MQTT.
psycopg	Conexión con PostgreSQL.
RoboDK API	Interacción con la estación de simulación.
threading	Ejecución concurrente de tareas.

Librerías utilizadas por la ESP32-S3

El firmware desarrollado para la ESP32-S3 utiliza las siguientes librerías instaladas mediante Arduino IDE:

Librería	Versión	Función
Arduino ESP32 Boards	2.0.18	Soporte para la programación de la placa ESP32-S3.
EventButton	1.0.3	Gestión de eventos asociados al pulsador de control.
PubSubClient	2.8	Comunicación MQTT con el broker EMQX.
LiquidCrystal_I2C	2.0.0	Control de la pantalla LCD mediante protocolo I2C.

El paquete Arduino ESP32 Boards incorpora además diversas bibliotecas internas utilizadas por el proyecto, entre las que destacan WiFi para la conexión inalámbrica, Wire para la comunicación I2C y WiFiClientSecure para comunicaciones seguras mediante TLS/SSL.

12.1.2 Instalación de RoboDK

RoboDK constituye el entorno principal de simulación utilizado en el proyecto. En él se ha desarrollado la estación automatizada, la programación de los robots industriales y la integración con los diferentes módulos software desarrollados en Python.

Para instalar RoboDK se deben seguir los siguientes pasos:

1. Acceder a la página oficial de RoboDK:

<https://robodk.com>

2. Descargar la versión correspondiente al sistema operativo utilizado.
3. Ejecutar el instalador descargado.
4. Seguir el asistente de instalación aceptando la configuración predeterminada.
5. Finalizar la instalación y comprobar que RoboDK se ejecuta correctamente.

Una vez instalado, se recomienda verificar que el programa inicia sin errores y que es posible abrir estaciones de simulación.

Apertura de la estación del proyecto

Tras completar la instalación, se debe abrir el archivo de estación desarrollado para el proyecto.

Para ello:

1. Iniciar RoboDK.
2. Seleccionar **Archivo** → **Abrir estación**.



3. Cargar el archivo correspondiente a la estación automatizada desarrollada.
4. Verificar que todos los robots, cintas transportadoras, sensores virtuales y elementos de la simulación aparecen correctamente cargados.

Verificación de funcionamiento

Antes de continuar con la instalación del resto de componentes se recomienda comprobar:

Comprobación	Resultado esperado
Apertura de RoboDK	El programa inicia correctamente.
Carga de la estación	Todos los elementos aparecen correctamente.
Visualización 3D	La estación se muestra sin errores gráficos.
Árbol de elementos	Robots, programas y objetos visibles.
Simulación manual	Es posible ejecutar movimientos de prueba.

La correcta instalación de RoboDK es fundamental para el funcionamiento del proyecto, ya que actúa como núcleo de la simulación y coordina gran parte de los procesos automatizados desarrollados.

12.1.3 Instalación de Python

Python constituye la capa de integración principal del proyecto. Los diferentes scripts desarrollados permiten la comunicación entre RoboDK, PostgreSQL, MQTT y la ESP32-S3, actuando como elemento coordinador de la estación automatizada.

Para instalar Python se deben seguir los siguientes pasos:

1. Acceder a la página oficial de Python:

<https://www.python.org/downloads/>

2. Descargar la versión más reciente compatible con el sistema operativo utilizado.
3. Ejecutar el instalador descargado.
4. Durante la instalación, activar la opción:

Add Python to PATH

5. Completar el proceso de instalación siguiendo la configuración predeterminada.



6. Reiniciar el equipo si fuera necesario.

Verificación de la instalación

Una vez instalado Python, se recomienda comprobar que la instalación se ha realizado correctamente.

Para ello, abrir una terminal o símbolo del sistema y ejecutar el siguiente comando:

```
python --version
```

Si la instalación se ha realizado correctamente, el sistema mostrará la versión instalada de Python.

También puede verificarse mediante:

```
pip --version
```

Instalación de dependencias

Los scripts desarrollados para el proyecto requieren la instalación de diversas librerías adicionales.

La instalación puede realizarse desde una terminal utilizando el gestor de paquetes pip.

Las principales dependencias utilizadas son:

```
pip install paho-mqtt
```

```
pip install pycopg
```

```
pip install robodk
```

Dependiendo de la versión utilizada y de las funcionalidades instaladas, pueden ser necesarias librerías adicionales utilizadas por algunos módulos auxiliares del proyecto.

Estructura de los scripts

Los programas desarrollados en Python realizan diferentes funciones dentro de la arquitectura del sistema:

Módulo	Función principal
Integración MQTT	Recepción y envío de eventos entre dispositivos.
Control RoboDK	Gestión de la simulación y ejecución de programas.

Módulo	Función principal
Gestión PostgreSQL	Inserción y consulta de datos en la base de datos.
Coordinación de procesos	Sincronización de los diferentes módulos del sistema.
Interfaces auxiliares	Ventanas de control y supervisión.

Verificación de funcionamiento

Una vez instaladas todas las dependencias, se recomienda ejecutar uno de los scripts principales del proyecto para verificar que el entorno se encuentra correctamente configurado.

La ejecución sin errores de los módulos de comunicación MQTT, acceso a PostgreSQL y control de RoboDK confirma que el entorno Python está preparado para integrarse con el resto de los componentes de la estación automatizada.

La correcta instalación de Python es fundamental para el funcionamiento global del sistema, ya que actúa como punto de unión entre la simulación desarrollada en RoboDK, la base de datos PostgreSQL y los dispositivos físicos controlados mediante la ESP32-S3.

12.1.4 Instalación de PostgreSQL y pgAdmin

PostgreSQL es el sistema gestor de bases de datos utilizado en el proyecto para almacenar la información generada durante la ejecución de la estación automatizada. Los diferentes módulos software desarrollados en Python registran información relacionada con la producción, la clasificación de los palillos, el proceso de serigrafiado y otros eventos generados durante la simulación.

Para facilitar la administración de la base de datos se utiliza pgAdmin 4, herramienta gráfica que permite crear bases de datos, ejecutar consultas SQL y supervisar el estado del servidor PostgreSQL.

Instalación de PostgreSQL

Para instalar PostgreSQL se deben seguir los siguientes pasos:

1. Acceder a la página oficial del proyecto:

<https://www.postgresql.org/download/>

2. Descargar la versión correspondiente al sistema operativo utilizado.
3. Ejecutar el instalador descargado.

4. Seleccionar los componentes a instalar:
 - PostgreSQL Server.
 - pgAdmin 4.
 - Command Line Tools.
5. Definir una contraseña para el usuario administrador **postgres(password = postgres)**.
6. Seleccionar el puerto de comunicación (por defecto 5432).
7. Completar la instalación siguiendo la configuración predeterminada.
8. Finalizar el proceso e iniciar el servicio PostgreSQL.

Verificación de la instalación

Una vez completada la instalación, se recomienda verificar que PostgreSQL se encuentra operativo.

Para ello se puede abrir pgAdmin 4 y comprobar que el servidor local aparece disponible y en ejecución.

También es posible realizar una comprobación básica mediante la siguiente consulta SQL:

```
SELECT version();
```

Si la instalación se ha realizado correctamente, PostgreSQL devolverá la versión instalada del servidor.

Configuración inicial de la base de datos

Una vez instalado PostgreSQL, se debe crear la estructura de la base de datos utilizada por el proyecto.

Los pasos son los siguientes:

1. Abrir pgAdmin 4.
2. Conectarse al servidor PostgreSQL local.
3. Crear una nueva base de datos denominada **produccion_palillos**.
4. Abrir la herramienta **Query Tool**.
5. Ejecutar el script SQL denominado **SQL_PALILLOS.sql**. [Enlace al archivo](#)

Este script crea el esquema principal de la base de datos y genera todas las tablas necesarias para el funcionamiento del sistema.



Entre las entidades implementadas se encuentran las relacionadas con clientes, proveedores, stock, producción, transporte, calidad, serigrafiado, robots, almacén y pedidos, así como las tablas auxiliares necesarias para mantener la integridad de la información.

Uso de pgAdmin

pgAdmin 4 se utiliza durante el desarrollo y las pruebas para realizar tareas de administración y supervisión de la base de datos.

Entre las operaciones más habituales se encuentran:

Operación	Descripción
Creación de bases de datos	Generación de nuevas bases de datos.
Ejecución de scripts SQL	Creación y modificación de tablas.
Consultas SQL	Visualización de registros almacenados.
Administración de usuarios	Gestión de permisos y accesos.
Supervisión del servidor	Comprobación del estado de PostgreSQL.

Verificación de funcionamiento

Tras ejecutar ambos scripts, se recomienda comprobar que la base de datos se ha creado correctamente.

Para ello pueden realizarse consultas de prueba sobre algunas de las tablas creadas:

```
SELECT * FROM produccion_palillos.robot;
```

```
SELECT * FROM produccion_palillos.clientes;
```

Si las consultas devuelven resultados sin errores, la base de datos se encuentra correctamente configurada y preparada para ser utilizada por los módulos software del proyecto.

Comprobaciones finales

Antes de continuar con la instalación del resto de componentes del sistema, se recomienda verificar que:



Comprobación	Resultado esperado
PostgreSQL iniciado	Servicio operativo.
pgAdmin accesible	Apertura sin errores.
Base de datos creada	Visible en pgAdmin.
Scripts SQL ejecutados	Sin errores de ejecución.
Tablas generadas	Estructura correcta.
Datos de prueba cargados	Registros visibles en las consultas.
Conexión desde Python	Acceso correcto a PostgreSQL.

La correcta instalación y configuración de PostgreSQL resulta fundamental para el funcionamiento del proyecto, ya que permite almacenar y consultar toda la información generada durante la ejecución de la estación automatizada, facilitando el seguimiento y la trazabilidad de los procesos simulados.

12.1.5 Instalación y configuración de MQTTX

Durante el desarrollo y las pruebas del proyecto se ha utilizado MQTTX como herramienta de monitorización y depuración de las comunicaciones MQTT. Esta aplicación permite visualizar en tiempo real los mensajes intercambiados entre la ESP32-S3, los scripts desarrollados en Python y la simulación ejecutada en RoboDK.

MQTTX no actúa como broker MQTT, sino como cliente de supervisión, permitiendo publicar mensajes manualmente, suscribirse a topics y verificar el correcto funcionamiento de las comunicaciones.

Instalación de MQTTX

Para instalar MQTTX se deben seguir los siguientes pasos:

1. Acceder a la página oficial del proyecto:

<https://mqttx.app>

2. Descargar la versión correspondiente al sistema operativo utilizado.
3. Ejecutar el instalador descargado.

4. Completar la instalación siguiendo la configuración predeterminada.
5. Iniciar la aplicación MQTTX.

Configuración de la conexión

Una vez iniciada la aplicación, se debe crear una nueva conexión MQTT.

Los principales parámetros de conexión son:

Parámetro	Valor
Nombre de conexión	Proyecto Palillos
Broker MQTT	Broker público utilizado durante las pruebas
Puerto	1883
Client ID	Identificador libre definido por el usuario. No definir
Usuario	Según configuración del broker. No definir
Contraseña	Según configuración del broker. No definir

Suscripción a topics

MQTTX permite suscribirse a los distintos topics utilizados por el sistema para visualizar los mensajes intercambiados durante la ejecución de la estación automatizada.

Durante las pruebas, esta herramienta se ha utilizado para:

- Verificar la recepción de mensajes procedentes de la ESP32-S3.
- Supervisar los eventos generados por sensores y actuadores.
- Comprobar la correcta publicación de mensajes desde los scripts Python.
- Validar la sincronización entre RoboDK y el resto de los componentes del sistema.

Verificación de funcionamiento

Una vez establecida la conexión con el broker MQTT, se recomienda comprobar:



Comprobación	Resultado esperado
Conexión MQTT establecida	Cliente conectado correctamente.
Recepción de mensajes	Visualización de eventos MQTT.
Publicación de mensajes	Mensajes enviados correctamente.
Suscripción a topics	Recepción de información en tiempo real.

La utilización de MQTTX facilita las tareas de depuración y validación del sistema, permitiendo observar el flujo de mensajes MQTT intercambiados entre todos los componentes de la arquitectura desarrollada.

12.1.6 Instalación y configuración de Arduino IDE y la ESP32-S3

La ESP32-S3 constituye el elemento hardware encargado de gestionar los sensores y actuadores físicos integrados en la estación automatizada. Además de realizar la lectura de sensores y el control de dispositivos de señalización, la placa permite intercambiar información con el resto de los componentes del sistema mediante el protocolo MQTT.

Para desarrollar y cargar el firmware de la ESP32-S3 se utiliza Arduino IDE junto con el soporte oficial para placas ESP32.

Instalación de Arduino IDE

Para instalar Arduino IDE se deben seguir los siguientes pasos:

1. Acceder a la página oficial de Arduino:

<https://www.arduino.cc/en/software>

2. Descargar la versión correspondiente al sistema operativo utilizado.
3. Ejecutar el instalador descargado.
4. Completar la instalación utilizando la configuración predeterminada.
5. Iniciar Arduino IDE.

Instalación del soporte para ESP32-S3

Una vez instalado Arduino IDE, es necesario añadir el soporte para las placas ESP32.

Para ello:



1. Abrir Arduino IDE.

2. Acceder a:

Archivo → Preferencias

3. En el apartado "Gestor de URLs adicionales de tarjetas" añadir la siguiente dirección:

https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json

4. Acceder a:

Herramientas → Placa → Gestor de tarjetas

5. Buscar:

esp32

6. Instalar la versión:

Arduino ESP32 Boards 2.0.18

Una vez completada la instalación, Arduino IDE dispondrá del soporte necesario para programar la ESP32-S3.

Instalación de librerías

El firmware desarrollado para el proyecto utiliza diversas librerías externas que deben instalarse desde el Gestor de Librerías de Arduino IDE.

Las versiones utilizadas durante el desarrollo del proyecto son las siguientes:

Librería	Versión	Función
EventButton	1.0.3	Gestión de eventos asociados al pulsador.
PubSubClient	2.8	Comunicación MQTT.
LiquidCrystal_I2C	2.0.0	Control de la pantalla LCD I2C.

Además, el paquete Arduino ESP32 Boards incorpora las bibliotecas internas necesarias para la conexión WiFi, la comunicación I2C y las comunicaciones seguras utilizadas por la ESP32-S3.

Configuración del firmware

Una vez instaladas las librerías necesarias, se debe abrir el proyecto de la ESP32-S3 desde Arduino IDE.



Antes de compilar el programa es necesario revisar los parámetros definidos en el archivo de configuración del proyecto.

Entre los parámetros que deben configurarse se encuentran:

- Nombre de la red WiFi.
- Contraseña de acceso a la red WiFi.
- Dirección del broker MQTT utilizado.
- Puerto de comunicación MQTT.
- Topics utilizados para el intercambio de mensajes.
- Identificador del dispositivo.

La correcta configuración de estos parámetros garantiza la comunicación entre la ESP32-S3 y el resto de los componentes del sistema.

Carga del programa

Una vez revisada la configuración:

1. Conectar la ESP32-S3 al ordenador mediante cable USB.
2. Seleccionar la placa correspondiente desde el menú Herramientas.
3. Seleccionar el puerto serie asignado por el sistema operativo.
4. Compilar el proyecto.
5. Cargar el firmware en la placa.

Tras finalizar la carga, la ESP32-S3 se reiniciará automáticamente y comenzará a ejecutar el programa.

Verificación de funcionamiento

Para comprobar que la programación se ha realizado correctamente se recomienda utilizar el Monitor Serie de Arduino IDE.

Durante el arranque deben observarse mensajes relacionados con:

- Inicialización del sistema.
- Conexión a la red WiFi.
- Conexión al broker MQTT.

- Inicialización de la pantalla LCD.
- Configuración de sensores y actuadores.

Si todas estas operaciones se completan correctamente, la ESP32-S3 estará preparada para integrarse con la estación automatizada.

Funcionalidades implementadas

El firmware desarrollado permite gestionar los distintos dispositivos físicos utilizados en el proyecto:

Dispositivo	Función
Pantalla LCD I2C	Visualización de mensajes procedentes del sistema.
Sensor ultrasónico HC-SR04	Supervisión del nivel de llenado del contenedor de rechazo.
Sensor LDR	Medición de la iluminación ambiental.
LEDs RGB	Indicación visual del estado de las cintas transportadoras.
Pulsador	Parada y reanudación del sistema.
MQTT	Comunicación con RoboDK y los scripts Python.

Gracias a esta arquitectura, la ESP32-S3 actúa como interfaz física de la estación automatizada, permitiendo conectar el entorno virtual de RoboDK con dispositivos reales mediante tecnologías IoT.

12.1.7 Configuración del proyecto

Una vez instalados todos los componentes software y hardware necesarios, es necesario realizar una serie de configuraciones para garantizar la correcta comunicación entre RoboDK, Python, PostgreSQL, MQTT y la ESP32-S3.

La arquitectura desarrollada se basa en una estructura distribuida en la que cada componente desempeña una función específica dentro del sistema de automatización. Por ello, antes de ejecutar la simulación es necesario verificar que todos los parámetros de configuración son correctos.

Configuración de PostgreSQL



La base de datos PostgreSQL debe encontrarse operativa y accesible desde los scripts desarrollados en Python.

Para ello se debe verificar:

- Que el servidor PostgreSQL se encuentra iniciado.
- Que la base de datos del proyecto ha sido creada correctamente.
- Que las tablas y registros iniciales han sido cargados mediante los scripts SQL.
- Que las credenciales de acceso utilizadas por los scripts Python son correctas.

Los principales parámetros de conexión son:

Parámetro	Descripción
Host	Dirección del servidor PostgreSQL. Host = localhost
Puerto	Puerto de conexión (5432 por defecto).
Base de datos	Base de datos utilizada por el proyecto.
Usuario	Usuario con permisos de acceso. Sin usuario
Contraseña	Contraseña asociada al usuario. Sin contraseña

Configuración de MQTT

La comunicación entre los distintos módulos del sistema se realiza mediante MQTT.

Antes de iniciar la ejecución del proyecto se debe comprobar:

- Disponibilidad del broker MQTT.
- Correcta configuración de la dirección del broker.
- Puerto de comunicación configurado.
- Acceso a Internet en caso de utilizar un broker público.
- Correcta definición de los topics utilizados por el proyecto.

La comunicación MQTT es utilizada por:

- ESP32-S3.



- Scripts Python.

Configuración de la ESP32-S3

La ESP32-S3 debe encontrarse programada con la versión más reciente del firmware desarrollado para el proyecto.

Antes de iniciar el sistema se recomienda verificar:

- Conexión correcta a la red WiFi.
- Conexión correcta al broker MQTT.
- Funcionamiento de la pantalla LCD.
- Funcionamiento de los LEDs RGB.
- Funcionamiento del pulsador.
- Lectura correcta del sensor ultrasónico.
- Lectura correcta del sensor LDR.

La verificación puede realizarse mediante el monitor serie de Arduino IDE o mediante MQTTX observando los mensajes generados por la placa.

Configuración de RoboDK

La estación de simulación desarrollada en RoboDK debe abrirse correctamente y contener todos los elementos necesarios para la ejecución del proceso automatizado.

Se recomienda verificar:

- Presencia de los robots ABB Delta y UR3e.
- Disponibilidad de los programas de simulación.
- Correcta carga de los modelos 3D.
- Configuración de las cintas transportadoras.
- Existencia de los objetos utilizados durante las pruebas.

Asimismo, se debe comprobar que los scripts de integración desarrollados en Python pueden comunicarse correctamente con RoboDK mediante la API correspondiente.

Configuración de los scripts Python

Los módulos desarrollados en Python constituyen el núcleo de integración del proyecto.



Antes de iniciar la simulación se debe comprobar que:

- Las dependencias necesarias se encuentran instaladas.
- Los parámetros de conexión MQTT son correctos.
- Los parámetros de conexión PostgreSQL son correctos.
- Los directorios utilizados por los scripts existen.
- Los programas pueden iniciarse sin errores.

La correcta configuración de estos módulos resulta fundamental para garantizar la sincronización entre todos los elementos del sistema.

Verificación global del sistema

Una vez configurados todos los componentes, se recomienda realizar una comprobación completa de la arquitectura antes de iniciar la producción.

Las verificaciones mínimas recomendadas son:

Comprobación	Resultado esperado
PostgreSQL operativo	Conexión correcta.
Base de datos accesible	Consultas ejecutadas correctamente.
Broker MQTT disponible	Intercambio de mensajes funcional.
MQTTX conectado	Visualización de eventos MQTT.
ESP32-S3 conectada	Recepción y envío de mensajes.
RoboDK operativo	Simulación accesible.
Scripts Python activos	Sin errores de ejecución.
Sensores y actuadores funcionales	Respuesta correcta ante eventos.

Una vez completadas estas comprobaciones, el sistema se encuentra preparado para iniciar la ejecución de la estación automatizada y realizar las pruebas de funcionamiento correspondientes.

12.1.8 Puesta en marcha y ejecución del sistema

Una vez completada la instalación y configuración de todos los componentes, el sistema puede ponerse en funcionamiento siguiendo una secuencia determinada. El orden de ejecución es importante, ya que algunos módulos dependen de que otros componentes se encuentren previamente disponibles.

La arquitectura desarrollada integra RoboDK, PostgreSQL, Python, MQTT y la ESP32-S3, por lo que es necesario verificar que todos los elementos se encuentran operativos antes de iniciar la simulación.

Secuencia de arranque

El procedimiento recomendado para iniciar la estación automatizada es el siguiente:

1. Iniciar el servicio PostgreSQL.
2. Abrir pgAdmin y comprobar que la base de datos se encuentra disponible.
3. Iniciar MQTTX si se desea monitorizar los mensajes MQTT intercambiados durante la ejecución.
4. Conectar la ESP32-S3 mediante alimentación USB.
5. Verificar que la ESP32-S3 se conecta correctamente a la red WiFi.
6. Verificar que la ESP32-S3 establece conexión con el broker MQTT.
7. Abrir RoboDK.
8. Cargar la estación automatizada desarrollada para el proyecto.
9. Comprobar que todos los elementos de la simulación aparecen correctamente cargados.
10. Ejecutar los scripts Python necesarios para la integración del sistema.
11. Ejecutar el programa principal de la estación.
12. Iniciar la simulación.

Una vez completados estos pasos, el sistema quedará preparado para comenzar el proceso automatizado de clasificación, transporte y serigrafiado de palillos.

Verificación inicial

Antes de iniciar una ejecución completa se recomienda comprobar el correcto funcionamiento de los principales elementos del sistema.

Las verificaciones recomendadas son las siguientes:



Elemento	Verificación
PostgreSQL	Acceso correcto a la base de datos.
MQTT	Recepción y envío de mensajes.
ESP32-S3	Conexión WiFi y MQTT establecida.
LCD	Visualización correcta de mensajes.
LEDs RGB	Cambio de estado según eventos recibidos.
Sensor ultrasónico	Detección de cambios de distancia.
Sensor LDR	Lectura de niveles de iluminación.
Pulsador	Generación de eventos de parada y reanudación.
RoboDK	Simulación operativa.

Funcionamiento durante la ejecución

Durante el funcionamiento normal de la estación, los distintos componentes intercambian información continuamente mediante MQTT.

Los sensores conectados a la ESP32-S3 generan eventos que son enviados al resto del sistema, mientras que RoboDK ejecuta las operaciones de clasificación, transporte y serigrafiado. Paralelamente, los scripts desarrollados en Python coordinan la comunicación entre los diferentes módulos y registran la información relevante en la base de datos PostgreSQL.

El usuario puede supervisar el estado general de la estación mediante la pantalla LCD, los LEDs RGB y la monitorización de mensajes MQTT utilizando MQTTX.

Finalización de la ejecución

Una vez finalizada la simulación, se recomienda cerrar los distintos componentes de forma ordenada para evitar procesos bloqueados o conexiones abiertas.

En primer lugar, se debe detener la ejecución de los programas principales y verificar que no quedan tareas activas. Posteriormente pueden cerrarse RoboDK, MQTTX y el resto de las herramientas utilizadas durante la ejecución.

Este procedimiento garantiza que la estación finaliza correctamente y queda preparada para futuras ejecuciones.

12.1.9 Uso de la aplicación desarrollada

Una vez iniciada la estación automatizada, el sistema comienza a ejecutar el proceso completo de clasificación, transporte y serigrafiado de palillos de forma automática. Todos los elementos de la arquitectura trabajan de forma coordinada mediante RoboDK, Python, PostgreSQL, MQTT y la ESP32-S3.

Durante la ejecución, el usuario puede supervisar el estado general de la estación mediante los distintos dispositivos físicos y herramientas software integradas en el proyecto.

Proceso general de funcionamiento

El proceso comienza con la generación y transporte de los palillos a través de la línea de producción simulada en RoboDK.

A medida que los palillos avanzan por la estación, los diferentes robots ejecutan las tareas asignadas:

- Clasificación de piezas.
- Manipulación de objetos.
- Transporte entre estaciones.
- Serigrafiado de los palillos válidos.
- Gestión de piezas defectuosas.

Toda la información generada durante el proceso es intercambiada mediante MQTT y almacenada en PostgreSQL cuando corresponde.

Supervisión mediante la pantalla LCD

La pantalla LCD conectada a la ESP32-S3 permite mostrar información relevante para el operario.

Entre los mensajes que pueden visualizarse se encuentran:

- Estado general de la estación.
- Mensajes procedentes de RoboDK.
- Información relacionada con el proceso de producción.
- Texto que será grabado durante el proceso de serigrafía.
- Avisos generados por distintos eventos del sistema.



La información se actualiza dinámicamente durante la ejecución mediante mensajes MQTT recibidos por la ESP32-S3.

Indicadores LED RGB

La estación dispone de dos LEDs RGB utilizados para representar visualmente el estado de las cintas transportadoras.

Los estados implementados son:

Color	Significado
Verde	Funcionamiento normal de la cinta.
Rojo	Cinta detenida o inactiva.

Esta señalización permite identificar rápidamente el estado de funcionamiento de la estación sin necesidad de consultar otras herramientas de supervisión.

Sensor ultrasónico

El sensor ultrasónico HC-SR04 supervisa el nivel de llenado del contenedor destinado a almacenar los palillos defectuosos.

Durante el funcionamiento del sistema se generan los siguientes estados:

Estado	Descripción
VACIO	El contenedor dispone de capacidad suficiente.
LLENO	El contenedor ha alcanzado el nivel máximo definido.

Estos eventos son enviados mediante MQTT al resto de componentes del sistema.

Sensor LDR

El sensor LDR monitoriza continuamente el nivel de iluminación ambiental.

Cuando los valores medidos superan o descienden de los umbrales configurados, se generan los siguientes eventos:

Evento	Descripción
MUCHA_LUZ	Nivel de iluminación superior al umbral establecido.
POCA_LUZ	Nivel de iluminación inferior al umbral establecido.

Estos eventos permiten simular sistemas de supervisión ambiental habituales en instalaciones industriales.

Pulsador de control

El pulsador conectado a la ESP32-S3 permite interactuar directamente con la estación automatizada.

Su función consiste en detener temporalmente la ejecución del sistema o reanudarla cuando sea necesario.

Los eventos generados son:

Evento	Descripción
PARAR	Solicita la detención de la estación.
INICIAR	Permite reanudar la ejecución del sistema.

Estos eventos son enviados mediante MQTT y procesados por los módulos software correspondientes.

Monitorización mediante MQTTX

Durante la ejecución es posible utilizar MQTTX para visualizar los mensajes intercambiados entre los distintos componentes del sistema.

Esta herramienta permite:

- Supervisar eventos MQTT en tiempo real.
- Verificar el funcionamiento de sensores y actuadores.
- Comprobar la sincronización entre RoboDK y la ESP32-S3.
- Facilitar tareas de depuración y validación.

Registro de información en PostgreSQL

Los distintos módulos software desarrollados en Python almacenan información relevante en la base de datos PostgreSQL.

Entre los datos registrados se encuentran:

- Información de producción.
- Resultados de clasificación.

Este registro permite mantener la trazabilidad de la información generada durante la ejecución de la estación automatizada.

En conjunto, la aplicación desarrollada permite simular una estación industrial completa integrando robótica, comunicaciones IoT, bases de datos y sistemas de supervisión en una única plataforma coordinada.

12.1.10 Reinicio y parada del sistema

Debido a la arquitectura distribuida utilizada en el proyecto, formada por múltiples scripts Python ejecutándose simultáneamente, módulos MQTT, RoboDK y la ESP32-S3, no se recomienda finalizar la ejecución cerrando directamente las aplicaciones o deteniendo RoboDK de forma manual.

Para garantizar una finalización segura del sistema se han desarrollado varios programas auxiliares que permiten detener correctamente todos los procesos activos y restaurar el estado inicial de la estación.

Procedimiento de parada

Cuando se desea finalizar la ejecución de la estación automatizada, se debe seguir el siguiente procedimiento:

1. Ejecutar el programa:

PararProgramas.py

2. Esperar a que todos los módulos activos reciban la orden de parada.
3. Verificar que los scripts Python abandonan sus bucles de ejecución y finalizan correctamente.
4. Comprobar que no quedan procesos del proyecto ejecutándose en segundo plano.

Este procedimiento permite detener de forma controlada todos los módulos de integración y evita que queden conexiones MQTT o tareas activas de ejecuciones anteriores.



Programa PararProgramas.py

El script **PararProgramas.py** ha sido desarrollado específicamente para finalizar de forma segura la ejecución de todos los programas que trabajan de manera concurrente dentro del sistema.

Su función principal consiste en modificar las variables de control utilizadas por los distintos módulos, permitiendo que estos abandonen los bucles de ejecución infinita implementados para la monitorización de eventos y comunicaciones.

Sin la ejecución previa de este programa, algunos procesos podrían permanecer activos incluso después de cerrar RoboDK o los scripts principales, provocando comportamientos inesperados en futuras ejecuciones.

Procedimiento de reinicio

Una vez detenidos correctamente todos los programas mediante **PararProgramas.py**, se puede proceder al reinicio completo de la estación.

Para ello se debe ejecutar:

ResetGeneral.py

Este programa restaura el estado inicial de la simulación y reinicia las variables necesarias para permitir una nueva ejecución del sistema.

Entre las operaciones realizadas se incluyen:

- Reinicio de variables globales.
- Restauración del estado inicial de la estación.
- Reinicio de contadores y estructuras de control.
- Preparación del entorno para una nueva simulación.

Programa ResetGeneral.py

El script **ResetGeneral.py** permite devolver la estación automatizada a las condiciones iniciales definidas para el proyecto.

Su ejecución garantiza que la siguiente simulación comenzará desde un estado conocido y libre de información residual procedente de pruebas anteriores.

Se recomienda ejecutar siempre este programa después de **PararProgramas.py** y antes de iniciar una nueva simulación.

Inicio de una nueva ejecución



Una vez completado el proceso de parada y reinicio, la estación puede volver a ponerse en funcionamiento ejecutando nuevamente el programa principal del proyecto:

ProgramaPrincipal.py

La secuencia completa recomendada es la siguiente:

ProgramaPrincipal.py



Funcionamiento de la estación



PararProgramas.py



ResetGeneral.py



ProgramaPrincipal.py

Recomendaciones de uso

Para garantizar la estabilidad del sistema se recomienda:

- No cerrar RoboDK mientras existan scripts activos.
- Ejecutar siempre PararProgramas.py antes de realizar un reinicio.
- Ejecutar ResetGeneral.py antes de comenzar una nueva simulación.
- Verificar que las conexiones MQTT han sido cerradas correctamente.
- Comprobar que no existen procesos Python del proyecto ejecutándose en segundo plano.

El seguimiento de este procedimiento garantiza una operación estable del sistema y evita problemas derivados de la ejecución simultánea de múltiples módulos software.

12.1.11 Resolución de incidencias frecuentes

Durante la instalación, configuración y ejecución del sistema pueden aparecer incidencias relacionadas con la comunicación entre componentes, la ejecución de los programas o la configuración del entorno. En este apartado se describen los problemas más habituales detectados durante el desarrollo del proyecto y las acciones recomendadas para su resolución.

Problemas de conexión WiFi de la ESP32-S3



Problema	Posible causa	Solución
La ESP32-S3 no se conecta a la red WiFi	Credenciales incorrectas	Verificar SSID y contraseña configurados en el archivo de configuración.
Conexiones intermitentes	Cobertura insuficiente o problemas de red	Comprobar la estabilidad de la red WiFi y la distancia al punto de acceso.
Reinicios continuos de la placa	Alimentación insuficiente	Verificar la alimentación mediante USB.

Problemas de comunicación MQTT

Problema	Posible causa	Solución
No se reciben mensajes MQTT	Broker no disponible	Verificar la dirección del broker y la conexión a Internet.
Los eventos no llegan a RoboDK	Error en los scripts Python	Comprobar que los programas de integración están en ejecución.
MQTTX no recibe mensajes	Suscripción incorrecta	Verificar los topics configurados.
Desconexiones MQTT frecuentes	Problemas de red	Comprobar la conectividad entre dispositivos.

Problemas relacionados con RoboDK

Problema	Posible causa	Solución
La estación no se abre correctamente	Archivo dañado o ruta incorrecta	Verificar la ubicación del proyecto.
Los robots no se mueven	Programa detenido o error de sincronización	Revisar los scripts de control.

Problema	Posible causa	Solución
La simulación queda bloqueada	Procesos activos de ejecuciones anteriores	Ejecutar PararProgramas.py y ResetGeneral.py.

Problemas relacionados con PostgreSQL

Problema	Posible causa	Solución
No es posible conectar con la base de datos	Servicio PostgreSQL detenido	Iniciar PostgreSQL desde el sistema operativo.
Error de autenticación	Usuario o contraseña incorrectos	Revisar las credenciales configuradas.
No se almacenan registros	Error en los scripts Python	Verificar la conexión entre Python y PostgreSQL.

Problemas relacionados con la pantalla LCD

Problema	Posible causa	Solución
La pantalla no muestra información	Error de conexión I2C	Revisar conexiones SDA, SCL, VCC y GND.
Caracteres incorrectos	Configuración errónea de la pantalla	Reiniciar la ESP32-S3 y verificar la dirección I2C.

Problemas relacionados con sensores y actuadores

Problema	Posible causa	Solución
El sensor ultrasónico no detecta correctamente	Obstrucción o error de cableado	Revisar conexiones y posición del sensor.
El sensor LDR no responde	Error en el divisor resistivo	Comprobar la resistencia de 10 kΩ y el cableado.

Problema	Posible causa	Solución
Los LEDs RGB no cambian de estado	Error en GPIO o mensajes MQTT	Revisar conexiones y recepción de eventos.
El pulsador no genera eventos	Error de configuración o conexión	Verificar el GPIO asignado y la configuración del firmware.

Procedimiento recomendado ante errores

Cuando se detecte un comportamiento anómalo durante la ejecución del sistema se recomienda seguir el siguiente procedimiento:

1. Verificar las conexiones físicas de la ESP32-S3.
2. Comprobar la conectividad WiFi.
3. Verificar la comunicación MQTT mediante MQTTX.
4. Comprobar que PostgreSQL se encuentra operativo.
5. Revisar la ejecución de los scripts Python.
6. Ejecutar PararProgramas.py para finalizar todos los procesos activos.
7. Ejecutar ResetGeneral.py para restaurar el estado inicial.
8. Reiniciar nuevamente el sistema mediante ProgramaPrincipal.py.

La aplicación de este procedimiento permite resolver la mayoría de las incidencias detectadas durante las pruebas realizadas en el proyecto.

12.2 Manual del programador

El presente manual describe la estructura interna del software desarrollado para el proyecto, así como la organización de los distintos módulos que forman parte de la solución de automatización implementada.

Su objetivo es facilitar la comprensión del código fuente y proporcionar la información necesaria para realizar tareas de mantenimiento, ampliación o modificación del sistema.

La arquitectura software desarrollada se compone de varios subsistemas independientes que colaboran entre sí mediante comunicación MQTT y acceso a bases de datos PostgreSQL.

Los principales componentes son:



- Simulación robótica desarrollada en RoboDK.
- Scripts de integración desarrollados en Python.
- Firmware ejecutado en la ESP32-S3.
- Base de datos PostgreSQL.
- Sistema de comunicaciones MQTT.

Cada uno de estos elementos se encuentra dividido en módulos especializados que permiten implementar las diferentes funcionalidades de la estación automatizada.

12.2.1 Arquitectura general del software

La arquitectura software del proyecto sigue un modelo distribuido basado en eventos, donde los distintos componentes intercambian información mediante MQTT.

La simulación desarrollada en RoboDK representa el entorno industrial automatizado y contiene la lógica de funcionamiento de los robots, cintas transportadoras y estaciones de trabajo.

Los scripts desarrollados en Python actúan como capa de integración entre RoboDK, la base de datos PostgreSQL y los dispositivos IoT conectados mediante MQTT.

La ESP32-S3 constituye la interfaz física del sistema, encargándose de la lectura de sensores, control de actuadores y generación de eventos.

Finalmente, PostgreSQL almacena la información generada durante la ejecución de la estación automatizada.

Las responsabilidades principales de cada componente son las siguientes:

Componente	Función principal
RoboDK	Simulación de la estación y ejecución de los procesos automatizados.
Python	Coordinación del sistema e integración entre componentes.
MQTT	Comunicación basada en eventos.
ESP32-S3	Gestión de sensores y actuadores físicos.
PostgreSQL	Almacenamiento persistente de información.

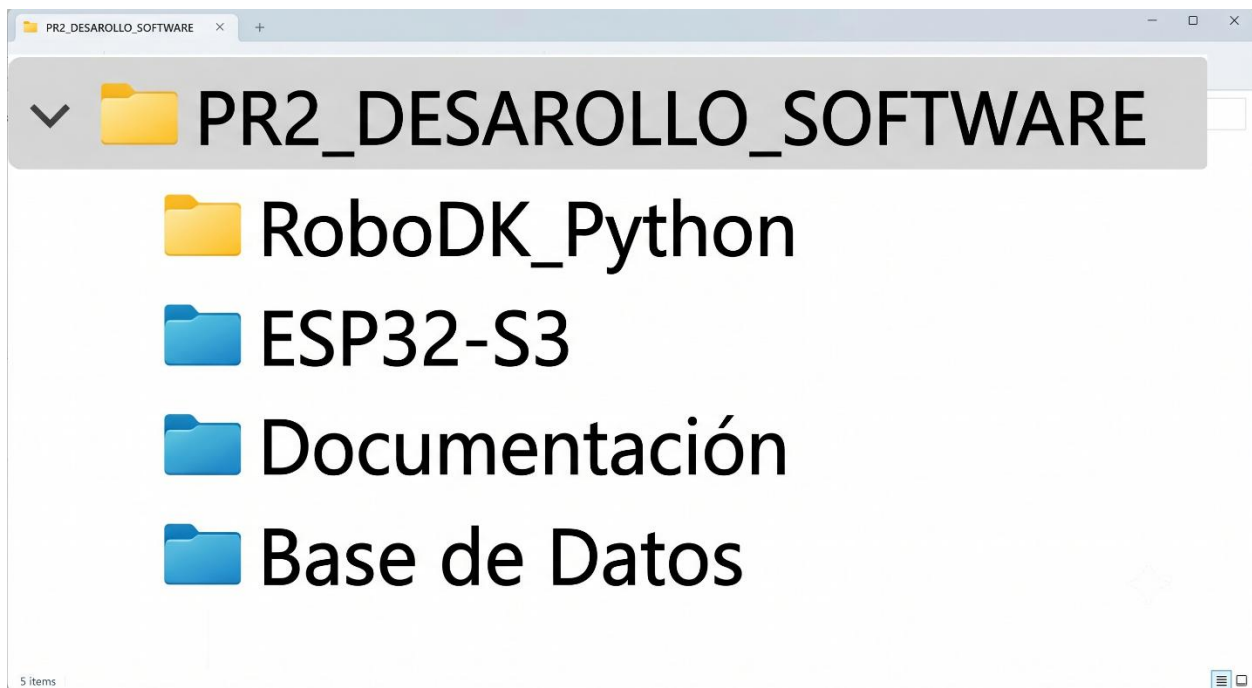
Esta separación funcional permite desacoplar los distintos módulos del sistema, facilitando tanto su mantenimiento como su futura ampliación.

12.2.2 Estructura de directorios del proyecto

El código fuente y la documentación desarrollados durante el proyecto se encuentran organizados en diferentes bloques funcionales dentro del repositorio GitHub. Esta organización permite separar claramente los distintos componentes de la solución y facilita las tareas de mantenimiento y ampliación del sistema.

La estructura general del proyecto es la siguiente:

Figura 12.2 1



Cada bloque agrupa los archivos relacionados con una funcionalidad específica de la estación automatizada.

RoboDK _Python

Este bloque contiene la mayor parte de la lógica de control del proyecto.

Incluye:

- Programas de RoboDK.
- Scripts de control de robots.
- Gestión de cintas transportadoras.

- Integración MQTT.
- Acceso a la base de datos PostgreSQL.
- Gestión de eventos.
- Coordinación entre estaciones.
- Programas de inicialización y reinicio.
- Integración de base de datos con python

La comunicación entre los distintos módulos se realiza mediante MQTT y acceso a PostgreSQL.

ESP32-S3

Esta carpeta contiene el firmware desarrollado para la ESP32-S3.

Incluye:

- Programa principal de la ESP32-S3.
- Definición de pines.
- Configuración de red WiFi.
- Configuración MQTT.
- Gestión de sensores.
- Gestión de actuadores.
- Tareas FreeRTOS.
- Control de la pantalla LCD.

Este bloque constituye la interfaz física de la estación automatizada.

Base de datos

Los archivos relacionados con la base de datos se encuentran integrados dentro de los módulos Python y complementados mediante scripts SQL específicos.

Incluyen:

- Scripts de creación de la base de datos.
- Scripts de inserción de datos de prueba.
- Consultas SQL utilizadas durante el desarrollo.

- Funciones de acceso a PostgreSQL implementadas en Python.

La estructura relacional completa de la base de datos se describe en el apartado correspondiente del presente manual.

Documentación

Este bloque contiene toda la documentación generada durante el proyecto.

Incluye:

- Memoria técnica.
- Diagramas de arquitectura.
- Diagramas de integración.
- Diagramas de base de datos.
- Esquemas eléctricos.
- Capturas de pantalla.
- Material utilizado durante la Feria de Proyectos.

La documentación permite comprender tanto el funcionamiento técnico como la organización general de la solución desarrollada.

Esta estructura modular facilita la localización de componentes específicos y permite que futuros desarrolladores puedan ampliar o modificar el sistema sin necesidad de realizar cambios significativos en el resto de los módulos.

12.2.3 Estructura interna de los módulos software

La descripción funcional detallada de los módulos desarrollados se encuentra recogida en el apartado 9.3 de esta memoria. En dicho apartado se describen los programas implementados en RoboDK, los módulos de integración MQTT y los componentes de acceso a la base de datos PostgreSQL.

El presente apartado complementa dicha información desde el punto de vista del mantenimiento y evolución del software, identificando los módulos principales y las áreas del sistema que deben modificarse para incorporar nuevas funcionalidades.

Organización de los módulos

Los programas desarrollados pueden clasificarse en tres grandes grupos:



Grupo de módulos	Función
Simulación y control robótico	Gestión de robots, cintas transportadoras y procesos de producción dentro de RoboDK.
Integración y comunicaciones	Intercambio de información mediante MQTT y coordinación entre componentes.
Persistencia de datos	Acceso y almacenamiento de información en PostgreSQL.

Relación entre módulos

El funcionamiento general del sistema sigue la siguiente secuencia:

ESP32-S3



MQTT



MqttListener.py



RobotController.py



RoboDK



ConexionBaseDatos.py



PostgreSQL

Los eventos generados por los sensores conectados a la ESP32-S3 son enviados mediante MQTT. Posteriormente, los módulos Python reciben dichos eventos, actualizan el estado de la simulación y registran la información necesaria en la base de datos.

Módulos principales del sistema



Módulo	Función principal	Modificaciones habituales
ProgramaPrincipal.py	Coordinación global de la estación.	Incorporación de nuevos procesos de producción.
Delta.py	Clasificación de palillos.	Modificación de criterios de clasificación.
MainConveyor.py	Gestión del transporte de piezas.	Ajuste de velocidades y flujo de producción.
StickConveyorMove.py	Alimentación de palillos.	Cambios en la generación de piezas.
SerigrafiaUR3e1.py	Control del primer robot UR3e.	Modificación de trayectorias.
SerigrafiaUR3e2.py	Control del segundo robot UR3e.	Modificación de trayectorias.
MqttListener.py	Recepción de eventos MQTT.	Incorporación de nuevos mensajes o topics.
RobotController.py	Procesamiento de eventos.	Modificación de la lógica de control.
ConexionBaseDatos.py	Acceso a PostgreSQL.	Nuevas consultas o tablas.
ResetGeneral.py	Reinicio de la estación.	Adaptación a nuevos módulos.

Procedimiento general para ampliar el sistema

Cuando se desea incorporar una nueva funcionalidad al proyecto, se recomienda seguir la siguiente secuencia:

1. Definir los nuevos eventos MQTT necesarios.
2. Actualizar el firmware de la ESP32-S3 si intervienen nuevos sensores o actuadores.
3. Modificar el módulo MqttListener.py para procesar los nuevos mensajes.
4. Incorporar la lógica necesaria en RobotController.py.
5. Actualizar los programas RoboDK afectados.
6. Añadir nuevas tablas o consultas en PostgreSQL si es necesario.

7. Actualizar la documentación técnica correspondiente.

Esta metodología permite mantener la modularidad del sistema y facilita la incorporación de nuevas funcionalidades sin afectar significativamente al resto de componentes.

12.2.4 Firmware de la ESP32-S3

La ESP32-S3 constituye el elemento encargado de la interacción física con la estación automatizada. Su función principal es gestionar los sensores y actuadores conectados al sistema y permitir el intercambio de información con RoboDK y los módulos Python mediante el protocolo MQTT.

El firmware ha sido desarrollado utilizando Arduino IDE y una arquitectura basada en tareas concurrentes, permitiendo la ejecución simultánea de diferentes procesos sin bloquear el funcionamiento general del sistema.

12.2.4.1 Estructura general del firmware

El programa se encuentra organizado en varios bloques funcionales independientes.

Bloque	Función
Configuración	Definición de parámetros de red, MQTT y hardware.
Inicialización	Configuración de dispositivos durante el arranque.
Comunicación MQTT	Intercambio de mensajes con el resto del sistema.
Sensores	Lectura de LDR y sensor ultrasónico.
Actuadores	Control de LCD y LEDs RGB.
Control de usuario	Gestión del pulsador de parada y reanudación.

Esta organización modular facilita la comprensión del código y permite incorporar nuevas funcionalidades sin modificar significativamente la estructura existente.

Inicialización del sistema

Durante la ejecución de la función setup() se realizan las siguientes operaciones:

- Inicialización de la comunicación serie.
- Configuración de los pines de entrada y salida.
- Inicialización de la pantalla LCD I2C.



- Creación de las tareas concurrentes utilizadas por el sistema.

Una vez completada la inicialización, el firmware queda preparado para comenzar la ejecución normal.

Bucle principal

La función loop() mantiene activa la ejecución del firmware.

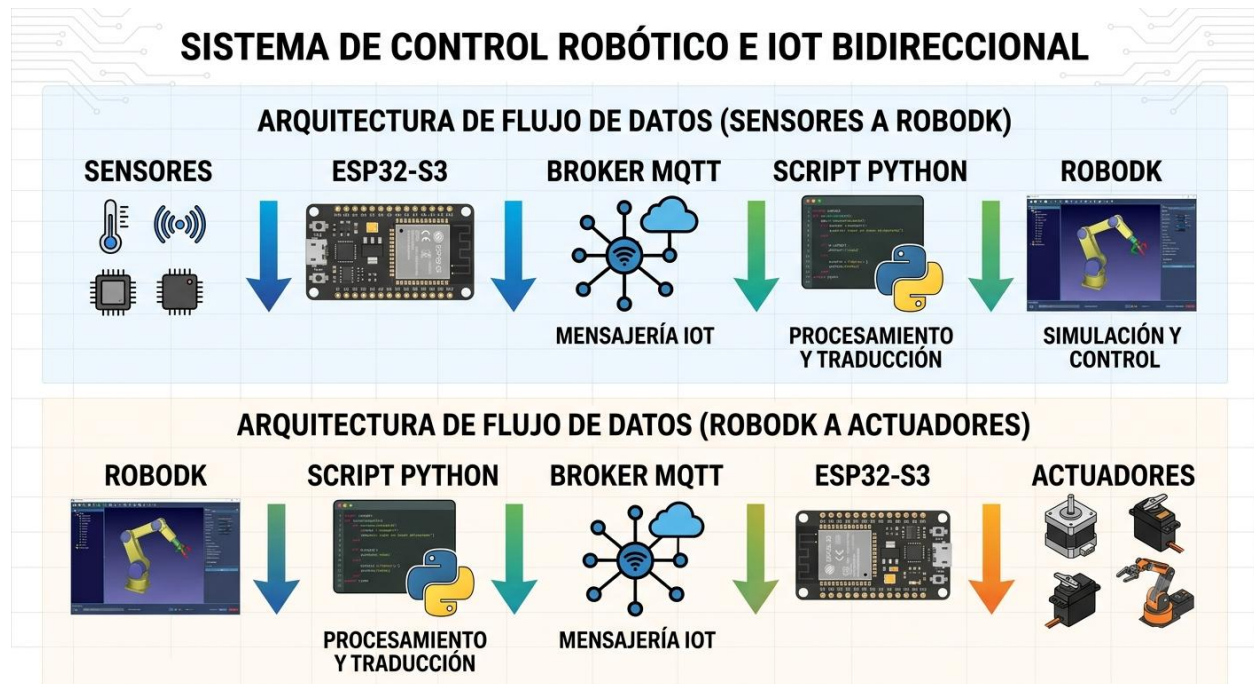
Su responsabilidad principal consiste en supervisar la conexión MQTT y garantizar que las distintas tareas continúan funcionando correctamente cuando se activa el pulsador.

La mayor parte de la lógica del sistema se ejecuta mediante tareas independientes, reduciendo la carga de trabajo del bucle principal y mejorando la capacidad de respuesta del dispositivo.

Flujo general de funcionamiento

El funcionamiento general de la ESP32-S3 puede resumirse mediante el siguiente esquema:

Figura 12.2.2



De esta forma, la placa puede tanto enviar información al sistema como recibir órdenes procedentes de la simulación para actuar sobre los distintos dispositivos conectados.

12.2.4.2 Archivo de configuración (Config.h)

Con el objetivo de facilitar el mantenimiento y la reutilización del firmware, los parámetros de configuración se encuentran centralizados en el archivo Config.h.

Este archivo contiene las constantes utilizadas por el sistema para configurar la red WiFi, la comunicación MQTT y los dispositivos hardware conectados a la ESP32-S3.

La utilización de un archivo de configuración independiente permite modificar parámetros del sistema sin necesidad de alterar la lógica principal del programa.

Configuración de red

Dentro de este archivo se almacenan los parámetros necesarios para la conexión de la ESP32-S3 a la red WiFi.

Entre ellos se encuentran:

- Nombre de la red WiFi (SSID).
- Contraseña de acceso.
- Parámetros de conexión inalámbrica.

Estos valores deben adaptarse al entorno donde se vaya a desplegar la aplicación.

Configuración MQTT

El archivo también contiene los parámetros utilizados para la comunicación MQTT.

Entre ellos destacan:

- Dirección del broker MQTT.
- Puerto de comunicación.
- Identificador del cliente MQTT.
- Topics de publicación.
- Topics de suscripción.

La modificación de estos parámetros permite conectar el sistema a diferentes brokers MQTT sin necesidad de alterar el resto del código.

Configuración de pines

Todos los dispositivos físicos utilizados en la estación automatizada se encuentran definidos mediante constantes dentro de este archivo.

La asignación utilizada es la siguiente:

Elemento	Pin ESP32-S3
LED RGB entrada (verde)	GPIO 4
LED RGB entrada (rojo)	GPIO 5
LED RGB salida (verde)	GPIO 16
LED RGB salida (rojo)	GPIO 17
Pulsador	GPIO 18
Sensor ultrasónico ECHO	GPIO 19
Sensor ultrasónico TRIGGER	GPIO 13
LCD SDA	GPIO 21
LCD SCL	GPIO 20
Sensor LDR	GPIO 10

La definición centralizada de los pines simplifica futuras modificaciones del hardware y reduce la posibilidad de errores durante el mantenimiento.

Parámetros de funcionamiento

Además de la configuración de hardware y comunicaciones, el archivo incluye diferentes parámetros utilizados por el sistema para la toma de decisiones.

Entre ellos pueden encontrarse:

- Umbrales de iluminación del sensor LDR.
- Distancia límite utilizada por el sensor ultrasónico.
- Tiempos de actualización de sensores.
- Intervalos de publicación MQTT.
- Parámetros de temporización de tareas.

Estos valores permiten ajustar el comportamiento de la estación sin necesidad de modificar la lógica de programación.

Recomendaciones de mantenimiento

Cuando se realicen modificaciones en el sistema se recomienda actualizar siempre el archivo Config.h antes de modificar otros módulos del firmware.

Las modificaciones más habituales suelen ser:

- Cambio de red WiFi.
- Cambio de broker MQTT.
- Incorporación de nuevos sensores.
- Reasignación de pines.
- Ajuste de umbrales de funcionamiento.

La centralización de estos parámetros facilita el mantenimiento del código y permite adaptar rápidamente el firmware a diferentes entornos de ejecución.

12.2.4.3 Gestión de sensores

El firmware desarrollado para la ESP32-S3 incorpora diferentes mecanismos de adquisición de datos procedentes de los sensores instalados en la estación automatizada. Estos sensores permiten supervisar determinadas condiciones de funcionamiento y generar eventos que posteriormente son enviados al resto de componentes mediante MQTT.

La arquitectura implementada separa la lectura de sensores de la lógica de control principal, facilitando el mantenimiento y permitiendo incorporar nuevos dispositivos sin afectar significativamente al resto del código.

Los sensores actualmente implementados son el sensor ultrasónico HC-SR04 y el sensor LDR.

Sensor ultrasónico HC-SR04

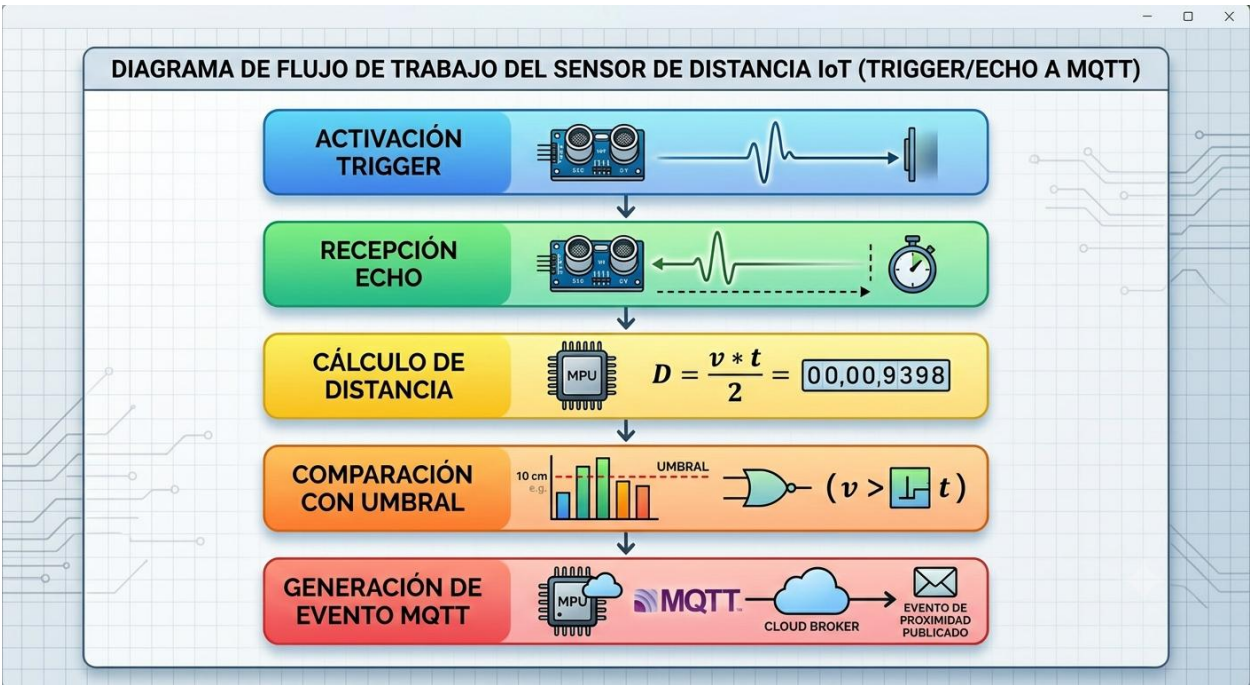
El sensor ultrasónico se utiliza para supervisar el nivel de llenado del contenedor destinado a almacenar los palillos defectuosos generados durante el proceso de clasificación.

Su funcionamiento se basa en la emisión de un pulso ultrasónico mediante el pin Trigger y la medición del tiempo necesario para recibir el eco reflejado mediante el pin Echo.

A partir de este tiempo se calcula la distancia entre el sensor y la superficie detectada.

El proceso de lectura sigue la siguiente secuencia:

Figura 12.2 3



Una vez calculada la distancia, el firmware compara el valor obtenido con los límites configurados en el sistema.

Dependiendo del resultado se generan los siguientes eventos:

Evento	Descripción
LLENO	El contenedor ha alcanzado el nivel máximo establecido.
VACIO	El contenedor dispone de capacidad suficiente.

Estos eventos son publicados mediante MQTT y utilizados por el resto de los componentes para supervisar el estado de la estación.

Sensor LDR

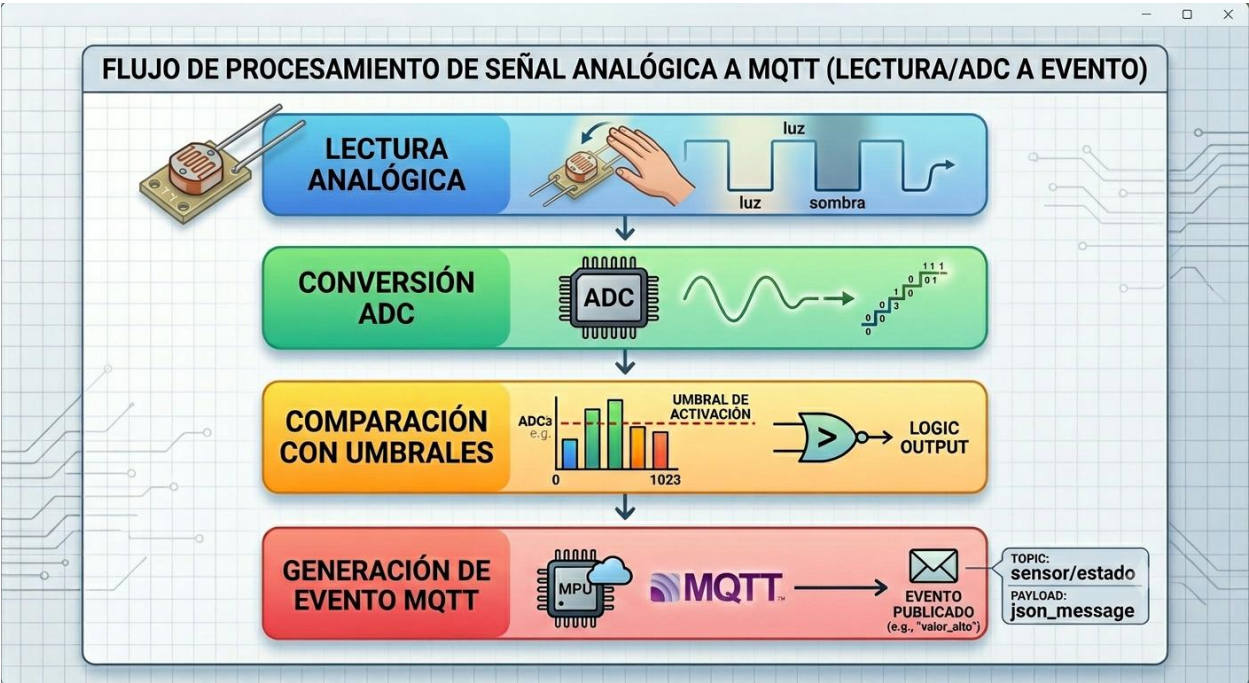
El sensor LDR permite medir el nivel de iluminación ambiental presente en el entorno de trabajo.

La lectura se realiza utilizando una entrada analógica de la ESP32-S3 conectada a un divisor de tensión formado por la LDR y una resistencia de 10 kΩ.

El valor analógico obtenido es procesado por el firmware y comparado con los umbrales definidos en la configuración del sistema.



El proceso de funcionamiento es el siguiente:



Dependiendo del nivel de iluminación detectado se generan los siguientes eventos:

Evento	Descripción
MUCHA_LUZ	Nivel de iluminación superior al umbral establecido.
POCA_LUZ	Nivel de iluminación inferior al umbral establecido.

Estos eventos permiten monitorizar las condiciones ambientales y demostrar la integración de sensores IoT dentro de la arquitectura del proyecto.

Procesamiento de eventos

Las lecturas de ambos sensores se realizan de forma periódica durante la ejecución del firmware. Para evitar la generación excesiva de mensajes MQTT, los eventos únicamente se publican cuando se detectan cambios significativos en el estado del sensor o cuando se supera alguno de los umbrales configurados.

Esta estrategia reduce el tráfico de comunicaciones y mejora la eficiencia general del sistema.

Ampliación del sistema de sensorización



La incorporación de nuevos sensores puede realizarse siguiendo la misma metodología utilizada durante el desarrollo del proyecto.

El procedimiento recomendado es el siguiente:

1. Definir el nuevo pin en el archivo Config.h.
2. Configurar la entrada correspondiente durante la inicialización.
3. Implementar la función de lectura del sensor.
4. Procesar los valores obtenidos.
5. Generar los eventos MQTT necesarios.
6. Integrar dichos eventos con los módulos Python y RoboDK.

Gracias a esta estructura modular resulta posible ampliar el sistema de adquisición de datos sin modificar significativamente el resto del firmware.

12.2.4.4 Gestión de actuadores

Además de la adquisición de datos procedentes de sensores, la ESP32-S3 es responsable del control de diversos actuadores utilizados para proporcionar información visual al usuario y representar el estado de funcionamiento de la estación automatizada.

Los actuadores implementados en el proyecto son la pantalla LCD I2C y los LEDs RGB de señalización.

Pantalla LCD I2C

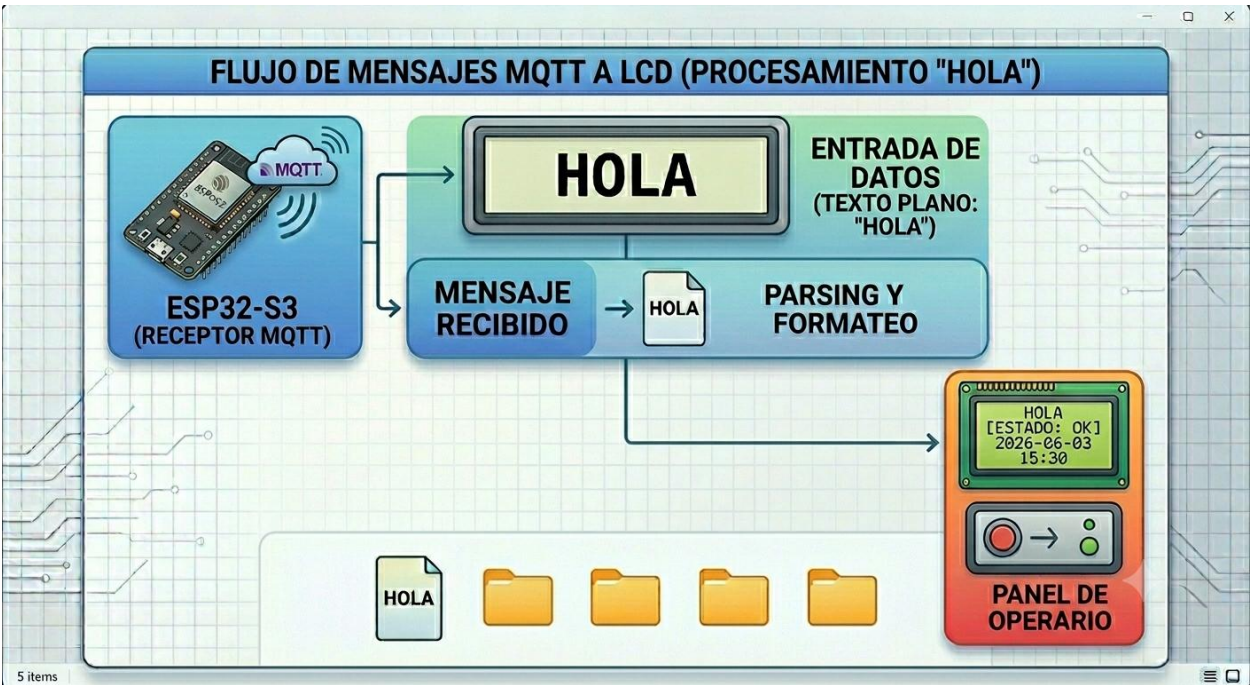
La pantalla LCD constituye el principal sistema de visualización local de la estación automatizada.

Su función consiste en mostrar información procedente de los distintos módulos del sistema, permitiendo al operario conocer el estado actual de la producción y visualizar mensajes generados por la simulación.

La comunicación se realiza mediante el protocolo I2C utilizando los pines SDA y SCL de la ESP32-S3.

El flujo de funcionamiento es el siguiente:

Figura 12.2.4



La pantalla puede mostrar distintos tipos de información:

- Estado general de la estación.
- Mensajes procedentes de RoboDK.
- Texto asociado al proceso de serigrafía.
- Avisos generados por eventos del sistema.

La actualización de la pantalla se realiza de forma dinámica conforme se reciben nuevos mensajes MQTT.

LEDs RGB de señalización

La estación dispone de dos LEDs RGB utilizados para indicar visualmente el estado de las cintas transportadoras.

Cada LED representa una zona diferente de la instalación:

LED	Elemento supervisado
LED RGB Entrada	Estado de la cinta de entrada.
LED RGB Salida	Estado de la cinta de salida.



El firmware controla los distintos canales de los LEDs mediante salidas digitales de la ESP32-S3. Los estados implementados son:

Estado	Color mostrado
Funcionamiento normal	Verde
Sistema detenido o inactivo	Rojo

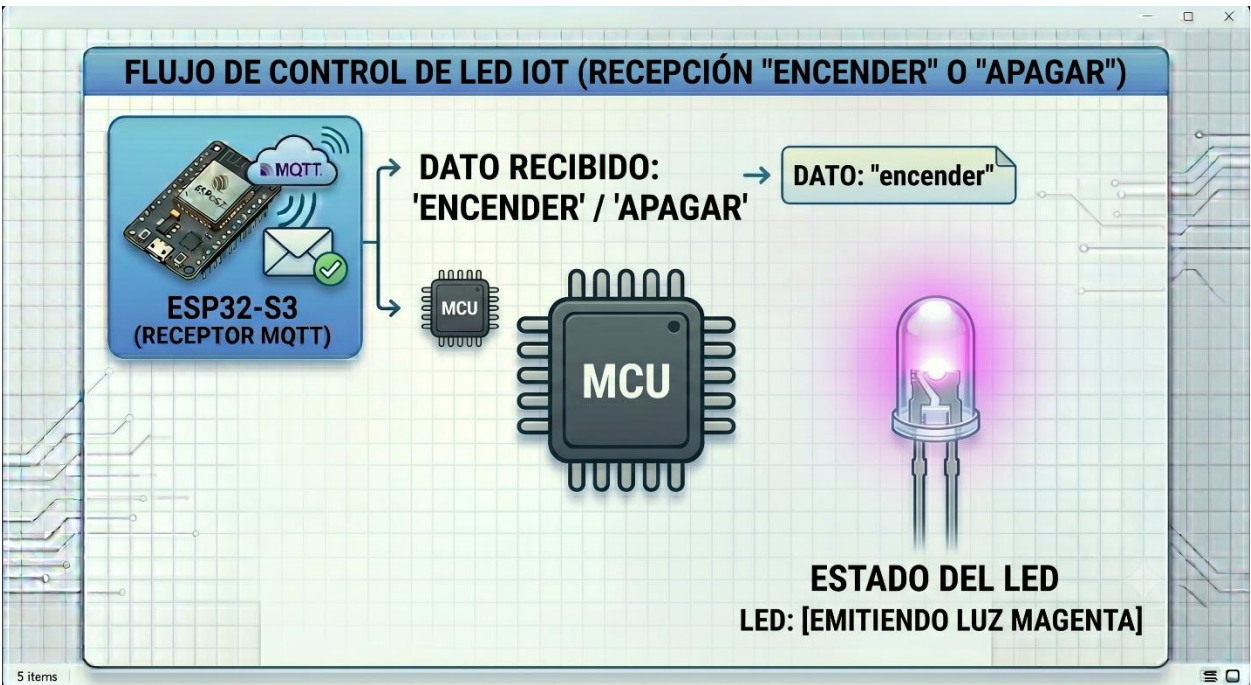
Esta señalización permite identificar rápidamente el estado operativo de la estación sin necesidad de consultar la simulación o herramientas externas.

Procesamiento de órdenes

Los actuadores no funcionan de forma autónoma, sino que responden a eventos recibidos mediante MQTT.

El proceso general de actuación es el siguiente:

Figura 12.2.5



De esta forma, cualquier cambio producido en RoboDK o en los módulos Python puede reflejarse inmediatamente sobre los dispositivos físicos conectados a la ESP32-S3.

Incorporación de nuevos actuadores

La arquitectura utilizada permite integrar nuevos dispositivos de salida de forma sencilla.

Para añadir un nuevo actuador se recomienda:

1. Definir los pines correspondientes en Config.h.
2. Inicializar el dispositivo durante la ejecución de setup().
3. Implementar la lógica de control necesaria.
4. Asociar los eventos MQTT correspondientes.
5. Actualizar la documentación del sistema.

Esta metodología permite ampliar las capacidades de interacción física manteniendo una estructura de programación organizada y fácilmente mantenible.

12.2.4.5 Gestión del pulsador

La ESP32-S3 incorpora un pulsador físico que permite interactuar directamente con la estación automatizada. Este dispositivo se utiliza para controlar la parada y reanudación del sistema mediante la generación de eventos MQTT.

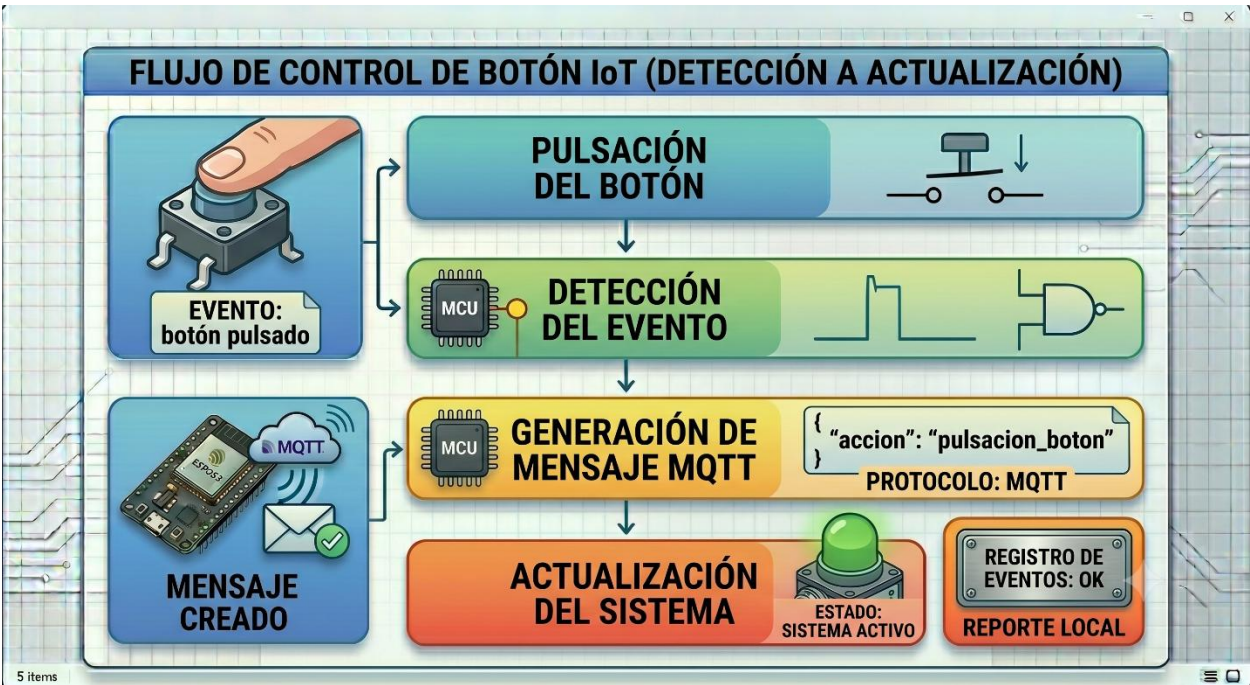
El pulsador se encuentra conectado al GPIO 18 de la ESP32-S3 y utiliza la resistencia pull-up interna del microcontrolador, evitando la necesidad de componentes externos adicionales para su funcionamiento.

Funcionamiento general

El firmware supervisa continuamente el estado del pulsador mediante la librería EventButton, encargada de detectar las transiciones de estado y generar los eventos correspondientes.

El flujo general de funcionamiento es el siguiente:

Figura 12.2 6



Gracias a este mecanismo, la interacción del usuario puede propagarse de forma inmediata al resto de componentes de la arquitectura.

Eventos generados

El pulsador puede generar dos eventos principales:

Evento	Descripción
INICIAR	Solicita la puesta en marcha o reanudación de la estación.
PARAR	Solicita la detención temporal de la estación.

Estos eventos son publicados mediante MQTT y posteriormente procesados por los módulos Python y la simulación desarrollada en RoboDK.

Integración con el sistema

La utilización de MQTT permite desacoplar completamente la lógica del pulsador del resto de componentes.

Como consecuencia, cualquier modificación en la forma de gestionar las órdenes de parada o reanudación puede realizarse en los módulos de control sin necesidad de modificar el firmware de adquisición.

Esta arquitectura facilita el mantenimiento y la escalabilidad del sistema.

Posibles ampliaciones

La implementación actual puede ampliarse fácilmente para incorporar nuevas funcionalidades asociadas al pulsador.

Algunos ejemplos son:

- Reinicio completo de la estación.
- Activación de modos de mantenimiento.
- Confirmación de alarmas.
- Ejecución de secuencias manuales.
- Selección de modos de funcionamiento.

La utilización de una librería especializada para la gestión de botones simplifica la incorporación de estas funcionalidades y mejora la robustez del sistema frente a pulsaciones accidentales o rebotes eléctricos.

12.2.5 Base de datos PostgreSQL

La base de datos PostgreSQL constituye el sistema de almacenamiento persistente utilizado por la estación automatizada. Su función es registrar la información generada durante la ejecución del sistema, permitiendo almacenar datos relacionados con la producción, el transporte, la clasificación de piezas y los procesos de serigrafía.

La estructura ha sido diseñada siguiendo un modelo relacional, garantizando la integridad de los datos y facilitando la realización de consultas complejas sobre la información almacenada.

Estructura general

La implementación de la base de datos se realiza mediante dos scripts SQL principales:

Archivo	Función
SQL_PALILLOS.sql	Creación del esquema y de todas las tablas del sistema.
SQL_PALILLOS_P.sql	Insertión de datos iniciales para pruebas y validación.

Estos archivos permiten reconstruir completamente la base de datos desde cero en cualquier instalación.

Esquema de la base de datos

Toda la información se encuentra agrupada dentro del esquema:

produccion_palillos

Esta organización facilita la separación de los datos del proyecto respecto a otras aplicaciones que puedan coexistir en el mismo servidor PostgreSQL.

Tablas principales

La base de datos se encuentra formada por diferentes entidades relacionadas con el proceso productivo.

Entre las tablas más importantes se encuentran:

Tabla	Función
clientes	Información de clientes.
proveedores	Información de proveedores.
compra	Registro de compras realizadas.
stock	Control de existencias disponibles.
robot	Información asociada a los robots de la instalación.
produccion	Registro de operaciones de producción.
serigrafiado	Información del proceso de marcado.
calidad	Resultados de inspección y control de calidad.
almacen	Gestión de almacenamiento de productos.
pedido	Registro de pedidos realizados.
transporte	Información relacionada con el movimiento de productos.

Estas tablas permiten representar las distintas etapas del proceso industrial desarrollado en el proyecto.

Relaciones entre tablas

La base de datos utiliza claves primarias y claves foráneas para mantener la integridad referencial entre las distintas entidades.

Este diseño permite:

- Evitar duplicidad de información.
- Garantizar la consistencia de los datos.
- Facilitar consultas complejas.
- Mantener la trazabilidad del proceso productivo.

Las relaciones implementadas reflejan el flujo lógico de información entre producción, calidad, almacenamiento y distribución.

Acceso desde Python

La comunicación entre los módulos Python y PostgreSQL se realiza mediante la librería psycopg.

Las operaciones más habituales implementadas son:

- Inserción de registros.
- Consulta de información.
- Actualización de datos.
- Recuperación de estadísticas de producción.

Estas operaciones se encuentran encapsuladas dentro de módulos específicos para facilitar el mantenimiento del sistema.

Flujo de información

El flujo general de almacenamiento de datos puede representarse mediante el siguiente esquema:

Figura 12.2 7



De esta forma, los eventos generados durante la simulación pueden persistirse y consultarse posteriormente.

Incorporación de nuevas tablas

Si en futuras versiones del proyecto fuese necesario ampliar la base de datos, se recomienda seguir el siguiente procedimiento:

1. Crear la nueva tabla dentro del esquema produccion_palillos.
2. Definir una clave primaria adecuada.
3. Establecer las relaciones necesarias mediante claves foráneas.
4. Actualizar los módulos Python encargados del acceso a datos.
5. Modificar los scripts SQL de creación e inicialización.
6. Actualizar la documentación técnica correspondiente.

Esta metodología garantiza que las futuras ampliaciones mantengan la coherencia del modelo de datos implementado.

Consideraciones de mantenimiento

Durante las tareas de mantenimiento se recomienda:

- Realizar copias de seguridad periódicas.
- Verificar la integridad de las relaciones entre tablas.
- Comprobar el correcto funcionamiento de las consultas SQL.
- Mantener actualizados los scripts de creación e inicialización.

La utilización de PostgreSQL proporciona una solución robusta y escalable para el almacenamiento de la información generada por la estación automatizada.

12.2.6 Sistema MQTT

El protocolo MQTT constituye el mecanismo principal de comunicación entre los diferentes componentes que forman la estación automatizada. Su utilización permite intercambiar información en tiempo real entre RoboDK, los módulos desarrollados en Python y la ESP32-S3, manteniendo una arquitectura distribuida basada en eventos.

La comunicación se realiza mediante un broker MQTT público, al que se conectan todos los elementos del sistema para publicar y recibir información.

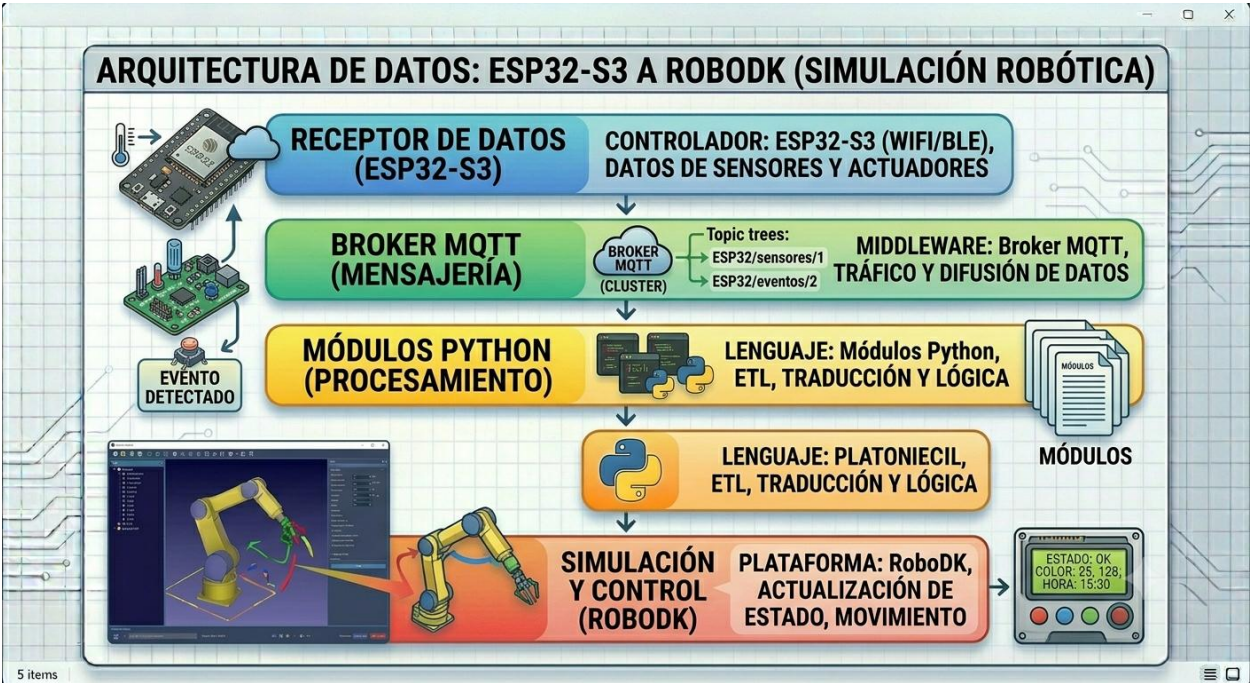
Arquitectura de comunicaciones

La arquitectura MQTT implementada sigue un modelo Publisher-Subscriber (Publicador-Suscriptor).

Los distintos componentes pueden actuar tanto como emisores como receptores de información dependiendo de la funcionalidad desarrollada.

El flujo general de comunicación es el siguiente:

Figura 12.2 8



Asimismo, RoboDK y los scripts Python pueden generar eventos que son enviados nuevamente a la ESP32-S3 para actualizar los actuadores físicos de la estación.

Componentes participantes

Los principales elementos conectados al sistema MQTT son:

Componente	Función
ESP32-S3	Publicación de eventos procedentes de sensores y recepción de órdenes para actuadores.
MqttListener.py	Recepción y procesamiento de mensajes MQTT.
RobotController.py	Interpretación de eventos y control de la simulación.
RoboDK	Generación de eventos asociados a la producción.
MQTTX	Herramienta de supervisión y depuración de mensajes.

Organización de eventos



La comunicación entre componentes se basa en eventos que representan cambios de estado dentro de la estación automatizada.

Entre los principales eventos gestionados por el sistema se encuentran:

Evento	Descripción
INICIAR	Solicitud de puesta en marcha de la estación.
PARAR	Solicitud de detención del sistema.
LLENO	Contenedor de rechazo lleno.
VACIO	Contenedor con capacidad disponible.
MUCHA_LUZ	Nivel elevado de iluminación.
POCA_LUZ	Nivel reducido de iluminación.
Actualización LCD	Mensajes enviados a la pantalla LCD.
Control LEDs	Cambios de estado de los indicadores luminosos.

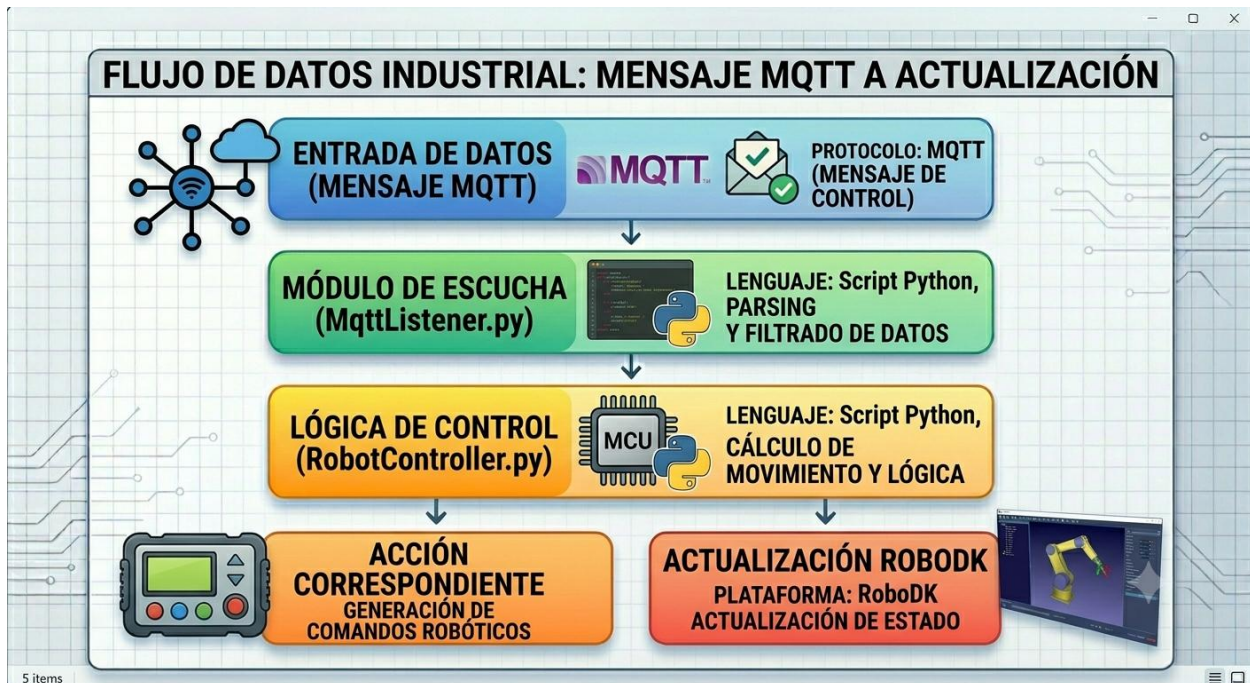
Estos eventos permiten desacoplar completamente la lógica de cada componente y facilitan la escalabilidad del sistema.

Procesamiento de mensajes

Cuando un mensaje MQTT es recibido por los módulos Python, se ejecuta una secuencia de procesamiento que permite actualizar el estado interno de la estación.

El flujo general es el siguiente:

Figura 12.2 9



Esta arquitectura permite mantener separadas las comunicaciones de la lógica de control.

Incorporación de nuevos eventos

La estructura implementada facilita la ampliación futura del sistema mediante la incorporación de nuevos eventos MQTT.

El procedimiento recomendado es el siguiente:

1. Definir el nuevo evento dentro del firmware o módulo correspondiente.
2. Publicar el mensaje MQTT asociado.
3. Actualizar el módulo MqttListener.py para reconocer el nuevo evento.
4. Implementar la lógica necesaria en RobotController.py.
5. Actualizar los módulos de RoboDK afectados.
6. Verificar el funcionamiento utilizando MQTTX.

Gracias a esta metodología resulta posible ampliar la funcionalidad del sistema sin modificar significativamente la arquitectura existente.

Herramientas de depuración

Durante el desarrollo del proyecto se ha utilizado MQTTX como herramienta principal para la monitorización de mensajes.

Esta aplicación permite:

- Visualizar mensajes en tiempo real.
- Publicar eventos manualmente.
- Verificar la recepción de mensajes.
- Comprobar la conectividad entre componentes.
- Facilitar las tareas de depuración y validación.

La utilización de MQTT como mecanismo de comunicación aporta flexibilidad, modularidad y escalabilidad a la solución desarrollada, facilitando la integración entre simulación, dispositivos IoT y sistemas software externos.